

TRADING DECODED

Artificial Intelligence Applications In Finance



AVIRATH KAKKAR
ARVIN SAHNI
RAJALINGAPPAA SHANMUGAMANI

TRADING DECODED

Artificial Intelligence Applications In Finance

AVIRATH KAKKAR
ARVIN SAHNI
RAJALINGAPPAA SHANMUGAMANI

Dedicated to our parents,
Brig (Dr.) Rajesh Kakkar (Vishisht Seva Medal)
and Mrs Shipra Kakkar
&
Mr Manjit Singh Sahni and Dr. Gurprit Kaur

And our beautiful daughter, Antara who hopefully,
will find errors in our research when she grows up.

¬ Avirath and Arvin

Dedicated to my mom Anandhi

– Raja

Copyright © 2020
All Rights Reserved

No part of this publication may be reproduced or distributed in any form or by any means without the prior written approval of the publisher.

ACKNOWLEDGEMENTS

Special thanks to Mr Ajay Agarwal Director & CEO of EIS global, without whose intellectual , financial and technological help this research would not have been possible and not half as worthwhile. Ajay and his brother Vivek founded EIS at the young age of 17 and have successfully traded the markets with a lot of success for more than two decades. We'll meet him again in the foreword.

Big thanks to Simon Sywak Director A stars Advisors for proofreading the manuscript in painstaking detail and providing extremely valuable feedback. Simon has over 25 years' experience in equity , fx and commodity markets. He recently led the Asian e-trading effort at JP Morgan.

Big thanks to Amitesh Sinha , who has been a pillar of support and provided us immense guidance along the way, always ready to discuss trading strategies. He has been the Global Head of foreign exchange exotic option business at ANZ.

Big thanks to Brandon Liu , eFX quant trader at Morgan Stanley Asia and Bank of America Merrill Lynch for providing valuable feedback on various technical aspects of the book.

And finally and importantly our dear friend Michael Annerl, for being a pillar of immense support throughout our career. Retired as the Head of commodity structuring Goldman Sachs Asia. As always he has very kindly shared his testimonial on the book.

FOREWORD

I am honoured and excited to write the foreword for ***“Trading Decoded – Artificial Intelligence applications in finance”***. I was truly impressed by Avirath at the first meeting at a conference. Since then he has always beaten my expectations with his intellect and knowledge each time we have interacted in our work together. The team of authors has extensive knowledge across various domains starting from advance engineering, technology, mathematics, statistics and their application to finance be it options, commodities trading or applying advance machine learning techniques to finance.

The readers of the book will get access to the vast experience and knowledge the team of authors has gathered over the years working in various banks and Hedge funds and technology firms in different roles and various domains of trading and AI algorithms. Special care has been taken to simplify complex mathematical, scientific and financial concepts to make them easier for the readers to absorb. The book will help readers develop concepts related to quantitative finance and give them new ideas to explore. The book will be a true treasure for all the readers and they will keep referring back to it for years.

Here's wishing them all the luck

Ajay Agarwal
Director
EIS Global Pte Ltd

AUTHORS BIO

Avirath Kakkar has spent the last 13 years in the finance industry in various capacities as a trader, structurer, strategist, entrepreneur and founder. He joined JP Morgan, Singapore in the structured commodity team to structure, price and execute exotic options and other derivative trades for clients on the Asset and Liability side. He was then responsible for Precious Metals Options trading and market-making for Standard Bank Asia. Avirath joined Balyasny Asset Management (Hong Kong) where he developed and executed systematic strategies across FX, Commodity and Equity markets. Avirath went on to establish his own trading venture. He is currently working with a family office building and executing AI based algorithms. He is a part time instructor at an online training institute where he teaches Machine Learning for trading.

Arvin Sahni has a total of 13 years of work experience in data science, financial markets and software engineering. She started her career with Oracle India as part of the E-business suite application development team. After completing her MBA, she joined Citibank Singapore's commodities team. She worked with the investor team providing structured investor solutions for commodity assets. She then moved to a sales role within the metals team covering corporates, hedge funds and other institutional clients for financial products in the precious and industrial metals space for Asia Pacific region ex Japan, Australia. With a strong career graph thus far, Arvin decided to shift her focus to her passion in Data Sciences. She enrolled in UC Berkeley's Master in Data Science course. Arvin also holds an MBA from IIM Calcutta (India) and Bachelor of Engineering (Computer Science & Engineering) degree from NIT Trichy (India). She currently works on developing algorithmic trading strategies based on machine learning

Rajalingappaa Shanmugamani is currently working at a technology company on Machine learning in London. Previously, he worked as a Senior Machine Learning Lead at SAP, Singapore and worked at various startups in developing machine learning products. He has a Master's from Indian Institute of Technology – Madras. He has authored 6 books on various applications of machine learning. He has published articles in peer-reviewed journals and conferences and applied for few patents in the area of machine

learning. In his spare time, he coaches programming and machine learning to school students and engineers.

TESTIMONIALS

“For an in-depth overview of applications of machine learning and artificial intelligence in modeling financial assets, this book is a much-have. It caters towards financial experts who have limited knowledge of these techniques as well as tech-savvy people who want to learn more about the working of financial markets and how to apply ML and AI to modelling financial assets. Throughout the book, the authors give excellent real-life examples to illustrate how to apply these techniques to predict the future. Avirath does an excellent job in warning the reader to avoid pitfalls like over-fitting AI models and how to properly measure the performance of your model. I can highly recommend this book to everyone who wants to start developing ML and AI models to apply on financial assets”

Jerome Cohen

Managing Member, Viseroij Capital Management
Former Global Head Complex FX Products

“Trading Decoded is a must read on several fronts. Firstly, for traders - whose knowledge of markets ranges from novice to expert – who have limited understanding of ML and AI techniques it clearly explains how to apply these techniques to improve their existing models and develop new insights and trading models. Models which would otherwise have been impossible to learn, test and deploy due to the massive time constraints resulting from the sheer number of permutations and combinations of variables. Moreover, Trading Decoded, improves the learning curve by providing a list of the pitfalls for each ML and AI technique saving the reader a lot of precious time and effort in deciding which techniques to employ. The real-life examples used also enable the reader to gain a greater understanding of the techniques used and how to implement them. Secondly, Trading Decoded is written for those with a strong understanding of AI and ML but who have little understanding of trading and financial markets. After reading Trading decoded, this second group of readers should be able to apply ML and AI to building trading systems, model financial time series and find important, and sometimes hidden, relationships among financial time series.”

“Trading decoded fills an important void in the trading and system development literature. It allows both traders and the technology literate to harness the power of advancement in computing power and technology to compete with the big players and not get left behind in traditional trading strategies whose alpha is being consistently eroded. Having read many trading books and invariably being left disappointed, Trading Decoded succeeds where other books fail because it gives the reader the tools to succeed.”

Michael Annerl

Managing Partner, Viseroij Capital Management
Ex - Head of Commodity Structuring Asia Goldman Sachs

PREFACE

We live in this wonderful era of knowledge-sharing and have gained a lot from the insights and experience shared by the community. This is our way of giving back and expressing our gratitude. Finance is going to use machines in a big way. That trend has already started but it is nowhere comparable to the scale at which other industries are adapting to this technology. Traders sitting at banks, funds and other financial institutions will also evolve with time.

In this book we have tried to amalgamate the technology of Artificial Intelligence with the science (as much of a science it can be) of investment management. The book starts off with a chapter introducing trading in the context of AI. Rest of the chapters explain various machine learning algorithms and concepts. Practical finance examples are used for explanation of AI concepts to help relate the two areas. The chapter on Carry trading is an exception where you can learn how to apply finance concepts for algorithmic trading. The last chapter has the biggest takeaways and an intuitive guide for everyone who is reading the book.

This book is for professional traders and investment management professionals who are looking to build their expertise in Artificial Intelligence algorithms and thereby enhance their search for alpha. This book will be equally useful for readers who are comfortable with Artificial Intelligence algorithms and are looking to build their expertise in trading and investments. We have tried to keep the explanations very intuitive and readers who are about to begin their journeys in the world of finance or in AI should find the contents helpful. Machine learning algorithms will be covered assuming no prior knowledge.

We must declare up front, we are not selling any trading system or methodology through this book. We have given python functions only where we thought their presence would aid the users understanding. Our aim has been to build concepts, highlight challenges and build a process which needs to be followed to search for alpha in the market.

Do share your comments, feedback and how would you like us to improve the book. Looking forward to hearing from you. Contact us at

tradingdecoded@gmail.com .

Avirath, Arvin, Raja

TABLE OF CONTENTS

[ACKNOWLEDGEMENTS](#)

[FOREWORD](#)

[AUTHORS BIO](#)

[TESTIMONIALS](#)

[PREFACE](#)

[CHAPTER ONE - INTRODUCTION](#)

[It is the Best of Times...](#)

[Trading](#)

[Momentum Trading: Buy High, Sell Higher](#)

[Price momentum](#)

[Momentum in related instruments](#)

[Regime variables](#)

[Market microstructure](#)

[Carry Trading](#)

[What is carry?](#)

[Drivers of carry](#)

[Hypothetical market setup](#)

[Trade 1](#)

[Trade 2](#)

[Trade 3](#)

[Summary](#)

[References](#)

[CHAPTER TWO - DATA TYPES](#)

[The General Setup](#)

[Numeric](#)

[Categorical data](#)

[One Hot Encoding](#)

[Label Encoding](#)

[Frequency Encoding](#)

[Mean Encoding](#)

[Summary](#)

[References](#)

[CHAPTER THREE - VALIDATION STRATEGIES AND PERFORMANCE METRICS](#)

[Organizing the Data](#)

[Characteristics of Validation Data](#)

[Validation Strategies](#)

[Separate validation data set](#)

[k-fold cross validation](#)

[Regression Metrics](#)

[Mean Square Error \(MSE\)](#)

[MAE](#)

[MSPE/MAPE](#)

[Classification Metrics](#)

[Confusion Matrix](#)

[Accuracy](#)

[Precision](#)

[Sensitivity/Recall](#)

[Specificity](#)

[F1 score](#)

[AUC](#)

[Summary](#)

[**CHAPTER FOUR - TREE MODELS**](#)

[Decision Trees](#)

[Construction of Decision Trees - Regression setting](#)

[Construction of Decision Trees - Classification setting](#)

[Weak learners](#)

[Summary](#)

[References](#)

[**CHAPTER FIVE - ENSEMBLE METHODS**](#)

[Early Stopping](#)

[Cost-Complexity Pruning](#)

[Bagging](#)

[Out-of-bag samples](#)

[Random Forest](#)

[Boosting](#)

[Regularization](#)

[XGBOOST package](#)

[Application](#)

[Summary](#)

[References](#)

[**CHAPTER SIX - BIAS VARIANCE TRADE-OFF**](#)

[Bias](#)

[Detecting a Biased Machine](#)

[Reducing Bias of a Machine](#)

[Variance](#)

[Detecting a High Variance Machine](#)

[How Do We Solve a High Variance Problem?](#)

[Summary](#)

[**CHAPTER SEVEN - APPLICATION: CARRY TRADING**](#)

[Carry in Foreign Exchange](#)

[Carry in Commodities](#)

[Carry in Equity Indices](#)

[Carry in Rates](#)

[Adjusting for the Roll](#)

[Assets Under Consideration](#)

[Data](#)

[Features](#)

[Target](#)

[Train/Validation/Test Data Sets](#)

[Result](#)

[Risk Management](#)

[Summary](#)

[References](#)

CHAPTER EIGHT - SUPPORT VECTOR MACHINES

[Basic Linear Algebra](#)

[Dot Product](#)

[Classification](#)

[Maximal Margin Classifier](#)

[Allowing Misclassifications](#)

[Introducing Non-Linearity](#)

[Summary](#)

[References](#)

CHAPTER NINE - DEEP LEARNING

[Neuron](#)

[Activation Functions](#)

[Sigmoid](#)

[Tanh](#)

[Relu \(Rectified Linear Unit\)](#)

[Leaky-Relu](#)

[Linear Activation](#)

[Softmax Activation](#)

[Layers](#)

[Input layer](#)

[Hidden layer](#)

[Output layer](#)

[Fully Connected Network](#)

[Epoch](#)

[Parameters and Hyper Parameters](#)

[Grid search](#)

[Random search](#)

[Coarse to fine search](#)

[Overfitting](#)

[More data](#)

[Regularization](#)

[Dropout](#)

[Early stopping](#)

[Methods to improve optimization](#)

[Scale/Normalize the inputs](#)

[Batch normalization](#)

[Parameter initialization](#)

[Mini-batch gradient descent](#)

[Gradient descent with momentum](#)

[RMSprop\(Root Mean Square Propagation\)](#)

[Adam's optimization](#)

[Loss Functions for classification and regression problems](#)

[Multiclass classification](#)
[Regression](#)
[Keras Implementation](#)
[Financial Application: dealing with unbalanced datasets](#)
[Summary](#)
[References](#)

CHAPTER TEN - CONVOLUTION NEURAL NETWORKS

[Convolution Operation](#)
[Financial Application: trading chart patterns](#)
[Padding](#)
[Strides](#)
[Multiple Filters](#)
[Multiple Channels as the Input](#)
[Pooling](#)
[Financial Application: extracting features for multi asset trading](#)
[Summary](#)

CHAPTER ELEVEN - RNN- RECURRENT NEURAL NETWORKS

[Implementing Recurrent Function in Neural Networks](#)
[Dimensions of Various Vectors](#)
[Financial Application : trading CME futures](#)
[Notations](#)
[Keras Implementation: Setting up the features](#)
[Keras Implementation : Setting up the target Y](#)
[Many to one](#)
[Many to many](#)
[Back propagation through time](#)
[LSTM](#)
[Summary](#)
[References](#)

CHAPTER TWELVE - K NEAREST NEIGHBOURS

[KNN Algorithm](#)
[Financial Application : macro timing of various equity styles](#)
[Data Set Targets](#)
[Pitfalls in the current strategy](#)
[Test Strategy performance](#)
[Optimal portfolio for a particular macro-economic environment](#)
[Optimal macro environment for a particular strategy](#)
[References](#)

CHAPTER ONE - INTRODUCTION

It is the Best of Times...

The subject of Artificial Intelligence (AI), or Machine Learning (ML), has been adequately researched since the 1960s, if not earlier, and it has recently received a lot of attention once again. While there could be several reasons for that (advances in computing, deep learning, and so on), what is most exciting for us is the disruptive nature of technology companies such as Google, Facebook, Coursera, etc.

Significant advances in technology, cutting-edge research, and discoveries daily are now readily available to anyone who cares to find out. The economy of sharing gives individuals and companies access to tools and information for research and product development, which would have only been available to a handful a few years ago.

The rising productivity and lowering barriers to entry have left many industries vulnerable to the threat of disruption. That, of course, is the opportunity; the challenge, however, is access to high-quality data.

In this chapter, we will take a look at some of the trading concepts that AI can handle.

Learning Objectives:

By the end of this chapter, you will be able to:

- Explain how AI can be used in trading to quantify trading ideas and for improving existing quant strategies.
- Identify inputs that can go into a machine trying to trade momentum.
- Hypothesize and analyze to improve the performance of a carry portfolio.

Trading

AI provides a generalized framework for quantifying our thoughts. While algorithmic trading has always used statistical tools, AI provides a set of tools that could quantify almost any trading idea. That, however, does not mean that the AI-based solution will necessarily be better or most optimal.

In our experience, AI is most suitable for problems where:

- The volume of data is huge. How huge is huge? Empirically and speaking entirely from our experience, we saw benefits of using deep learning models as we increased our sample size from tens of thousands to 20–30 million.
- The signal is weak, and the noise is high. We risk generalizing here but for most problems in trading, this is the case.
- There is a structure to the data and there is information content to be learned. There is an old English adage, “if all you have is a hammer then everything looks like a nail”. AI is a very powerful tool, but when misused, it is fully capable of generating trading signals from purely random noise. We will discuss various potential pitfalls along the way. For one, we like to focus on data sets and trading strategies, which have an economic rationale to exist. We can use AI to quantify it or to improve an existing strategy.

Smaller data sets that have a much stronger signal and a lower noise content would probably be best tackled using traditional statistical tools. The huge benefit of using traditional statistical tools is that the models are interpretable, which is of immense value in trading.

Momentum Trading: Buy High, Sell Higher

What is the economic rationale for momentum to exist? We believe it is deeply rooted in the process of price discovery. Fundamentals and news drive price action but we do not know with certainty the price level at which all fundamentals and news actions have been correctly and fully priced in. This, coupled with asymmetric information and understanding of markets among various investor types, has made it one of the best performing factors over several decades and in several asset classes.

Momentum is the first derivative of price. As a trading strategy, it is most prevalent, even today. The ambiguity lies in defining momentum, identifying the period where it is most prevalent, and identifying assets that will most likely benefit from it. As an insight, Figure 1.1 shows the astronomical rise of palladium futures in 2019 (60% in a matter of a few months). Figure 1.2 plots momentum defined as the one-day change in price. Figure 1.2 is much noisier than the seemingly linear/exponential price action in Figure 1.1. It is a challenge to fish for a signal in this noisy Figure 1.2 data set.



Figure 1. 1 is the futures price of palladium plotted against time.

Figure 1. 2 Momentum, which is defined as the day-on-day change in price.

The question is: what would we want to include to define momentum?

How should we define something as abstract as momentum?

Price momentum

Price momentum can be defined in multiple ways. Who is to say that day-on-day change in price is the best way to define momentum? How about a

week-on-week, a month-on-month, a year-on-year or for that matter any nonstandard period? Alternatively, how about using all the aforementioned and trying to fish for a signal in this extremely noisy data set? Essentially, we are implying that, since there is no known optimal way of defining momentum, we will accept all possible definitions to be legitimate, and use AI to learn a somewhat optimal definition.

Momentum in related instruments

Palladium is a relatively small market and its price action is also affected by what is happening in gold, or what is happening to risky assets (S &P futures, for instance) in general or what is happening to the broad dollar index.

If you have spent some time at the trading desk (trading macro instruments), you probably only need to know what your one or two instruments of choice have done overnight and that will give you an indication of other instruments' performance.

Therefore, to try and define momentum for palladium, we may also consider including momentum of related instruments, that is, momentum of gold, momentum of the broad dollar index, and, momentum of risky instruments (such as equity indices). So, if we originally had 100 ways of defining momentum, we now have another 300 (3 instruments × 100 definitions per instrument).

Regime variables

The point to note is that this relation to other instruments may change with time.

On some occasions, a risk-on move may be associated with dollar weakness, which may be supportive of things such as palladium. However, the same risk-on environment being driven by a search for high yield/dividend bearing instruments may be bad for assets such as gold, which do not provide any yield/dividends. In addition, palladium, being a close ally of gold, may actually suffer in performance.

Therefore, we may want to add information about sentiment or information about the regime that the market is currently in and this further adds to our data set.

Market microstructure

We may also consider adding information about the market microstructure to our definition of momentum. For example, Figure 1.3 gives a snapshot of what the level one order book looks like. Let us go through it step by step.

S. No.	Bid Qty	Bid Price	Ask Qty	Ask Price	TradedQty	TradedPrice
1	16	6989	7	6990	0	6989
2	16	6989	7	6990	5	6990
3	16	6989	2	6990	0	6990
4	16	6989	2	6990	2	6990

Figure 1. 3 A snapshot of what the limit order book looks like.

S.No. 1:

16 lots at the bid price and 7 lots at the offer price in a relatively liquid instrument. Since the bid quantity of 16 is greater than the ask quantity of 7, it is possible that the buying pressure may be more than the selling pressure. However, the evidence is weak so far.

S.No. 2:

5 lots trade at a price of 6990.

S.No. 3:

We now have 16 lots at the bid and only 2 lots left at the offer. The “momentum” is leaning on the buy side at this stage for two reasons:(1) 16 is greater than 2 and (2) a significant portion of the offer (5) was taken away.

S.No. 4:

2 lots traded at 6990 as expected, purely on momentum of the order book.

Now we include this information about the order book in our definition of momentum.

Therefore, to summarize, momentum for us is an abstract noisy signal, which we define under four broad headings:

- Momentum of the instrument we are trying to predict
- Momentum of related instruments
- Regime
- Momentum in the market microstructure

The question now is how one should combine all this information to take a decision to buy, to hold, or to sell. The data set has exploded - we have something like 500 different yet related definitions. In a scenario like this, how do we extract content to take one of the three decisions? This is where AI models come in to help us.

That is a fair bit on what can be used to define momentum trading strategies. Let us now endeavor to understand Carry Trading and see how AI can be used to enhance performance.

Carry Trading

Carry trading is a strategy that involves borrowing at a low-interest rate and investing in an asset that provides a higher rate of return. The hypothesis we are trying to establish with carry trading is whether assets with a positive carry appreciate over time. We do that in the succeeding text using a single instrument and not AI. Ideally, we would like to test this hypothesis over several instruments and several regimes. We then briefly talk about how AI could be used to improve performance of this portfolio. The aim of this exercise is twofold:

- To understand what “carry” is.
- To protect against the hammer and nail syndrome, where we use AI to improve, fine tune, and quantify.

What is carry?

When we talk about carry, we essentially mean we are paid for owning an asset. There are various ways by which one can earn carry in financial markets. Portfolios constructed with the aim of benefitting from carry could include stocks that pay a high dividend, bonds that pay high coupons, short option positions - thereby receiving premium, and short positions in futures with a steep upward slope (contango), long position in futures with a steep downward slope (backwardation).

These strategies have attracted a fair bit of capital over the last several years driven by low benchmark rates and investors' desire to earn a higher yield. The strategies have done quite well too; however, that is not to suggest that the future performance is also expected to be as good.

Drivers of carry

Let us look at the risks and profile of one of these strategies. It will give us some insights into what we can do to enhance the performance of these strategies. We will analyze a short position in VX-Cboe Volatility Index (VIX) futures, which for the most part has been in a contango (upward sloping futures curve).

We will begin by clearly defining two types of returns that futures trading exposes us to:

- Spot return
- Roll return

It is best understood through a toy example.

Hypothetical market setup

Let us say we have a hypothetical commodity contract trading on a particular exchange with monthly listings and priced at \$100 for the nearby active contract. Let us also assume the volatility of this contract is zero on all days, essentially implying the price does not change with time. On the expiry of a particular month, however, the next month always starts trading \$2 higher. Figure 1.4 shows what the 3-month price history would look like.

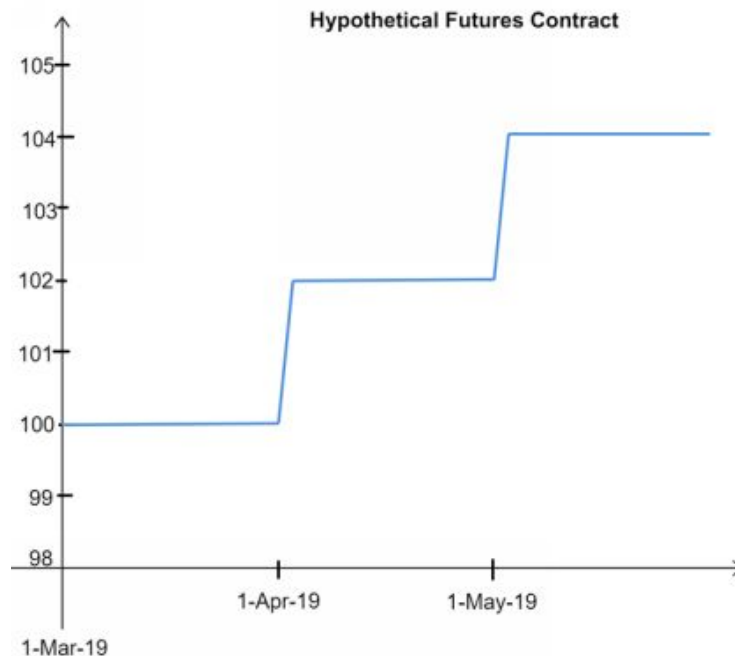


Figure 1. 4 Plot showing price history of our toy example, which starts trading at \$100, and stays there for all of March, jumps to \$102 in April and stays there for all of April, and finally jumps to \$104 in May.

Trade 1

On March 1, 2019, we took a long position in one lot of the hypothetical contract, which was trading at \$100. The price remained \$100 until expiry on March 31, 2019, which is when we sold it.

Trade 2

On April 1, 2019, we took a long position in one lot of the hypothetical contract, which was trading at \$102. This contract remained at \$102 for a

month until expiry on April 30, 2019, when we sold it.

Trade 3

Finally, we went long on one lot of the contract on May 1, 2019 at \$104, which we closed on May 31, 2019 at \$104.

Now, what is our PnL (Profit and Loss) on all these trades?

When the commodity increased from \$100 to \$104, we made \$4 in “spot returns”. We bought the contract on March 1, 2019 at \$100 and sold it for \$104 on May 31, 2019, which is the **spot return** of \$4. However, we did a few more trades.

On March 31, 2019, we sold the March contract for \$100 and on April 1, 2019, we bought the commodity for \$102, incurring a loss of \$2.

On April 30, 2019, we sold the April contract for \$102 and on May 1, 2019, we bought the May contract for \$104, incurring another loss of \$2.

The total loss on these inter-period trades is \$4, which we call the “**roll return**,” which exactly offsets what we made as “spot return.” The total PnL on this trade is zero.

Just to be clear, we are incorrectly calling the change in price as return here, but we will just move on. Finally, to summarize this hypothetical toy example, we could redraw Figure 1.4 as Figure 1.5, where we have adjusted the price at each stage by the cumulative roll up until that point.

TIP ADDITIONAL MATERIAL

Backwardation: A futures curve is backwardated when the deferred contract is trading at a discount to the spot contract.

Contango: A futures curve is in contango when the deferred contract is trading at a premium to the spot contract.

The roll return is positive for a long position in backward dated markets.

The roll return is negative for a short position in contangoed markets

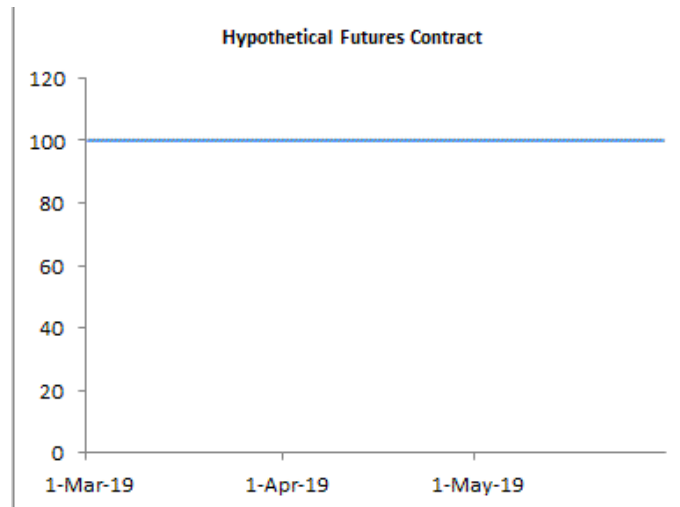


Figure 1. 5 Alternate representation of the price history of the futures contract.

Now, let us move back to the real world. Figure 1.6 shows what the cumulative “spot return” is for a long position in the front month VIX futures.

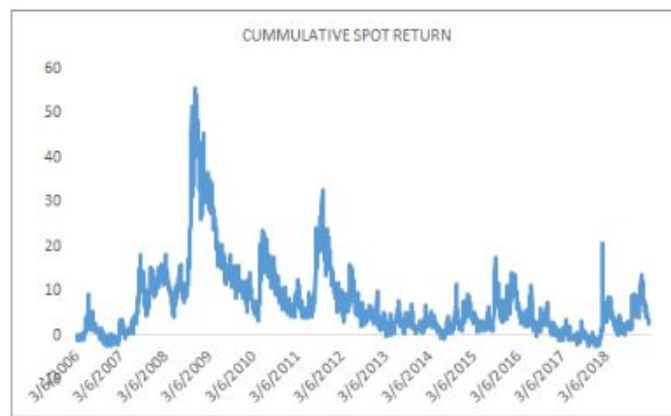


Figure 1. 6 Cumulative spot return for a long position in the front month VIX futures

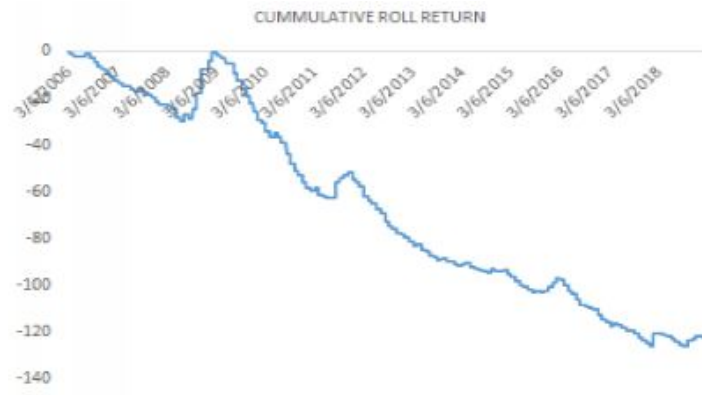


Figure 1. 7 Cumulative roll return for long position in VIX futures

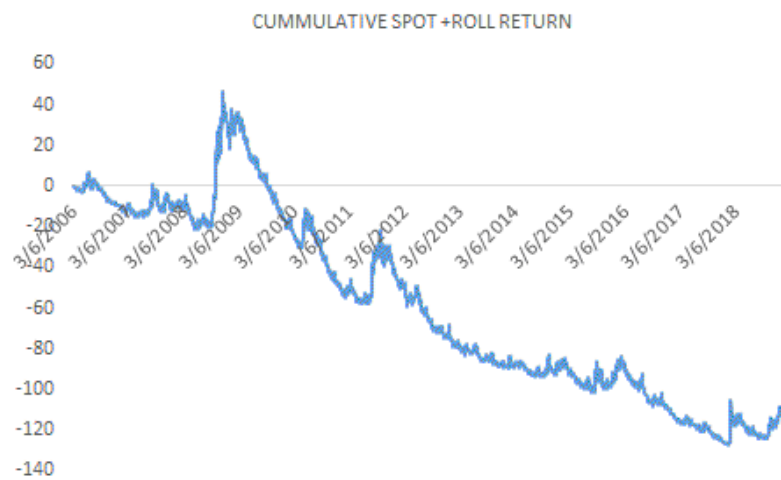


Figure 1. 8 Cumulative spot and roll returns for a long position in VIX futures

Figure 1.9 shows the cumulative PnL for a short position.



Figure 1. 9 Cumulative spot and roll return for a short position in VIX futures

A few things to take away from this analysis are:

1. Roll return is a significant driver of performance.
2. During periods of exceptional volatility (2008 great recession and 2011 European debt crisis), spot returns tend to dominate and pull down the performance.
3. Roll return changes direction during periods of stress.

We now have clues on what our machine needs to do. We can stay short the VIX front month futures and rely on our machine to give us a signal to cover our short. The machine needs to focus on finding:

1. When the roll return is about to change direction
2. When the spot returns are about to explode

Hence, a good start could be:

1. To provide historical roll returns as an input to be able to predict when the roll returns may change direction.
2. To provide inputs about an impending economic slowdown, a debt crisis, or a war that may be able to predict when spot returns are about to explode.

Summary

AI provides a set of very powerful tools for quantifying our thoughts. In trading, AI can be used to quantify our trading ideas and for improving existing quant strategies. In this chapter, we discussed what inputs can go into a machine trying to trade momentum. We also established a hypothesis that positive carry instruments tend to appreciate over time but are subjected to drawdowns. The analysis gave us hints on how we can improve the performance of a carry portfolio.

References

- John C hull Options, Futures, and Other Derivatives, for basic definitions of futures and options.
- Lemperiere, Yves & Deremble, Cyril & Seager, Philip & Potters, Marc & Bouchaud, Jean-Philippe (2014). Two centuries of trend following. The Journal of Investment Strategies. 3. 10.21314/JOIS.2014.043.
- Baz, Jamil and Granger, Nicolas and Harvey, Campbell R. and Le Roux, Nicolas and Rattray, Sandy, Dissecting Investment Strategies in the Cross Section and Time Series (December 4, 2015). Available at SSRN: <https://ssrn.com/abstract=2695101> or <http://dx.doi.org/10.2139/ssrn.2695101>
- Jusselin, Paul and Lezmi, Edmond and Malongo, Hassan and Masselin, Côme and Roncalli, Thierry and Dao, Tung-Lam, Understanding the Momentum Risk Premium: An In-Depth Journey Through Trend-Following Strategies (September 24, 2017). Available at SSRN: <https://ssrn.com/abstract=3042173> or <http://dx.doi.org/10.2139/ssrn.3042173>

CHAPTER TWO - DATA TYPES

In this chapter, we will discuss various types of data sets as well as data types we may come across in finance. We will also look at the best way to present them to a machine to extract maximum intelligence. We will discuss a general framework that needs to be applied to all artificial intelligence (AI) tasks.

Learning Objectives

1. Readers will be introduced to numeric, categorical and date/time features
2. Readers will be presented with various data pre-processing alternatives

The General Setup

Imagine a typical spread sheet with a whole lot of data presented in rows and columns. For the task of applying AI algorithms to finance, we present the data in a tabular form that would look very much like Figure 2.1. The rows of the data set are called samples, observations, or instances. The columns are called the features or the predictors.

seq

target

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

ES08simu

Figure 2. 1 What a typical tabular data set looks like

The samples are observations recorded for the purpose of discovering a pattern or an empirical relationship. The observation frequency could be anything - monthly, daily, or even every tick. The features or the columns are attributes of the data that we think will help us uncover the pattern. These features could be of various different data types.

- **Numeric:** Typically, price returns, fundamental data such as earning releases, inventory data for commodities, and so on. Typically, these are set to be of data type float or integer in any programming language.
- **Categorical/Ordered categorical:** These are data types that represent a finite set of discrete items. For instance, “January” represents the first 31 days of a year; “Week 1” represents the first 7 days of a month, and “better than expectations” represents all those companies whose

earnings beat expectations, and so on. How do we present this information to a machine in a meaningful way? We will discuss that in the next section.

- **Date time:** There could be meaningful information in the date and time of the day. There are seasonal patterns not only in the calendar months, but also in the week of the month, and the day of the week. Even the session of the day that may have some predictive power. Below are a few examples that we see in the market.

1. Consistent outperformance of platinum and palladium futures in January and February (Figure 2.2).
2. Higher price volatility of various instruments in the data intensive first two weeks of the month.
3. Intraday seasonality pattern in volumes traded for various futures contracts (Figure 2.2).



Figure 2. 2 A screenshot from Bloomberg showing the expected positive returns in the month of January and February ("Source: Bloomberg Finance L.P. Used with permission of Bloomberg Finance L.P.")



Figure 2. 3 Intraday seasonality in volume for e-mini S&P 500 futures as represented by the bimodal red colored lines in the second panel ("Source: Bloomberg Finance L.P. Used with permission of Bloomberg Finance L.P.")

- **Images:** Satellite data to study the prospective yield of agricultural commodities. We have not researched or traded this concept and therefore it will not be covered this in this book.

Before diving into each of these, it is worth highlighting that the way you present the data to the machine sometimes depends on the kind of machine that you intend to deploy. From this perspective, there are two types of machines we encounter. Models such as decision trees (see Chapter 4) rely on putting the samples into various rectangular regions of the feature space. These models are scale independent wherein changing the scale of the inputs will not have a meaningful impact on the solution. In our experience, it sometimes adversely affects performance.

Additionally, there are models such as logistic regression and artificial neural networks where the scale of the features has a very significant impact on the solution.

Let us discuss an example to understand the intuition behind scaling of features. Let us say we have only two features: x_1 and x_2 . A decision tree solution would typically be interpreted as follows:

1. All samples with $x_1 < a$ and $x_2 > b$ belong to the same compartment and have a predicted value of .35. Here a and b are the parameters

which the algorithm has solved for. All samples meeting this condition are in region 1.

2. All the other samples have a predicted value of .75. All samples meeting this condition are in region 2.

Changing the scale of x_1 by say a factor of 3, and x_2 by a factor of 2 is only going to change the values of a and b by similar scales. The samples that belong to region 1 and region 2 will not change. In this case, the solution does not change as we are interested in the regions that a specific sample belongs to. In particular, we are interested in the samples that are grouped together.

A regression (scale-dependent model), on the other hand, has a solution such as

$$Y = F(a \cdot x_1 + b \cdot x_2)$$

The a and b values here are again the output of the model. If the scale of x_1 and x_2 changes, the values of a and b change and the output Y changes.

Let us now start our discussion on how to handle various data types.

Numeric

As a general guideline for scale-independent models such as decision trees, numeric features can just be used in their given format. In scale-dependent models, it is best to pre-process them. Scikit Learn's Min Max scaler or standard scaler could be used. The Min Max scaler ensures all feature values are within a predefined range.

The simple way to do scaling in Scikit Learn is shown in the box below after loading the data.

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler()  
scaler.fit(TRAINDATA)  
TRAINDATA= scaler.transform(TRAINDATA)
```

The Standard Scaler, on the other hand, ensures the mean of all scaled feature values is zero and the standard deviation is 1. All it does is to subtract the mean from the feature and divide the result by the standard deviation of the result. These scaling techniques can have a huge impact on optimization techniques such as gradient descent used in some scale-dependent models, like Artificial Neural Networks. The simple way to implement this in Scikit Learn is as follows.

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
scaler.fit(TRAINDATA)  
TRAINDATA= scaler.transform(TRAINDATA)
```

Categorical data

These are features that essentially represent certain categories, like months of the year or days of the week, for example. These are extremely useful and powerful set of features that, when pre-processed correctly, can add a lot of value to the machine.

If we are interested in capturing seasonality effects on a financial instrument, we need to start by giving the month of the year as a feature. Some days of the week are more volatile than the rest. For instance, on Wednesdays, oil inventory data is being published, Thursday's WASDAE reports on crops, and US employment report on the first Friday of every month. These days are known to be more directional/volatile days than the rest. Needless to say, the day of the week may therefore add value to the machine. Most of the US economic data is published in the first two weeks of a month, which may set the tone for the rest of the month. It may therefore make sense to use it as a feature of whether we are in the first half of the month or the second half of the month.

If the categories of the categorical data represent some form of an order or ranking, then the categorical data sets are also known as ordinal data sets.

Let us say we are tracking by how much the markets expectation of rate cuts has deviated from what the median projection of the FED (US Federal Reserve) is. We may categorize the difference in expectations as High, Inline, and Low, where:

High—implies that markets are way more hawkish than the FED

Inline—implies that markets have the same view as the FED

Low—implies that markets are way more dovish than the FED

Here High, Inline, and Low imply some form of a rank or an order to the data they represent.

The question then remains, how do we present this data to the machine?

One Hot Encoding

Let us say our data set looks like Figure 2.4, where we have name of the month as one of the features.

Serial No	Average return over period of 10 days	Month	Average return over next 10 days
1	-2%	DEC	-3.00%
2	-2.50%	NOV	-3.00%
3	3%	JAN	-2.00%
4	1%	FEB	0.75%
5	2%	JAN	1.50%
6	1.70%	MAR	2.30%
7	-8%	DEC	-2.00%
8	7%	JAN	-2.20%

Figure 2. 4 Example of dataset containing categorical data.

A one hot encoding transformation would look like Figure 2.5:

Serial No	Av. return over prev 10 days	Month	Av. Return over next 10 days	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
1	-2%	DEC	-3.00%	0	0	0	0	0	0	0	0	0	0	0	1
2	-2.5%	NOV	-3.00%	0	0	0	0	0	0	0	0	0	0	1	0
3	3%	JAN	-2.00%	1	0	0	0	0	0	0	0	0	0	0	0
4	1%	FEB	0.75%	0	1	0	0	0	0	0	0	0	0	0	0
5	2%	JAN	1.50%	1	0	0	0	0	0	0	0	0	0	0	0
6	1.70%	MAR	2.30%	0	0	1	0	0	0	0	0	0	0	0	0
7	-8%	DEC	-2.00%	0	0	0	0	0	0	0	0	0	0	0	1
8	7%	JAN	-2.20	1	0	0	0	0	0	0	0	0	0	0	0

Figure 2. 5 ONE HOT representation of data in Figure 2.4

In Figure 2.5 the “Month” feature has been replaced by 12 features. For any one sample (row), only one of these 12 is assigned a value of one. The others are all set to zero. This is a rather simple representation. As a downside, it is a very sparse representation with a whole lot of redundant zeros.

```
# data is a two-dimensional array of shape [n_samples,n_features]  
from sklearn import preprocessing  
enc = preprocessing.OneHotEncoder()  
enc.fit(data)  
ohe = enc.transform(data).toarray()
```

Label Encoding

Continuing with data in Figure 2.4 we could simply replace the Month feature by a label as shown in the following table.

Serial No	Av. Return over prev 10 days	Month	Label
1	-2%	DEC	12
2	-2.50%	NOV	11
3	3%	JAN	1
4	1%	FEB	2
5	2%	JAN	1
6	1.70%	MAR	3
7	-8%	DEC	12
8	7%	JAN	1

Figure 2. 6 label encoding.

This would work fine for decision trees but would be inappropriate for Artificial Neural Networks.

```
#data is a two-dimensional array of shape [n_samples,n_features]  
from sklearn import preprocessing  
enc = preprocessing.LabelEncoder()  
enc.fit(data)  
le = enc.transform(data).toarray()
```

Frequency Encoding

In each sample, the categorical feature is replaced by its frequency of occurrence, for example in Figure 2.4, there are eight samples, DEC (December) appears in two of them and NOV (November) appears in only

one. We will then replace DEC by its frequency of occurrence which is $2/8 = 25\%$ and NOV by 12.5% . The simple code to do it is:

```
#df contains the data frame of Figure 2  
df.groupby("Month").size()/df.shape[0]
```

We need to be careful of using this method in this form if we have time series data. If we calculate the frequency encodings from the entire data set, then we would have looked into the future. The first sample has DEC as the categorical feature value. Replacing this with 25% essentially tells us that there is only one sample in the future that has a value of DEC for this feature. This practice is not acceptable for data sets that have time series in it. The correct approach is to have an "ENCODING" data set from which we derive the frequency encodings. We then use frequency embeddings derived from the "ENCODING" data set in all data sets constructed from subsequent time steps. This thinking will become more apparent in subsequent chapters.

Mean Encoding

Continuing with the data of Figure 2.4, we could replace each categorical value by the mean value of some other column. Let us say we were setting up a machine to predict the "Average return over the next 10 days," which would be our target variable. We could replace the "DEC" values of the categorical feature with the average value of the target when the categorical feature is "DEC." In this instance, we have two samples that have "DEC" as a categorical feature and the target or the "Average return over the next 10 days" has been -3% and -2% , averaging -2.5% .

```
#df contains the data frame from Figure 2  
mean_encode = df.groupby("Month")["Target"].mean()
```

We will encounter numeric and categorical data in all data sets in finance. Scaling numeric features using a minmax scaler or a standard scaler can be extremely helpful in making our optimizers converge when we are dealing with machines like Neural networks or support vector machines. On the other hand, frequency encoding and mean encoding are simple techniques, however, they are immensely beneficial modifications to data that help

almost any machine to learn better. As a matter of fact, it may be beneficial to convert numeric data to categorical data (see Chapter 4) and then use mean encoding or frequency encoding to present it to a machine.

Summary

1. We have introduced various data types we encounter in finance and various pre - processing that are necessary to perform on the data set.
2. The type of pre-processing we choose may also depend on the machine learning algorithm that we intend to deploy and the kind of information we intend to convey to the machine.

References

1. <https://www.fast.ai/2018/04/29/categorical-embeddings/>
2. Pairs Trading Strategy with Geolocation Data — The Battle between Under Armour and Nike Jim Kyung-Soo Liew, Tamas Budavari, Zixiao Kang, Fengxu Li, Xuzhi Wang, Shihao Ma and Brandon Fremin. The Journal of Financial Data Science Winter 2020, 2 (1) 126-143; DOI: <https://doi.org/10.3905/jfds.2019.1.024>

CHAPTER THREE - VALIDATION STRATEGIES AND PERFORMANCE METRICS

In the previous chapter, we have discussed ways to deal with various types of data that you may come across in finance. We have also discussed various preprocessing techniques.

In this chapter, we introduce a broad classification of various Artificial Intelligence (AI) algorithms. We then discuss a general framework on how to organize the data for appropriate learning. We also highlight some pitfalls that need to be avoided - some obvious, some not so obvious.

Broadly speaking, there are two types of learning algorithms: supervised and unsupervised.

As the name suggests, supervised learning has a “supervisor,” which is the target or the dependent variable that is to be predicted. All decisions that we make to design the machine are taken with the sole purpose of being able to accurately predict the target. All tools and tricks that we present and have presented will be useful in trying to improve the quality of this prediction. Needless to say, whether this target prediction meets a business objective is outside the scope of the prediction algorithm. Therefore, you may have a machine that has a great accuracy because the machine has been designed to predict just that. Whether that makes money, or whether that makes enough money to meet costs, or whether it has the right risk profile, is outside the scope of what the machine is expected to do.

It may seem obvious, but it is important that we reflect on it before we jump in and invest precious time in designing the algorithm. Therefore, we may be designing an algorithm to predict the probability of prices breaking a trend line. The business objective, however, is to make money. Does the prediction help do that?

At the other end of the spectrum, we have unsupervised learning. As the name suggests, in this case we do not have a supervisor or a target. We

present data to the machine and expect the machine to return some intelligence. For instance, the machine could cluster data into similar subgroups. Dimension reduction algorithms come under this as well. Our further discussions shall not be focusing on this.

Organizing the Data

Some of the algorithms we present are very powerful. Give them a dataset and they are very capable of learning everything and producing almost a zero-error prediction. The problem is that the zero-error prediction is limited only to the data that the machine has been trained on, known as the training data. If we use this machine on unseen data such as incoming market data, the prediction error skyrockets, and you quite easily infer that the machine has zero intelligence. We call this second type of data, the test dataset.

Therefore, we want a machine that works well on data that it has seen (training data) and does reasonably well on test data as well. However, does that mean we keep iterating through various possibilities such that we eventually find a machine that works well on the training data and the test data? The answer is “no” because if we do infinite iterations on the training data to get good results on the training and the test data, we have effectively trained the machine on both datasets, thereby merging the two.

We therefore need another data set, known as the validation dataset. We now do this back and forth iteration between the training and validation data, and once we have a satisfactory model doing well on training and validation data, we bring in the test dataset only once to see performance.

Characteristics of Validation Data

- It should be very similar in quality and distribution to the test data we are expected to perform well on.
- Zero leakage between training data and validation data, between validation data and test data.
- The samples should be IID (Independent and Identically Distributed).

A useful question that we like to answer while designing the validation data set is: If the validation data were our test data, would we still get the same results? And would we be able to realistically implement the exact same process of validation? Would all the information of the validation dataset be available when required so as to make necessary predictions for the validation dataset?

These seemingly theoretical considerations are extremely important and can be the reason behind why an algorithm works and why it does not. Before we discuss these considerations in more detail, let us discuss which validation methods are generally adopted.

Validation Strategies

Separate validation data set

We divide the entire universe of data into three unequal parts, namely train, validation, and test. The training data generally is 60–75% of the entire set; the balance is split equally between test and validation data. The more important consideration is the validation data, which should be similar in distribution and characteristics to test data. This method of validation is simple and robust. However, there are two limitations.

- **Expensive:** The training is forced to happen on a smaller data set, as about 20% is being kept aside for validation.
- **Regime-specific model selection:** Let us say you are designing a strategy to trade gold futures based on end of day data. The previous 20 years would give us about 5000 samples, 3000 of which we keep for training, 750 for validation, and 750 for test. Generally speaking, 3000 samples will never be enough for most AI algorithms and definitely not enough for deep learning algorithms. Furthermore, we are forced to keep 3 years of samples aside for validation. More importantly, if our validation data included 2009, 2010, 2011 and the test data included 2012, 2013, 2014, then the validation period for most part was a quiet upward trend (except for a short period in 2011) and the test data were primarily a downward trend. The problem with this kind of a setup is that the validation data is inherently different in characteristics to the test data, thereby models expected to perform well on the validation data may actually perform poorly on the test data.

k-fold cross validation helps us address both these concerns.

k-fold cross validation

1. We divide the universe of data into train and test. The validation data is derived from the training data in the following manner: We RANDOMLY divide the training data into $K(=3)$ equal parts, and call them A, B, and C. Then we perform three operations.

- Train the model on datasets A and B and validate on dataset C.
- Train the model on datasets A and C and validate on dataset B.
- Train the model on datasets B and C and validate on dataset A.

K can assume any value like 3, 5, and 10. This is how most software packages would implement it. K -fold cross validation datasets can be created with few lines of code.

```
from sklearn.model_selection import KFold  
kf = KFold(n_splits=2)  
kf.get_n_splits(X)
```

This splits the original data into different subsets.

It may be obvious to the reader that this process of validation does address the two limitations discussed in the previous section. For most financial data sets however, this process is wrong and fails at multiple levels.

We have done things in the aforementioned process on the validation data, which we would not be able to do if the validation data were to be treated as our test data. We have also exposed ourself to allowing leakages to occur between the training and validation data.

Let us take a simple example to get the intuition.

In our very small training data of 2 months, spanning January 1, 2018 to February 28, 2018, let us say we are trying to predict what the market did 5 days later. The target is set to “1” if it went higher, “-1” if it went lower, else it is set to “0”. As features, all that we are providing is the daily market return for the previous 5 days.

1. By RANDOMLY assigning samples from our training into $k = 3$ folds, we may very often be using data from the future to predict the past. That is something we obviously cannot do if our validation data were to be treated like our test data, or data that we are to see in the future.

Therefore, we want to avoid RANDOM selection of samples into each fold.

2. Independence of data splits: To understand this better, let us look at the following example:

FOLD	TRAIN DATA										
A	1-Jan-18	2-Jan-18	3-Jan-18	4-Jan-18	5-Jan-18	8-Jan-18	9-Jan-18	10-Jan-18	11-Jan-18	12-Jan-18	15-Jan-18
	VALIDATION DATA.....										
B	16-Jan-18	17-Jan-18	18-Jan-18	19-Jan-18	22-Jan-18						

Figure 3. 1 Example of training and validation dates, which violate the independent part of the IID criteria.

A visual representation of our training and validation dates for a particular fold is shown in Figure 3.1 for calendar year 2018. The target for January 15 (last day of our training data) is known only on January 22. At this point, we need to ask ourselves the ephemeral question: Can we use training data up to January 15 and predict the validation data on January 16? The answer is an obvious 'no' because on January 16, we do not know the targets for samples from January 9 to January 15. The information between January 9 and January 15 cannot be used for learning to predict an outcome on January 16. This definitely needs to be accounted for when including January 16 in our validation data. However, as stated earlier while designing our validation data set, we should look at it as a test data set. If it can serve as a test data set, it can serve as a validation data set.

So why is January 9 to January 15 not suitable for training if January 16 is included in our validation data? What we need to appreciate is that if there is a very strong relationship between the targets from January 9 to January 15 on one hand and January 16 on the other, our machine will focus and pick up this relationship. We may be impressed by the validation accuracy, but the test data accuracy shall be poor. Importantly, if there is some actual intelligence in the training data, which could have resulted in discovering meaningful alpha for a test data set, we have obstructed that process.

Very often, the machine will pick up the easier relations faster and first. If these easier relations are wrong, we may not be able to reach a stage where

we actually allow the machine to focus on less obvious but valid relationships, which actually do exist.

A visualization for a recommended $k = 3$ -fold cross-validation strategy for time series data is shown in Figure 3.2, where we have assumed train, validation, and test splits of the data set have been made, taking into account the aforementioned two concerns of nonrandom selection of samples and independence of training and validation data sets.

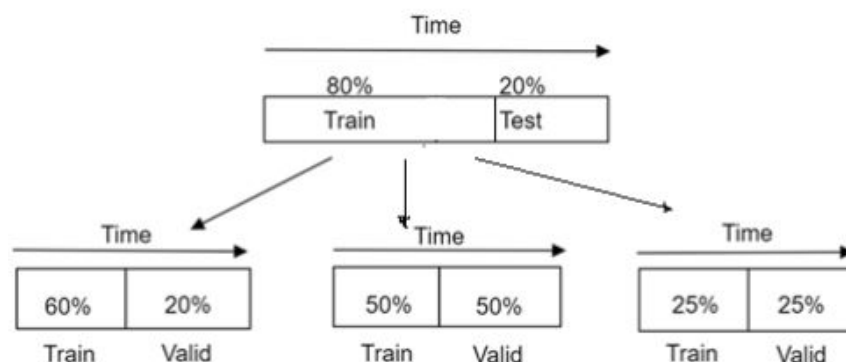


Figure 3. 2 A plausible threefold cross-validation strategy for time series data.

We have discussed in fair bit of detail how to create a validation dataset and what are the pitfalls we need to avoid. Next we discuss various metrics for measuring performance.

Regression Metrics

We discussed various factors that could serve as inputs to a momentum model in Chapter 1. Based on these inputs we may be interested in predicting the returns for the next day, the next week or the next month. Essentially, we are interested in predicting a numeric outcome. The following are the metrics used to evaluate a regression machines performance

Mean Square Error (MSE)

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y}_i)^2$$

MSE is the sum of the squared difference between the actual and the predicted values over all the samples.

Where N are the total number of samples, y_i is the target of the i^{th} sample, and $\bar{y}_i$ is the prediction for the i^{th} sample. Some observations that will help us understand the usage of this metric are as follows:

Calculating the MSE requires squaring the error of the prediction, as a result the MSE tends to be influenced more by samples where the predictions are significantly different from their actual values. Do we really need that for our problem?

What therefore is a good MSE value? Here are a few guidelines:

- Replace each prediction by the mean value of all targets and calculate the MSE; our machine should have an MSE lower than this.
- In trying to predict the returns for the next day we can use today's actual return as a proxy and calculate a benchmark MSE value. The machine we eventually deploy should have an MSE value lower than this benchmark. In doing so we are necessarily ensuring that today's price is not the best predictor of tomorrow's price.

- Replace the prediction with the average actual value for previous several days and calculate a benchmark MSE value. Again, we want our machine to have an MSE lower than this benchmark.

MAE

The Mean Absolute Error (MAE) is defined as the absolute value of the difference between the actual value and the predicted value of a sample. The formula given as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where n is the total number of samples, y_i is the target of the i^{th} sample, and $\hat{y}_i$ is the prediction for the i^{th} sample.

In a regression setting, if we do not want to overemphasize the larger error terms, we deploy MAE.

1. Since we are NOT squaring the error of the prediction like we did while calculating the MSE, the MAE calculation does not lay more stress on larger errors.
2. What is a good MAE value?
 - a. Replace each prediction by the median value of all actual values and calculate the MAE; our machine should have a MAE lower than this.
 - b. In trying to predict the returns for the next day we can use today's actual return as a proxy and calculate a benchmark MAE value. The machine we eventually deploy should have a MAE value lower than this benchmark. In doing so we are necessarily ensuring that today's price is not the best predictor of tomorrow's price
 - c. Replace the prediction with the average actual value for previous several days and calculate a benchmark MSE value. Again, we want our machine to have a MAE lower than this benchmark.

MSPE/MAPE

These are just MSE and MAE standardized by the actual values of the dependent variable.

$$MSPE = \frac{100\%}{N} \sum_{i=1}^N \left(\frac{y_i - \bar{y}_i}{y_i} \right)^2$$

$$MSPE = \frac{100\%}{N} \sum_{i=1}^N \left| \frac{y_i - \bar{y}_i}{y_i} \right|$$

It is best to use it in a setting where we are trying to predict the returns of several assets at the same time. To exemplify let us suppose we are trying to predict the next tick of E-mini S&P 500 futures, treasury futures, and crude oil future through the same machine. E-mini S&P 500 futures have a tick size of 25 cents, treasury futures of 0.015625 cents, and crude oil of 1 cent. It makes sense to use a standardized measure of error in this case. MSPEs/MAPEs are the recommended metrics.

Classification Metrics

Building on the momentum setup we spoke about in Chapter 1; we may be interested in predicting whether palladium prices are expected to rise or fall the next day. If they are expected to rise, we assign those samples a class label of 1. Similarly, if they are expected to fall, we assign those samples a class label of zero. The classification machines will help us put a probability against these occurrences.

The output of any classification algorithm is a conditional probability. For a two-class classification problem denoted by 1 and 0, the output is:

$P(Y=1 \mid \text{Feature set})$: Probability of Y equal to 1, given the Feature set.

$P(Y=0 \mid \text{Feature set})$: Probability of $Y=0$, given the Feature set.

Once we have a probability measure, we need to make an assumption of a certain threshold value, which is the probability value above which samples are assigned to class 1 and below which samples are assigned to class 0.

That is to say, if:

$P(Y = 1 \mid \text{Feature set}) > \text{Threshold}$ implies class 1 or

$P(Y = 0 \mid \text{Feature set}) > \text{threshold}$ implies class 0

What follows is a list of metrics best suited for a classification problem. Before that we need to understand a confusion matrix.

Confusion Matrix

A confusion matrix is a table of four quadrants that helps us analyze the performance of a (binary) classification machine. In a binary classification problem with class labels 1 and 0, we enter the number of samples that belong to each quadrant. Each quadrant contains samples, which satisfy both conditions of the actual and predicted class values, for example, quadrant 2 in the succeeding text will contain all those samples, which were predicted to belong to class 1 and actually belonged to class 0.

		Predicted	
		0	1
Actual	0	1	2
	1	3	4

It is constructed in sklearn using the following code:

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_train, y_train_pred)
```

Based on the confusion matrix, the following metrics are defined and easy to understand:

Accuracy

Of all the predictions made, what fraction of the total was predicted correctly? It is the number of samples in quadrants 1 and 4 divided by the total number of samples.

```
from sklearn.metrics import accuracy_score
accuracy_score(y_train, y_train_pred)
```

Precision

Let us assume we are a long only fund. We are only interested in timing buy signals. Once our machine gives us the probabilities, we assume a certain threshold and convert the probabilities to an actionable buy signal. Our universe is now limited to these buy signals. How many of these buy signals made money for us? Or, how many of these predicted buy signals belong to Class 1? That is precision, which would be defined as the number of samples in quadrant 4 divided by the combined number of samples in quadrant 2 and quadrant 4.

```
from sklearn.metrics import precision_score  
precision_score(y_train, y_train_pred)
```

Sensitivity/Recall

Continuing with the example of the long only fund, we had certain buy signals that we got from the buy probabilities and a threshold assumption. We may find that these signals are too few in number to make the estimate statistically reliable, or we may find that there are way too many signals, thereby significantly reducing the profitability per trade. Either way, we are interested in knowing how many buy signals we are getting compared to the total number of buy signals the market gives us. That is recall. That would be computed as the number of samples in quadrant 4 divided by the combined number of samples in quadrant 3 and quadrant 4.

Specificity

Specificity is the number of samples correctly predicted to belong to class 0 divided by the number of samples actually belonging to class 0. That would be computed as the number of samples in quadrant 1 divided by the combined number of samples in quadrant 1 and quadrant 2 of the confusion matrix.

F1 score

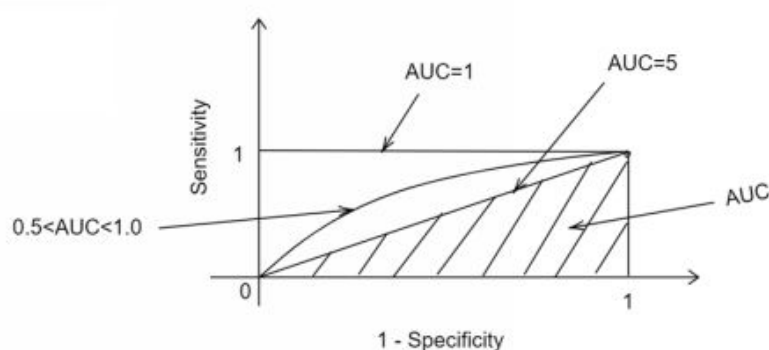
From the discussion of precision and recall, it is reasonable to assume that we would not only like to maximize the accuracy of our buy predictions (precision) but also to maximize the total number of buy predictions as a percentage of the total number of buy signals that the market can give us. The F1 score represents just this. It is the harmonic mean of the accuracy and the precision.

```
from sklearn.metrics import classification_report  
classification_report(y_train, y_train_pred)
```

The above code can provide precision, recall, f1-score and accuracy.

AUC

You have probably noticed that which quadrant of the confusion matrix does a sample get assigned to depends on the threshold. To take a step back, which class a sample gets assigned to once we have the probability depends on the threshold we chose. A plot of sensitivity (y-axis) vs $1 - \text{specificity}$ on the x-axis for various values of threshold, gives us the ROC (receiver operating characteristics) curve. The area under the ROC curve (AUC) is a metric for measuring classification performance. $\text{AUC} = 0.5$ essentially implies no learning, and $\text{AUC} = 1$ implies the best possible machine.



The biggest take away from this chapter is the methodology we discuss on how to build the validation strategy. An error at this point could well be the difference between success and failure.

We have also introduced you to various metrics for evaluating a classification and a regression machine.

Summary

1. We have discussed in some detail the qualities and requirements of a validation data set.
2. The key is to think of validation data as test data or data from the future being streamed sample by sample as opposed to reading the full chunk from the database. Both predictions should give the exact same results. The point being that all information needed to predict a validation sample should be available at that time and should not be leaked in from the future.
3. Finally, we've discussed various metrics to measure the performance of the machine.

CHAPTER FOUR - TREE MODELS

In this chapter, we introduce decision trees as one of the most powerful and interpretable algorithms deployable in finance. In our experience, these are best suited for problems where the amount of data is limited, but not scarce. Some would say that decision trees are similar to human learning process, that they rely on compartmentalization and exclude lower probability scenarios.

At the outset, it is important to note that decision trees are inherently weak learners, but a few simple and extremely intuitive modifications can make them extremely powerful. And what do we mean by “weak learners”? Weak learners are those algorithms that perform really well on the training data set but find it hard to generalize well on validation or test data sets.

By the end of this chapter you will understand the step by step process of constructing a decision tree for classification and regression tasks. This understanding is instrumental in understanding the applications we discuss in the next few chapters

Decision Trees

Let us look at a decision tree in a regression setting, where we have only five samples and two features. This is only a toy example and we need more samples for meaningful learning and ideally more features too. As a target we are trying to predict what the E-mini S&P 500 futures would return the following day. As feature 1, we have the previous day returns of e-mini futures, and as feature 2, we have the previous day returns of active crude oil futures. Our hypothetical data set looks something like Figure 4.1.

Sample No	Feature 1	Feature 2	Target
1	0.50%	1%	0.25%
2	1%	2%	0.25%
3	2%	-2%	-1.00%
4	-0.50%	-1%	-0.25%
5	-1%	-2%	-0.25%

Figure 4. 1 Hypothetical data set.

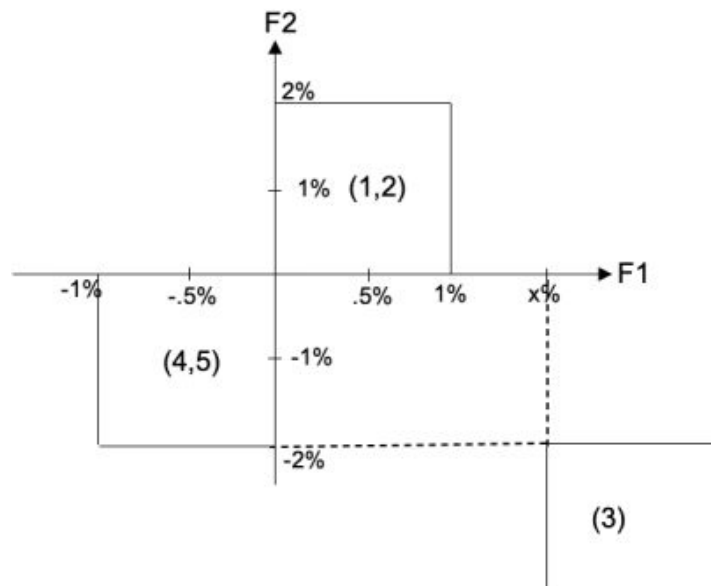


Figure 4. 2 Representation of samples in the F1, F2 feature space.

One way of representing the in the F1, F2 feature space is shown in Figure 4.2, where the number in the brackets represents the sample number. We have divided the feature space F1 and F2 into rectangular regions. In addition, based on the values of F1 and F2, we have assigned the samples to these rectangular regions. As an example, samples with sample numbers 1 and 2 belong to a rectangular region where:

$$0 \leq F1 \leq 1\%$$

$$\text{and } 0 \leq F2 \leq 2\%$$

The prediction is simply an average of the target values of samples that have been assigned to that region. Sample numbers 1 and 2 belong to the same rectangular region, the average value of their targets being 0.25%. Therefore, any sample from the test data set that gets assigned to this rectangular region, will have a prediction of 0.25%.

That is how simple decision trees are.

To summarize, decision trees divide the feature space into rectangular regions as a result of which samples from the training data set are assigned to these rectangles. When a test sample arrives, it is assigned to a rectangular region based on its feature values F1 and F2. The dependent variable simply assumes a value equal to the average of the “target” values of the samples from the training data set that belong to that particular region.

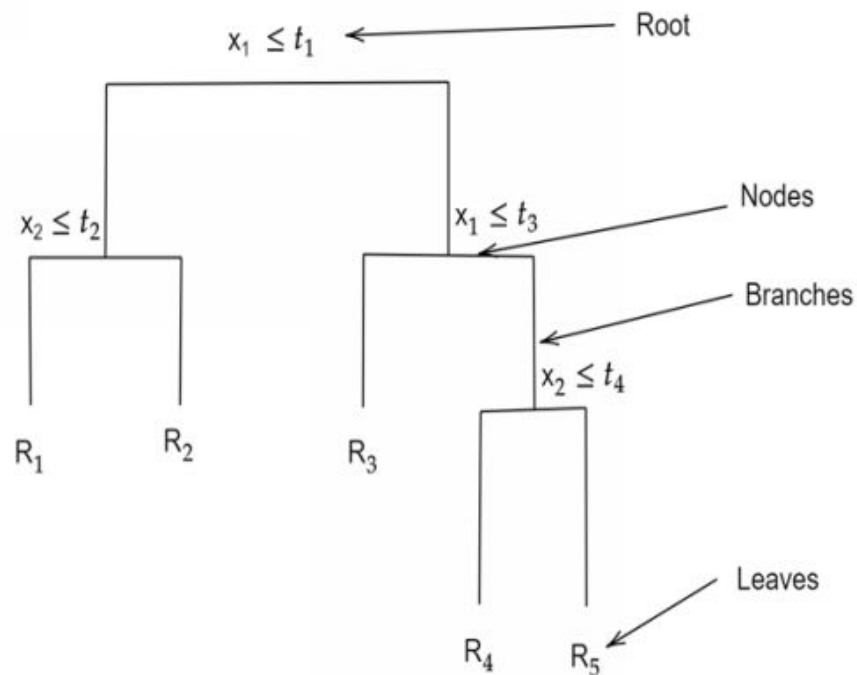


Figure 4. 3 Representation of samples in the F_1, F_2 feature space.

The tree-like representation of the data set looks like Figure 4.3 where we have roots, nodes, branches, and leaves.

Nodes are where decisions are taken. The lines connecting the nodes are called branches. The top-most node is the root. The bottom most nodes are the leaves or terminal nodes. The terminal nodes are like the rectangular regions where the samples of our training data essentially reside. Let us now discuss what goes into the construction of these decision trees.

Construction of Decision Trees - Regression setting

The decision tree algorithm essentially takes as input our tabular data and constructs the decision tree.

We expect the algorithm to yield two outputs:

1. The split points that help us take a decision at each node or define the boundaries of the rectangular regions in the earlier representation. So one of our nodes could be taking a decision such as $x \leq 2\%$. This split point of 2% will be output by the algorithm.
2. The correct hierarchy of features to do the split. Which feature comes at the root, which comes at the nodes below the root and so on.

Let us take a cursory look into the algorithm that gives us these outputs. For the time being, we stick to a regression setting. This level of detail is necessary for innovative application of the algorithm.

We start by considering all the features as a potential candidate to form the root. For each feature, we consider various possible split points. For each possible feature and split point combination, we calculate the value of the *Equation 4.1* across all the samples of the training data set.

$$\text{MSE} = \sum (Y_i - Y_{\text{avg}})^2 \quad \text{Equation 4.1}$$

Where

Y_i : the target for each sample

Y_{avg} : the average value of the targets for each of the samples

Let us go back to our rectangular region representation to understand this completely. We start with feature F1 and take a split point as 1%. We denote the (feature, split point) combination as (F1, 1%). We have four samples (1, 2, 4, 5) in the region to the left of F1=1% and one sample (3) to the right.

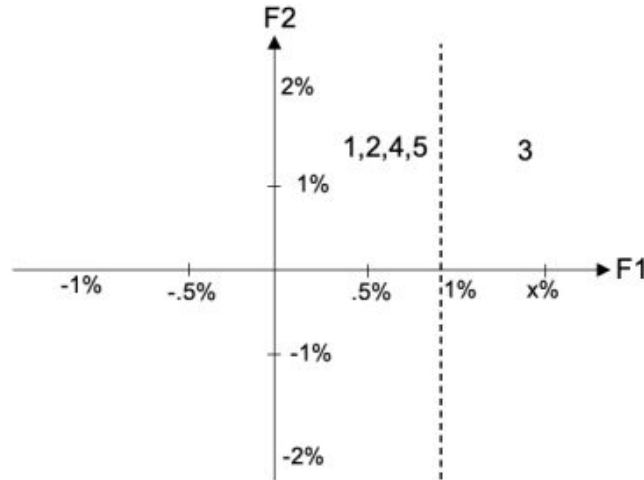


Figure 4. 4 (F1,1%) forms the root.

For samples 1,2,4, and 5, we calculate Equation 4.1

$$Y_{\text{avg}} = 0 \text{ and}$$

$$\sum (Y_i - Y_{\text{avg}})^2 = .025\% \quad \text{Equation 4.2}$$

Likewise, for sample 3, we calculate Equation 4.1 as well

$$\sum (Y_i - Y_{\text{avg}})^2 = 0 \quad \text{Equation 4.3}$$

We add the findings in *Equation 4.2* and *Equation 4.3*. This calculation is repeated for each feature, and split point combination. The combination that gives the lowest recording forms the root. For purposes of this example let us say (F1,1%) forms the root.

The process detailed earlier is repeated to find the feature/split point combination for all the nodes. The only modification is that we limit ourselves to the samples of the region that we are trying to further subdivide. So, in our toy example, samples 1,2,4 and 5 are now clubbed together to form region A (with $F1 < 1\%$) and sample 3 forms region B (with $F1 > 1\%$). We now focus only on region A to find the node for the second level of the tree. Region B contains only one sample and cannot be split any further which is effectively a terminal node or a leaf. This process is repeated until stopping criteria are met.

The way we implement this in Scikit Learn is as follows:

```
from sklearn import tree
clf = tree.DecisionTreeRegressor()
clf = clf.fit(X, y)
```

Some parameters that go into DecisionTreeClassifier

- **criterion:** “mse”
- **max depth:** The maximum depth of the tree controls the overfit. Default results in all leaves being pure.
- **min_samples_split:** The minimum number of samples required to split an internal node.

Construction of Decision Trees - Classification setting

The classification problem is dealt with in a very similar way with a few changes.

1. Prediction is done based on voting. So, once we have formed the rectangular region, the class label associated with it will be that which occurs a majority of the times.
2. The metric used for finding the feature, split point combination is no longer the mean squared error (MSE). We use either the Gini index or cross entropy.

Let us look at some definitions to understand the latter point.

We need the *classification error rate*, which is the proportion of samples in a region that have been misclassified. Figure 4.1 is represented for a classification setting. The target class 1 implies return >0 and target class 0 implies return <0 .

S.No.	F1	F2	Target
1	0.50%	1%	1
2	1%	2%	1
3	2%	-2%	0
4	-0.50%	-1%	0
5	-1%	-2%	0

Figure 4. 5 Hypothetical data set for classification setting.

For a region defined by $F1 \leq 2\%$, we have most of the samples belonging to class 0. The error rate is therefore 40%.

P_0 : proportion of samples in the region with target 0 (60%).

P_1 : proportion of samples in the region with target 1 (40%).

Classification error rate = $1 - \max(P_0, P_1)$

Cross entropy = $-P_0 \log(P_0) - P_1 \log(P_1)$

Gini index = $P_0(1 - P_0) + P_1(1 - P_1)$

Any of the, the classification error rate, the Gini index or the cross entropy (see Chapter 9 for more detail), are minimized if all the samples in that region belong to a specific class. From here on the process of constructing a classification tree is the same as what we have described for the regression tree setting. Instead of minimizing *Equation 4.1*, we minimize any one of the classification error rate, Gini index or cross entropy. Preferably cross entropy or Gini index.

The way we implement this in Scikit Learn is

```
from sklearn import tree
clf = tree.DecisionTreeClassifier ()
clf = clf.fit(X_train, Y_train)
clf.predict_proba (Y_test)
```

Some parameters that go into DecisionTreeClassifier

- **criterion**: “gini” or “entropy”
- **max_depth**: The maximum depth of the tree controls the overfit. Default results in all leaves being pure.
- **min_samples_split**: The minimum number of samples required to split an internal node.

Weak learners

As defined earlier, a weak learner is one that finds it hard to generalize on unseen data but does well on training data. A weak learner is of very little value to us. Decision trees are weak learners. This was subtly alluded to in *Equation 4.3*, where the MSE for that region turned out to be zero, because there was only one sample. This means we can keep dividing our rectangular regions until we have only one sample per region. In such a case the MSE (or the relevant metric for classification) will be zero and we would have perfectly fit the training data. However, when used to predict test or unseen data, the prediction will fail massively. We will explain how we overcome this in the next chapter.

Summary

So now we know how to construct a decision tree and the various cost functions associated with a classification and a regression tree. We understand the problem of weak learners.

References

- <https://scikit-learn.org/stable/modules/tree.html>
- <https://faculty.marshall.usc.edu/gareth-james/ISL/ISLR%20Seventh%20Printing.pdf>
- <http://appliedpredictivemodeling.com/>

CHAPTER FIVE - ENSEMBLE METHODS

In the previous chapter, we have discussed the algorithm for constructing a decision tree in detail. It became apparent that this algorithm could overfit the training data easily. In this chapter, we will discuss modifications that can help us deal with the problem of overfitting.

There are several ways of reducing overfit in decision trees. These include the following:

- Early stopping
- Cost-complexity pruning
- Bagging
- Random forest
- Boosting trees

Bagging, random forest, and boosting trees are the more popular and powerful approaches.

Early Stopping

As discussed in the previous chapter, while constructing a tree we need to start by specifying a metric that we intend to minimize. For a regression problem, it could be mean squared error (MSE) and for classification, it could be cross-entropy. With early stopping, in case the MSE/cross-entropy stops changing by a predetermined threshold, we could stop the compartmentalization of the feature space. Alternatively, we could stop the further splitting of our samples and our tree would be ready. This is a shortsighted approach. The eventual aim of the decision tree algorithm is to get leaves that contain samples with a very similar set of target values. Accordingly, the leaves should be as pure as possible. Simply stopping the compartmentalization process because the MSE/cross-entropy did not change adequately at a particular level obstructs a potential split in the future, which could have created a pure leaf. This method is rarely deployed.

Cost-Complexity Pruning

Cost-complexity pruning improves the short-sighted nature of the early stopping criteria. It starts by building a complete tree. From this complete tree, it creates various sub trees. Any one of the generated trees could be the optimal solution. The optimal tree is chosen by cross-validation.

What is interesting and important is a little bit of insight into the cost-complexity-pruning algorithm by which we are able to generate the subset of trees. The cost function that helps in implementing the cost complexity pruning algorithm is in Equation 5.1.

$$[\sum(Y_i - Y_{avg})^2 + \alpha |T|]$$

Equation 5.1

The first part of this equation does not require any explanation and is the same as discussed in chapter 4 . The $\alpha |T|$ is the regularization term where $|T|$ represents the number of terminal nodes or leaves in the tree and α is a parameter that we need to provide. So, if $\alpha = 0$, Equation 5.1 reduces to simply minimizing MSE. The tree that we get is the entire un-pruned tree. However, as we start increasing α , there is a price to be paid for having a large number of terminal nodes and therefore Equation 5.1 will tend to be minimized for a tree with a smaller number of terminal nodes. As α increases, we get smaller and smaller trees, minimizing the value of the cost function. For instance, for α equal to infinity we would need T to be zero, reducing the tree to a single node. This way we are able to map α to $|T|$ and hence the particular sub-tree.

The tree we would use is the one that gives the best performance on a validation data set.

Bagging

Another way of reducing the overfitting or high variance problem in decision trees is bagging. Bagging is a two-step process comprising first of “bootstrapping” and then of “averaging”.

As illustrated in Figure 5.1, we start by creating several samples of the original dataset and then build an ensemble of decision trees on each of the datasets. The final ensemble of trees is just the average of all the predictions coming from each of these decision trees. The idea is that this process creates a set of uncorrelated trees - each learning some information and inevitably some noise. The averaging over several such trees would reduce the noise component and retain the information.

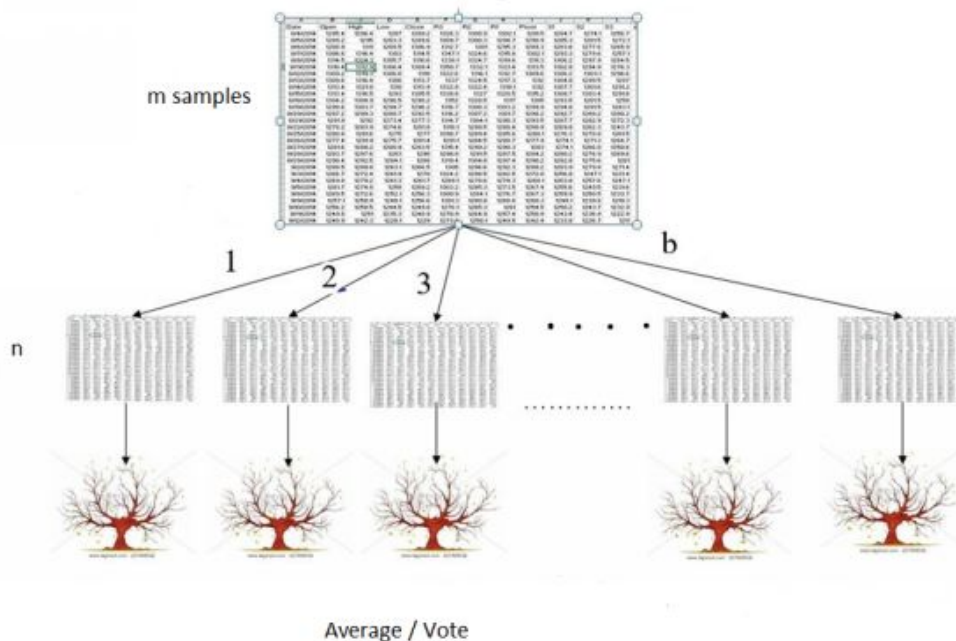


Figure 5. 1 The concept of bootstrapping illustrated.

The process by which we extract multiple datasets from the original dataset is called bootstrapping. Let us say the original data set has m samples. We select $n(<m)$ samples randomly with replacement. We do this b number of times. Therefore, we now have b data sets with n samples each, rather than one data set with m samples. We create b trees, one on each newly created data set. The final machine is an aggregation of the results from each of these b trees.

Now if this is a regression problem, we will simply average the result of each b tree, and if it is a classification problem, we will take a majority vote on the predictions made by each of these classifiers.

Out-of-bag samples

Each sample from the original data set has either been used to create a particular tree or has been left out in the bootstrapping process. Accordingly, each tree has its own training data on which it has been built. Each tree also has some data that it was not seen and that can be used as the validation data. Therefore, to get the validation performance for all the b trees that we have built, we will simply aggregate the results of each tree on its corresponding validation data set. This is great and really data efficient, as we do not really need a separate validation data set.

Only one important issue needs some attention - while bootstrapping, we randomly select m ($< n$) samples with replacement. As we have highlighted earlier in this book, we are not entirely comfortable with this approach when dealing with time series data. Furthermore, as we are using our validation data to be out-of-bag samples, we are hindering the process of learning by destroying the time series aspect of this data set (that is what random selection does).

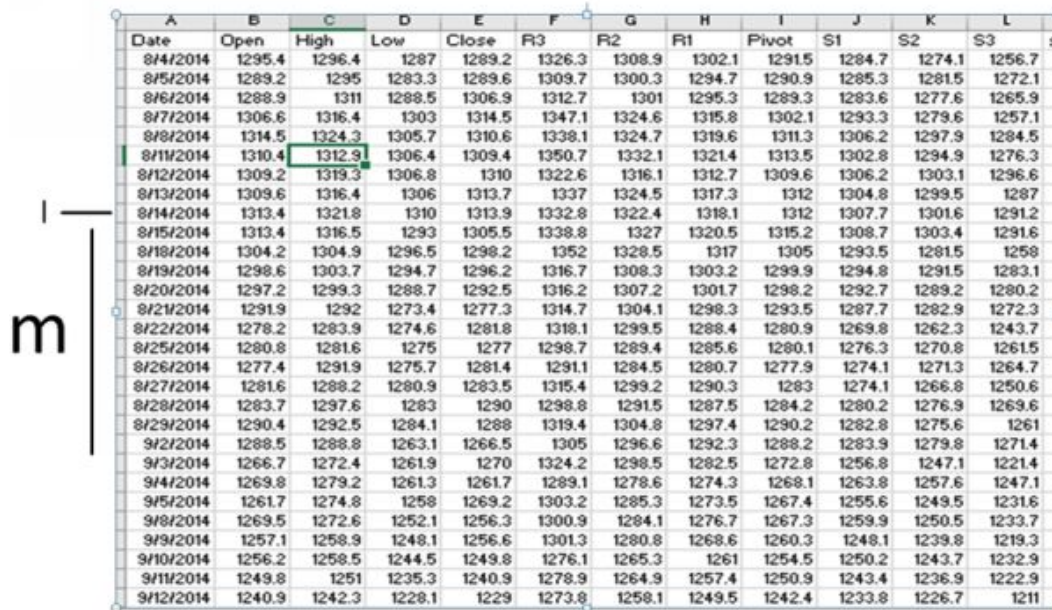
The samples are not independent of each other. As a reminder while designing our validation strategy, we need to be able to answer the following question: If our validation data were our test data, would we have all the information in time to predict our test data?

If the answer to the aforementioned question is “no,” then we are using a suboptimal machine to make our prediction. There may be information in the data, which we have not attempted to learn yet.

To resolve this, each time we do bootstrapping, we need to ensure that the samples selected are in order. A scheme that could be used is as follows (also depicted in Figure 5.2):

1. Randomly select a number i from 1 to $(n - m)$
2. Select i to $(i + m)$ samples as a bag

3. Repeat b times



Date	Open	High	Low	Close	R3	R2	R1	Pivot	S1	S2	S3	S
8/4/2014	1295.4	1296.4	1287	1289.2	1326.3	1308.9	1302.1	1291.5	1284.7	1274.1	1256.7	
8/5/2014	1289.2	1295	1283.3	1289.6	1309.7	1300.3	1294.7	1290.9	1285.3	1281.5	1272.1	
8/6/2014	1288.9	1311	1288.5	1306.9	1312.7	1301	1295.3	1289.3	1283.6	1277.6	1265.9	
8/7/2014	1306.6	1316.4	1303	1314.5	1347.1	1324.6	1315.8	1302.1	1293.3	1279.6	1257.1	
8/8/2014	1314.5	1324.3	1305.7	1310.6	1338.1	1324.7	1319.6	1311.3	1306.2	1297.9	1284.5	
8/11/2014	1310.4	1312.9	1306.4	1309.4	1350.7	1332.1	1321.4	1313.5	1302.8	1294.9	1276.3	
8/12/2014	1309.2	1319.3	1306.8	1310	1322.6	1316.1	1312.7	1309.6	1306.2	1303.1	1296.6	
8/13/2014	1309.6	1316.4	1306	1313.7	1337	1324.5	1317.3	1312	1304.8	1299.5	1287	
8/14/2014	1313.4	1321.8	1310	1313.9	1332.8	1322.4	1318.1	1312	1307.7	1301.6	1291.2	
8/15/2014	1313.4	1316.5	1293	1305.5	1338.8	1327	1320.5	1315.2	1308.7	1303.4	1291.6	
8/18/2014	1304.2	1304.9	1296.5	1298.2	1352	1328.5	1317	1305	1293.5	1281.5	1258	
8/19/2014	1298.6	1303.7	1294.7	1296.2	1316.7	1308.3	1303.2	1299.9	1294.8	1291.5	1283.1	
8/20/2014	1297.2	1299.3	1288.7	1292.5	1316.2	1307.2	1301.7	1298.2	1292.7	1289.2	1280.2	
8/21/2014	1291.9	1292	1273.4	1277.3	1314.7	1304.1	1298.3	1293.5	1287.7	1282.9	1272.3	
8/22/2014	1278.2	1283.9	1274.6	1281.8	1318.1	1299.5	1288.4	1280.9	1269.8	1262.3	1243.7	
8/25/2014	1280.8	1281.6	1275	1277	1298.7	1289.4	1285.6	1280.1	1276.3	1270.8	1261.5	
8/26/2014	1277.4	1291.9	1275.7	1281.4	1291.1	1284.5	1280.7	1277.9	1274.1	1271.3	1264.7	
8/27/2014	1281.6	1288.2	1280.9	1283.5	1315.4	1299.2	1290.3	1283	1274.1	1266.8	1250.6	
8/28/2014	1283.7	1297.6	1283	1290	1298.8	1291.5	1287.5	1284.2	1280.2	1276.9	1269.6	
8/29/2014	1290.4	1292.5	1284.1	1288	1319.4	1304.8	1297.4	1290.2	1282.8	1275.6	1261	
9/2/2014	1288.5	1288.8	1263.1	1266.5	1305	1296.6	1292.3	1288.2	1283.9	1279.8	1271.4	
9/3/2014	1266.7	1272.4	1261.9	1270	1324.2	1298.5	1282.5	1272.8	1256.8	1247.1	1221.4	
9/4/2014	1269.8	1279.2	1261.3	1261.7	1289.1	1278.6	1274.3	1268.1	1263.8	1257.6	1247.1	
9/5/2014	1261.7	1274.8	1258	1269.2	1303.2	1285.3	1273.5	1267.4	1255.6	1249.5	1231.6	
9/8/2014	1269.5	1272.6	1252.1	1256.3	1300.9	1284.1	1276.7	1267.3	1259.9	1250.5	1233.7	
9/9/2014	1257.1	1258.9	1248.1	1256.6	1301.3	1280.8	1268.6	1260.3	1248.1	1239.8	1219.3	
9/10/2014	1256.2	1258.5	1244.5	1249.8	1276.1	1265.3	1261	1254.5	1250.2	1243.7	1232.9	
9/11/2014	1249.8	1251	1235.3	1240.9	1278.9	1264.9	1257.4	1250.9	1243.4	1236.9	1222.9	
9/12/2014	1240.9	1242.3	1228.1	1229	1273.8	1258.1	1249.5	1242.4	1233.8	1226.7	1211	

Figure 5. 2 Ideal way to bootstrap time series data.

Bagging derives its benefits from the following two sources:

- Averaging over multiple predictors reduces variance.
- Each bag forces the tree to focus on fewer samples and not just on the prominent ones, thereby forcing the tree to work hard to learn information content that may be overlooked.

Let us see what else we can do to deal with the high variance problem by using Random Forest.

Random Forest

As mentioned earlier, one of the drivers of the bagging algorithm was that each tree was focusing on a different set of samples and was therefore forced to learn less obvious intelligence. The random forest does all of that and a little more. Figure 5.3 shows the steps.

While forming the nodes of a tree, we need to identify which feature we place at the node and at what point we split it. Our current algorithm has the option to choose this split at each node from all available features. Random forest changes that. We randomly select a subset of m features from the original p set of features ($m/p \leq 1$). It is from this subset that we identify the best feature at a particular node. We no longer have the luxury of selecting a feature from the original p set of features. We are limited to selecting the best splitting candidate from the subset of features.

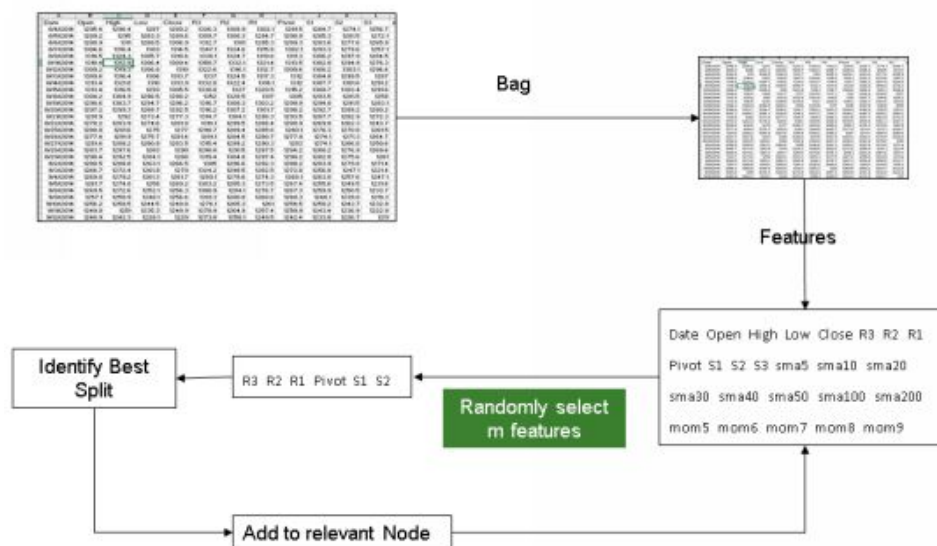


Figure 5. 3 Random forest schematic diagram.

Random forest derives its benefits from the following two sources:

- Averaging over multiple predictors reduces variance.
- Each bag forces the tree to focus on a fewer samples and not just on the prominent ones, thereby forcing the tree to work hard to learn information content that may be overlooked.

What happens very often in data sets is that only a few features are correlated. Moreover, sometimes it is only a subset of the entire feature space that contributes to the bulk of the predictive power of the model. By focusing only on a smaller subset of the features each time a split is being made, the random forest forces learning from even the less prominent features. This not only reduces the correlation between the various tree models of the ensemble thereby reducing the variance when averaging is done, but also learns more valuable information from the data set.

Boosting

Let us get straight to the algorithm first and then work out the intuition later.

The steps for a boosting algorithm are as follows:

1. Fit a tree (T_1) to the training data set.
2. Calculate the residuals/errors/misclassifications.
3. Fit a tree T_2 to the residuals of T_1 .
4. Update Model as $T_1 + \text{shrinkage} * T_2$.
5. Fit a model T_3 to the residuals of T_2 .
6. Update Model as $T_1 + \text{shrinkage} * T_2 + \text{shrinkage} * T_3$.
7. Fit a model T_N to the residuals of T_{N-1} .

Before we jump into more details, let us quickly understand what shrinkage is. It is a hyper-parameter (i.e. a parameter that needs to be given by the user) which also goes by the name of the learning rate. It helps us control the amount of learning (reduce overfit) at each stage that we want to incorporate into the final model. If the shrinkage is too high, we risk overfitting to the training dataset; if it's too low, we risk underfitting and not learning anything. Moving on, the first intuition we need to understand is that this is a stage wise additive model. It is additive in the same sense as bagging and random forest, where the final model is an aggregation of predictions from N models.

Where boosting is different from bagging and random forest is that it is a stage wise additive model. T_2 can be built only after T_1 , T_3 can be built only after T_2 , and so on. In bagging and random forests, T_1 , T_2 , and T_3 can be built simultaneously and are independent of each other. The same cannot be said about boosting. We first need to build T_1 , calculate the errors on the training data, build T_2 to fit these residuals, calculate the errors of T_2 on the training data, and then build T_3 to fit to the errors of T_2 's prediction on the training data. Accordingly, T_1 , T_2 , T_3 , and so on are dependent on each

other and cannot be created in a parallel way. They need to be created serially one after the other.

Therefore, unlike most other algorithms where we fit the model on the independent variable, here we fit several models. The first is fit on the independent variable and the rest are fit on the errors of the previous model. So, let us try to understand why we need to build models to fit the errors of the previous model.

The basic construct of a stagewise additive model is as follows:

$$F_m(X) = F_{m-1}(X) + b_m * T_m(X, Y_{m-1}) \quad \text{Equation 5.2}$$

$F_m(X)$ is the ensemble after m th iteration.

T_m is the tree built on the m th iteration.

Y_{m-1} is the error using T_{m-1} as the predictor for $m > 1$.

$Y_{m-1} = Y$, for $m = 1$.

b_m is the learning rate that controls the amount of learning we want to take forward from each stage.

All that Equation 5.2 is saying is that a model at any stage m (F_m) is equal to the model at the previous stage m (F_{m-1}) and a new model T_m .

Why fit models on residuals of previous stage?

Now our loss function is a simple MSE:

$$L(y, f(x)) = (y - f(x))^2$$

Using equation 5.2 we modify the loss function as

$$\begin{aligned} L(y, F_{m-1}(X) + b_m * T_m(X, Y_{m-1})) &= (y - F_{m-1}(X) - b_m * T_m(X, Y_{m-1}))^2 \\ &= (r_m - b_m * T_m(X, Y_{m-1}))^2 \end{aligned}$$

Therefore, the tree at the m th stage has to be built to fit the residual r of the previous stage.

Before we look at the practical implementation, let us quickly understand regularization in trees.

Regularization

Regularization is a way of reducing the high variance problem. The objective is achieved by modifying the loss function. So instead of simply minimizing a loss function L we minimize a loss function $L + \textit{gamma} * |T|$, where $|T|$ represents the number of leaves. The complexity of the tree is directly proportional to $|T|$. The *gamma* is a parameter that needs to be supplied by the user. If *gamma* is zero, then $\textit{gamma} * T = 0$, which effectively means we are left with the original loss function and hence do not intend to reduce the high variance problem. As we increase the value of *gamma* to say infinity, the only way we will be able to reduce the loss term from exploding is by reducing the value of T , which effectively means we are reducing the complexity of the tree. Hence, by varying *gamma* in the aforementioned equation we are in a position to control the complexity of the tree and hence influence bias variance.

XGBOOST package

The XGBOOST package has two parts to the loss function:

1.
$$\overline{L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma))}$$

This is the same as what we've discussed in Equation 5.2 and is calculated over all samples.

2. A **regularizing** part is denoted by

$$\gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

The γT has been introduced earlier in Equation 5.1 when we discussed cost complexity pruning and has the same interpretation. T is the number of terminal nodes in a tree. The second term is the **L2 regularization term**, where w is the weight or score per leaf of each tree. If λ is large then w would need to be small, i.e. the score (e.g. accuracy, AUC etc.) in a leaf would need to be small, which would mean we reduce the overfit. If λ is small, we can allow for a higher accuracy and thereby allow for a better fit on the training data. Each tree is subjected to this regularization, so this part is summed over all trees we additively build.

The final loss function that is used is as follows:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

where $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$

Where i is the summation over all samples and k is the summation over all the trees.

Other important parameters in XGBOOST library are as follows:

1. “Eta”: The shrinkage we have discussed earlier is the learning rate
2. Gamma minimum loss reduction require
3. Lambda: L2 regularization on weights
4. Alpha: L1 regularization on weights
5. Maximum depth
6. Minimum child weight
7. Sample columns by tree, level, and node

(Refer to the XGBOOST package documentation for a more detailed description of all the parameters that could be used)

Application

Now that we have learnt the intuition behind ensemble of decision trees, let us consider a simple example. We alluded to this in Chapter 2. Mean encoding is a very useful tool and sometimes it may be beneficial to try and mean encode numeric features. Before we can mean encode numeric features, we need to bin them or make them categorical. For the purposes of this example the top 10 rows of our data set look like this (Figure 5.4 below), where each column is essentially the return of a future traded on CME. So, GC_ret, for instance, is the tick return for gold futures (multiplied here by a factor of 1000 just for the purposes of increasing resolution).

Index	GC_ret	CL_ret	HG_ret	PL_ret	ES_ret	6E_ret	6B_ret	6J_ret	NQ_ret
0	37.6733	0	80.50882	-149.574	-101.199	83.8469	38.6578	-27.474	0
1	37.6733	0	80.50882	-149.574	-101.199	83.8469	38.6578	-27.474	-41.7049
2	37.6733	0	80.50882	-149.574	-101.199	83.8469	38.65631	-27.474	-41.7049
3	37.6733	0	-80.5023	-149.574	-101.199	83.8469	38.65631	-27.474	-41.7049
4	37.6733	0	-80.5023	-149.574	-101.199	83.8469	38.65631	-27.474	-20.8533
5	37.6733	0	-80.5023	-149.574	-101.199	-41.9199	38.65631	-27.474	-20.8533
6	37.6733	-212.427	-80.5023	-149.574	-101.199	-41.9199	38.65631	-27.474	-20.8533
7	37.6733	-212.427	-80.5023	-149.574	-101.199	-41.9199	38.65631	27.47479	-20.8533
8	37.6733	-212.427	-80.5023	-149.574	-101.199	-41.9199	38.65631	27.47479	-41.7075
9	37.6733	-212.427	-80.5023	-149.574	-101.199	-41.9199	38.65631	27.47479	-20.8546

Figure 5. 4 Return of various instruments traded on CME (Gold (GC), crude oil (CL), Copper (HG), platinum (PL), Emini S&P 500 (ES), Euro (6E), sterling 6B, Dollar yen 6J, Nasdaq (NQ)).

This is our matrix of independent variables or our features. We want to use this information to predict what Emini S&P 500 will do 10 ticks later. Where we have 3 alternatives, up, down or flat encoded as 2, 1, and 0 respectively. Figure 5.5 shows a snapshot of the data set.

Index	ES_target
0	2
1	2
2	2
3	2
4	2
5	2
6	2
7	2
8	1
9	1

Figure 5. 5 The targets of our classification problem.

We now define a random forest classifier and fit it on our dataset as below

```
rf = RandomForestClassifier(max_depth=3, n_estimators=10, n_jobs=4)
rf.fit(X, y)
```

Where we have built an ensemble of 10 trees. The next line helps to get the index of the leaf for each tree where the sample X resides in

```
d=rf.apply(X)
```

The shape of d is (n_samples, 10) in our example, and these 10 values are the categorical representation of the features in our input. Each sample in X will now be represented by an array of 10 integers. Each integer references the index of the terminal node where the sample resides in a particular tree. Since there are 10 trees in our ensemble, we will have 10 such terminal nodes which serve as categorical representation of our data. Once we have these 10 categorical values we can go ahead and mean encode each one of them.

One very important consideration that has not been explicitly stated so far pertains to the data set we get our mean encodings from. So far, we have always and only been concerned with a training, validation and test data set. However, when we are getting the mean encodings for our training data set, we cannot use the full training data set. If we do that, we have essentially embedded information from the targets into our features and our models would overfit. Ideally, and most appropriately, we need to keep a separate data set called 'pre training' from which we could get our mean encodings. Alternatively, we could design a k-fold validation strategy to get our mean encodings for our training data set.

To get the mean encodings for our validation data set, we can use the entire training data set.

Finally, to get the mean encoding of our test data set, we can use the training and validation data set.

Summary

We have covered a fair bit in this chapter. The general theme has been to try and modify weak learners to get powerful learning algorithms. Therefore, we have discussed early stopping and cost complexity pruning, the most used techniques however are bagging, random forest and boosting. We discussed an application on how decision trees can be used to convert numeric data to categorical data and the pitfalls of mean encoding we need to avoid.

References

- Constructing Equity Portfolios from SEC 13F Data Using Feature Extraction and Machine Learning. Alexander Fleiss, Han Cui, Sasha Stoikov and Daniel M. DiPietro The Journal of Financial Data Science Winter 2020, 2 (1) 45-60; DOI: <https://doi.org/10.3905/jfds.2019.1.022>
- <https://www.kaggle.com/c/two-sigma-financial-news/notebooks>
- <https://xgboost.readthedocs.io/en/latest/tutorials/model.html>
- <https://medium.com/kaggle-blog/two-sigma-financial-modeling-code-competition-5th-place-winners-interview-team-best-fitting-279a493c76bd>

CHAPTER SIX - BIAS VARIANCE TRADE-OFF

This is a short chapter, but we would like to dedicate a full chapter to this topic simply based on the universal applicability of this concept. By the end of this chapter you should be able to understand the bias variance trade-off, identify a biased machine, identify a high variance machine, and be able to implement the bias variance trade-off in any machine.

What do you do if your machine is not giving desirable results? We have an entire set of options. We could:

1. Change features
2. Change target
3. Change machine
4. Change hyper parameters
5. Train longer
6. Change validation strategy

We could also use any combination of the points. Each of these and their combinations has several sub options that significantly increases the number of things we can do to fit our machine. An exhaustive search is practically impossible. Understanding the bias variance trade-off gives us a framework wherein our search is more focused. Generally speaking, understanding the bias variance trade-off is the first step in troubleshooting most machine learning problems.

Bias

As the name suggests, a biased machine is one that has a biased view of the outcome. At an extreme, the outcome does not even change with varying inputs. A machine such as the one defined in **Equation 6.1** for independent variable X taking values between $[0, 1]$ is

$$P(Y = 1 | X) = 0.6 + .00002 * X$$

Equation 6. 1

Here X is the only feature. This is a highly biased machine as the coefficient of X (.00002) is very small and the outcome is almost always very close to 0.6. Some machines are more prone to give biased results than others. For instance, there is a greater chance of getting a biased machine using logistic regression rather than decision trees. Why does this happen? The simplicity of the feature and the lack of an easily discoverable relationship can create a biased machine. In the relationship, the probability and X may not have a linear relationship that the machine is set up to capture. If we create a feature X^2 , we may find the coefficient of X^2 is larger and thereby the probability value may start getting less biased. Some machines may get biased due to heavy regularization.

Detecting a Biased Machine

The following two guidelines help us identify a biased machine

- The error rate on the training data is high - essentially implying that the machine has not been able to learn any information.
- The range of probabilities/values predicted by the machine are quite tight.

Reducing Bias of a Machine

- Add more nonlinearity in features.
- Reduce regularization.
- Add a more complex machine.

Variance

A machine suffering from a high variance problem is one that has learnt the training data well but gives very poor prediction on unseen (validation/test) data. As an analogy, consider a student who has studied the textbook very well, but finds it very difficult to apply those learning's in solving real-life problems.

Some algorithms are more prone to overfitting than others. As discussed, decision trees divide the feature space into rectangular regions and assign samples to each of these regions. If allowed to run long enough, they could divide the feature space to a very high level of granularity where there is only one sample per rectangular region, essentially creating zero-error on the training data. Very likely this machine will do quite poorly on validation data even though the features may only be slightly different.

If we increase the number of features and are not able to provide more samples, we may cause overfitting.

Detecting a High Variance Machine

- Performance is very good on training data but very poor on unseen data.
- Predictions change a lot with a small change in feature values.

How Do We Solve a High Variance Problem?

- Feature selection, dimension reduction techniques, and regularization.
- Reducing the run time.
- Reducing the complexity of the machine.

For illustrative purposes we present some simulations on training the XGBOOST algorithm on a sample dataset. This sample data set was a binary classification problem where 46% of the samples belonged to class 1 and 54% of the samples belonged to class 0. We are interested in predicting

samples that belong to class1. The idea here is to give a bit more of an insight into the bias variance trade-off.

We focus on three parameters of the XGBOOST algorithm namely learning rate (lr), number of trees (n_est) and max_depth. As the value of each of these parameters increases, the training error should reduce. For the set of values shown in figure 6.1, we have a high variance machine. Training accuracy of 70.4% is much higher than the validation accuracy of 46.6%.

lr	n_est	max_depth	validation_logloss	validation_acc	training_logloss	training_acc
0.05	300	4	1.013	0.466	0.797	0.704

Figure 6. 1 High Variance machine

To reduce this high variance, we try reducing the n_est and as shown in Figure 6.2. The overfit as measured by the difference between train loss/accuracy and validation loss/accuracy, has reduced. We also got a marginal improvement in the validation accuracy

lr	n_est	max_depth	validation_logloss	validation_acc	training_logloss	training_acc
0.05	200	4	1.008	0.469	0.852	0.657
0.05	100	4	1.011	0.465	0.917	0.599
0.05	50	4	1.007	0.471	0.958	0.568

Figure 6. 2

We try and reduce the overfit further by reducing the max depth to 2 as shown in Figure 6.3. It is also followed by a marginal increase in validation accuracy

lr	n_est	max_depth	validation_logloss	validation_acc	training_logloss	training_acc
0.05	50	2	1.010	0.476	0.991	0.500

Figure 6. 3

We then move the learning rate lower to .01 and further reduce the overfit as shown in Figure 6.4

lr	n_est	max_depth	validation_logloss	validation_acc	training_logloss	training_acc
0.01	50	2	1.053	0.473	1.049	0.482

Figure 6. 4

At this point we have probably moved too far into the high bias setup. We do some more parameter searches on max_depth and n_est and see the validation accuracy increase to 48.9 at the cost of allowing some variance to come in. As shown in Figure 6.5

lr	n_est	max_depth	validation_logloss	validation_acc	training_logloss	training_acc
0.01	200	4	1.013	0.489	0.961	0.548

Figure 6. 5

Summary

- Biased machines are opinionated and cannot even learn the trading set.
- A high variance machine learns the data set really well but finds it hard to generalize on unseen data.

CHAPTER SEVEN - APPLICATION: CARRY TRADING

Let us look at some practical applications of the tools and techniques we have learnt so far. In this chapter, we will address several important considerations on how to use Artificial Intelligence (AI) to trade. We will discuss pre-processing of data, feature engineering, hyper parameter optimization, risk management, and, most importantly, analysis of results to identify what the machine is doing well and what it is not.

By the end of this chapter you will be able to:

- 1) Understand the concept of carry trading and how it applies to commodities, equities, currencies and rates.
- 2) Understand all the building blocks that go into using AI to convert an economic concept into a trading algorithm.
- 3) Understand and appreciate several details addressed in earlier chapters.

There are several financial concepts that this approach could be applied to, which were discussed in Chapter 1. We chose to uncover “Carry Trading” here. Carry essentially refers to the yield that assets provide if you own them, after deducting the cost of funding their purchase. This net yield can be positive or negative, which means one could receive a net positive increment simply by owning the asset or may have to pay a net amount for owning the asset.

Carry in Foreign Exchange

Let us make this more concrete with a simple example from the foreign exchange markets. AUD versus USD. At the time of writing this chapter, AUD benchmark interest rate is at 1% and USD benchmark rate is at 2%. The AUD/USD spot exchange rate is 0.6771 USD per AUD.

The US central bank seems to be more hawkish on the US economy, while the Australian central bank holds a more dovish view on the Australian economy. Therefore, if we borrow AUD for a maturity of say a month and use the proceeds to buy USD, we need to pay 1% in AUD rates, as that is the currency, we have borrowed in. In return, we receive 2% on the USD holdings, as that is our asset. The yield on our asset (2%) is more than the cost of funding the purchase in AUD (1%). Therefore, the carry on this trade is positive.

Apparently, the market already prices this in. So, at that very instant, the 1-month forward rate for AUD/USD is .6777, which implies that the AUD spot rate 1 month from now is expected to be .6777. Therefore, the AUD is expected to strengthen from .6771 to .6777, which exactly offsets the net carry yield that we were expecting by buying USD and selling AUD. That is the way the market prices instrument as per efficient market hypothesis.

Effectively, if we borrow AUD and convert it to USD at a rate of .6771, we will earn 2% on the USD asset and pay 1% on our AUD liability. At the same time, we enter a commitment to repay our borrowing of AUD in one month's time. We intend to do this by selling our asset (USD) and using the proceeds to buy AUD, which effectively means we buy AUD and sell USD for 1-month maturity. The forward price will be such that the net gain in yield of 1% in holding USD over AUD for 1 month gets exactly offset. The forward price therefore will be $.6771 \times (1 + .01/12) = .6777$.

The hypothesis we want to test here is that the forward price does not accurately represent the gain or loss that may be incurred in holding AUD/USD for a month. We would like to see if there is a benefit of holding instruments that are net carry positive against a set of instruments that are net carry negative. If we can make money by consistently holding assets that are net carry positive against a set of instruments that are net carry negative, then we can conclude that the forwards do not accurately price future

outcomes. We obviously intend to use machine learning and some of the techniques we discussed earlier to prove/disprove this hypothesis. Before that, let us understand the concept of carry as applied to other asset classes. In foreign exchange markets, as discussed earlier, carry is simply the difference between the foreign interest rate and the domestic interest rate.

Assets that pay to be owned are positive carry. Assets that you need to pay to own, are negative carry. We want to see if owning positive carry instruments outperforms negative carry instruments.

One last insight that we would like to share is that interest rates are the “price” of money. If the price of a good is high, it means it is in demand and vice versa. We want to test if we systematically own assets, which are in demand, do we make money.

Let us see how carry is calculated in other assets.

Carry in Commodities

Most liquid commodities have at least two futures, which reflect the supply and demand conditions for those 2 months.

The futures (F) contract can be priced as in Equation 1

$$F = S_0 * (1 + (r_{\text{funding}} - r_{\text{commodity}}) * T/365) \quad \text{Equation 1}$$

Here T is the number of days to maturity, F is the futures price (first or second liquid contract), S_0 is the spot price (spot or first futures contract, respectively), r_{funding} is the interest rate of the currency the commodity is denominated in, and $r_{\text{commodity}}$ is the implied interest rate of the commodity (more commonly known as convenience yield). If $r_{\text{commodity}} > r_{\text{funding}}$, $F < S$ and the commodity is in *backwardation*. If $r_{\text{commodity}} < r_{\text{funding}}$, $F > S$ and the commodity is in *contango*. If $r_{\text{commodity}}$ is high, then it implies demand for that commodity is high and one should expect it to *outperform*.

Carry in Equity Indices

Most liquid indices are traded through futures contracts. Futures are priced similarly

$$F = S_0 * (1 + (r_{\text{funding}} - r_{\text{index}}) * T/365) \quad \text{Equation 2}$$

Here T is the number of days to maturity, F is the futures price (first or second liquid contract), S_0 is the spot price (price of the cash index), r_{funding} is the interest rate of the currency the index is denominated in, and r_{index} is the implied dividend yield.

Quite often, there is difference between the closing time of the cash index and the futures, so we need to be careful if we are using end of day data.

More appropriately we can use the actual cash dividends paid out by the companies forming the index and weigh them by the weights used to construct the index. We can also imply the dividend yield from “put call parity” relationships in the option market for the index futures, assuming sufficient liquidity.

Carry in Rates

The difference between short-term rates and long-term rates (2-year maturity vs. 10-year maturity) can be used as a measure of carry in the government bond markets.

In this chapter, we limit our focus to commodity futures. Let us do some pre-processing first.

Adjusting for the Roll

Our aim here is to create a continuous contract and adjust for the discontinuities in price which we see while rolling positions at expiry.

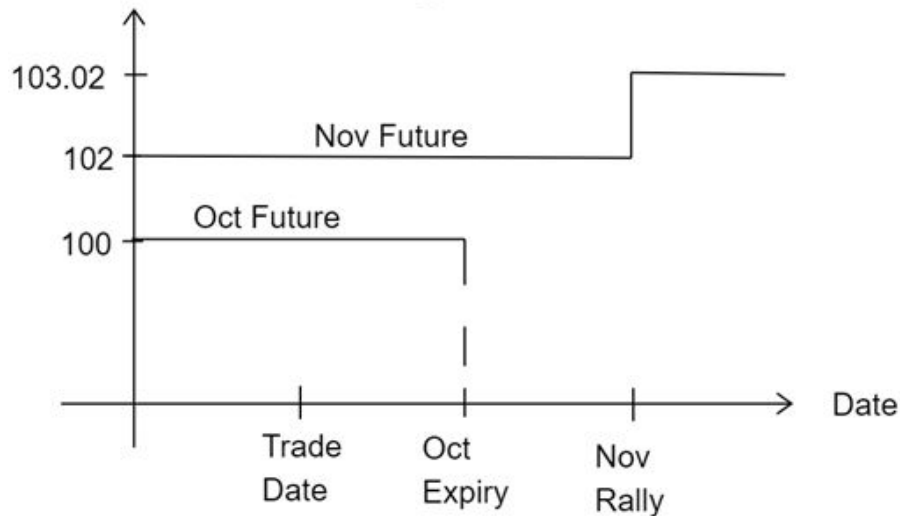


Figure 7. 1 OCT and NOV futures price action.

As shown in Figure 7.1, the **OCT** future for a commodity is trading at \$100, and the **NOV** future (the second active contract) is trading at \$102. For a \$1000 investment, we purchase 10 units of the **OCT** future. We keep it till the **OCT** contract matures and then we roll it to the **NOV** contract. Let us assume the price of the **OCT** and **NOV** contract at the time of rolling is still \$100 and \$102, respectively. After rolling we have 1000/102 units of the **NOV** contract. The price now rallies 1% to \$103.02 and we close our position. Our capital is now \$1010.

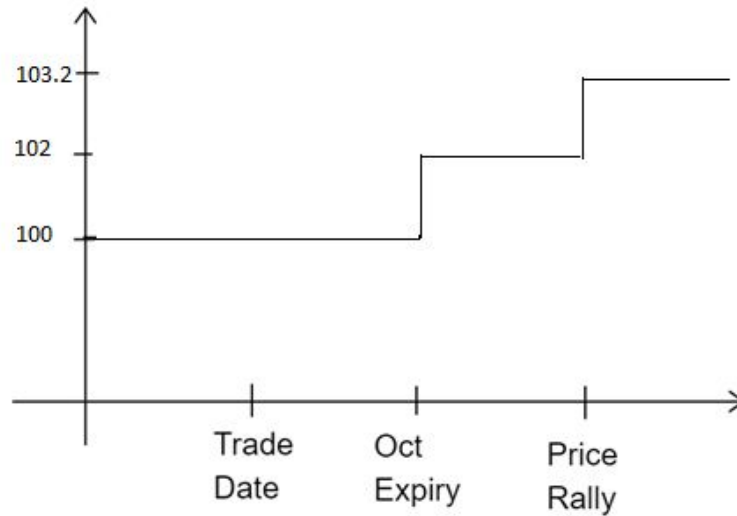


Figure 7. 2 Price of the generic first contract.

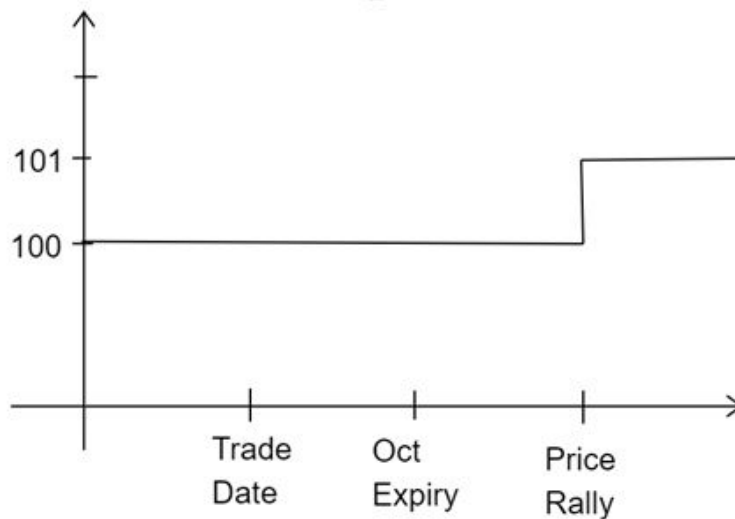


Figure 7. 3 Price of the generic first contract.

If we download the data from Bloomberg, for instance, the price of the generic first month contract would look like Figure 7.2a. Let us look at it another way. Our aim is to create a continuous contract. We do not want the hassle of the roll. We simply adjust the **Nov** contract by the $100/102 = 98.039\%$. Therefore, what was \$102 becomes \$100 (Figure 7.2b). In addition, when **Nov** rallied 1% to \$103.02, our modified **Nov** rallies to $103.02 \times 98.039 = \$101$. The benefit is obvious; we do not need to bother about the roll in our simulations.

We started with the commodity trading at \$100, we parked \$1000 of capital at \$100 and when the modified **Nov** contract moved to \$101, we liquidated our position.

Assets Under Consideration

Below is the list of commodity futures we would like to trade. As discussed earlier, we pre-process all the prices to account for rolls:

1. Corn (C 1)
2. Soybeans (S 1)
3. Copper (HG1)
4. Crude oil (CL1)
5. Bean oil (BO1)
6. Soybean meal (SM1)
7. Platinum (PL1)
8. Palladium (PA1)
9. Heating oil (HO1)
10. IPE gasoil (QS1)
11. Wheat (W1)

Data

We have used data from January 2000 to July 2019.

Features

Let us define a few variables. At this point, a snapshot of our data is shown in Figure 7.3, where each column contains the roll-adjusted asset price. W1, for instance, is the first active roll-adjusted contract for wheat futures, W2 in the second active roll-adjusted wheat contract, and so on.

A	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT
Date	W1	W2	W3	C1	C2	C3	S1	S2	S3	CL1	CL2	PL1	PL2	HG1	HG2	BO1	BO2	OS1	OS2	SBI	SBI2	PA1	PA2
1/3/2000	247	258	267.5	201	208	214.75	465	471.5	478	25.5	24.8	414.8	405	84.85	85.55	15.69	16.01	213	204.5	6.13	6.27	442	442
1/4/2000	247.5	258	268	203	210	216.25	472	478.5	485	24.9	24.3	402.5	399	85.65	86.4	15.83	16.14	213	205.25	5.77	5.99	437	437
1/5/2000	250	260.25	271	203.25	210.25	217	477	484	491.5	24.7	24.1	394.5	384	85.35	86.1	15.95	16.27	211.25	205	5.8	6.04	436.8	433
1/6/2000	248.5	259.5	269.5	204	210.75	217.75	475.75	483.5	490.5	24.2	23.7	396.5	394	85.4	86	16.06	16.37	205	199.75	5.79	6	444	441
1/7/2000	252	261.5	273	207	214	221	479.25	487.5	493	24.6	24.2	399	395	84.7	85.5	16.21	16.5	195.75	193.5	5.88	6.07	452	445
1/10/2000	253.5	264.25	274.5	208.5	215.75	222.25	474	482	488	25.71	25.05	398	392.5	84.4	85.3	15.99	16.31	201.5	198.25	5.83	6.06	450	445
1/11/2000	253.75	264.25	274.75	207	214.5	221.75	475	482.5	489	26.28	25.55	402.9	392.5	86.65	87.35	15.78	16.11	201.5	205	5.67	5.89	453	445
1/12/2000	259	269.75	280	215.25	222.75	229.75	480	488.25	495.75	26.74	25.98	405	399	85.7	86.55	15.93	16.25	204.449	208.327	5.45	5.71	447	445
1/13/2000	263.25	274.25	284.5	220.75	228.5	235.5	489.75	498	506	27.9	27.05	410	406	86.2	87.05	16.4	16.74	210.101	213.702	5.47	5.78	441	445
1/14/2000	265.5	275.5	286	219.25	227	234.5	497	503.75	512.5	28.7	27.68	416	406.5	86.6	87.5	16.3	16.63	216.795	220.1	5.44	5.7	439	437
1/18/2000	264.25	274.75	286	220.5	228.25	235.5	503	511	519	29.68	27.99	418.5	415	88.3	89.1	16.56	16.8	217.227	221.635	5.29	5.5	443	440
1/19/2000	266.5	277	287.5	221	228.75	236.25	501.5	508.5	519	29.15	27.8	424	416	87.3	88.2	16.33	16.65	217.227	221.635	5.5	5.69	451.8	446
1/20/2000	265	275.75	286	221	228.5	235.5	505.25	514.5	522.5	29.5694	27.8514	427.5	420	87.35	88.05	16.27	16.56	223.862	224.707	5.54	5.77	457.5	451
1/21/2000	264.5	275.75	284.5	223	230.75	238.25	505.25	514	522	29.171	27.7281	436.3	422	85.7	86.6	16.18	16.46	231.234	230.337	5.45	5.68	470.5	460
1/24/2000	266.25	277	287.5	227.25	235	242.25	511.5	520.5	529	29.6323	28.0158	442.5	432	84.3	85.2	16.44	16.72	221.404	224.451	5.3	5.51	479	476
1/25/2000	264.75	276.5	285.25	227.5	235.5	242.5	516.5	525.75	534	29.15	27.615	432.5	422	84.2	85.2	16.47	16.78	222.141	225.73	5.33	5.51	472	473.5
1/26/2000	260	271.25	281.5	223.25	230.5	238	513.25	522	530.5	28.6152	27.0292	446	433	84.5	85.4	16.49	16.8	223.616	226.754	5.41	5.56	483	473.5
1/27/2000	263.25	274.25	284.5	225.5	233.5	240.5	527	535.25	544.5	28.4684	27.132	454.2	443	83.75	84.45	17.15	17.43	218.21	222.659	5.4	5.57	486	482
1/28/2000	259.5	270.75	280.75	223.5	231.25	238.75	523.25	532.5	541.25	28.8879	27.4609	455	444	85	85.85	17.05	17.33	214.278	219.076	5.46	5.59	485	482
1/31/2000	256.5	268	278	219.75	227.5	234.5	508.5	517.75	526.75	29.5485	27.9542	453.5	442.5	84.4	85.35	16.57	16.87	215.752	220.356	5.44	5.55	493	486
2/1/2000	258.25	269	279.75	222.25	230	237.25	510.75	520	528.5	28.8354	27.3375	467	455	82.3	83.3	16.49	16.77	217.473	222.915	5.38	5.51	503	498
2/2/2000	257.25	268.5	279	220.75	228.75	235.75	508	516.25	525	29.4646	27.8	475	461	82.65	83.7	16.21	16.5	218.455	222.659	5.51	5.65	515	513
2/3/2000	253.5	264.75	275	218	225.75	234	500.25	509.75	517.5	30.0937	28.5194	480	469	82.85	83.9	15.95	16.26	223.616	227.522	5.61	5.76	528	518
2/4/2000	257.5	269	278	220	227.5	235.25	505.5	514.25	523	29.7477	28.2008	475	465	82.7	83.8	15.79	16.11	221.65	226.498	5.8	5.93	541	535
2/7/2000	258.5	271	281.5	221.75	230	237.75	507	516.5	524	29.3597	27.8	500	485	82.05	83.1	15.98	16.28	223.37	228.29	5.7	5.82	580	579
2/8/2000	259	270.5	280.75	221.25	229.25	237.5	499.75	508.5	518.75	30.1461	28.5194	530	509.5	82.2	83.2	15.76	16.08	218.21	223.171	5.53	5.69	591	588
2/9/2000	265.5	277.5	287.75	222.75	231	238.75	503.5	513.25	521.75	30.8487	29.0127	518.5	507	85	86	15.8	16.1	216.981	222.915	5.51056	5.68044	580	569
2/10/2000	271	281.5	293.5	225.75	234.5	241.75	514	523	531.5	30.7753	29.1566	527	516	84.6	85.65	15.9472	16.2471	217.718	220.612	5.50084	5.69956	599	596

Figure 7. 4 Data set.

Ret: Log return of prices.

Carry: $(r_{\text{funding}} - r_{\text{commodity}})$

Based on the formula in **Equation 3**

$$F = S_0 * (1 + (r_{\text{funding}} - r_{\text{commodity}}) * T/365)$$

Equation 3

We calculate $(r_{\text{funding}} - r_{\text{commodity}})$ simply by inverting the equation, which gives us

$$(r_{\text{funding}} - r_{\text{commodity}}) = (F/S_0 - 1) * 365/T \quad \text{Equation 4}$$

Which, in the case for wheat, turns out to be

$$(r_{\text{funding}} - r_{\text{commodity}}) = (W2/W1 - 1) * 365/T$$

We call $(r_{\text{funding}} - r_{\text{commodity}})$ to be carry. The higher the $r_{\text{commodity}}$, the lower the carry, the higher the demand for that commodity.

standard deviation:

We also calculate the 20-day standard deviation of the returns, σ_{ret} .

carry vol ratio:

We define the carry-vol ratio = $\text{carry}/\sigma_{\text{ret}}$.

As features, we give carry-vol calculated for the previous 2 weeks as an input.

Target

We treat this as a classification problem, and therefore there are several ways we could define the target. For simplicity, we define it as the following:

Target is:

1 if the next day return > 0

-1 if the next day return < 0

0 if the next day return = 0

Note that we have defined the target for day 1 to be **dependent** on day 2's return. Day 2's return, however, is $\log(\text{price on day 2} / \text{price on day 1})$. Admittedly, there is some error introduced in doing it this way; it assumes we can trade on the close of day 1, which is not entirely true as the model requires the close as an input to calculate the feature and it will take some time to run and calculate the signal. To avoid this, define the returns as $\log(\text{close on day 1} / \text{open on day 1})$, which implies we are expected to trade at the open of the next day.

Train/Validation/Test Data Sets

We train the model on 500 days of data; we test it on 50 days.
An appropriate validation strategy (as shown in fig 7.5, 7.6 & 7.7) for time series as in this case is presented as follows: We build three models on three different validation data sets.

Model 1:

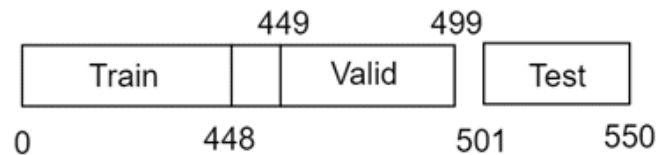


Figure 7. 5 Validation strategy that creates Model 1.

Model 2:

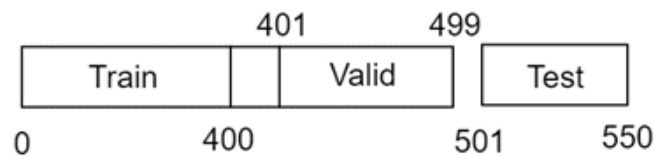


Figure 7. 6 Validation strategy that creates Model 2.

Model 3:

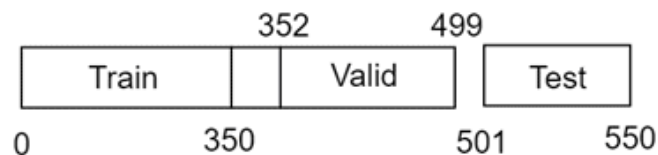


Figure 7. 7 Validation strategy that creates Model 3.

Machine:

We have used the random forest classifier.

The **hyper parameters** that we have searched for are the following:

max_depth:

[1, 2, 3, 4, 5, 6]

max_features:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]: between 1 to the maximum number of features provided as an input

min_samples_leaf:

[1, 3, 5, 7, 9, 11]

Number of estimators: 50

The overall approach is illustrated in Figure 7.8.

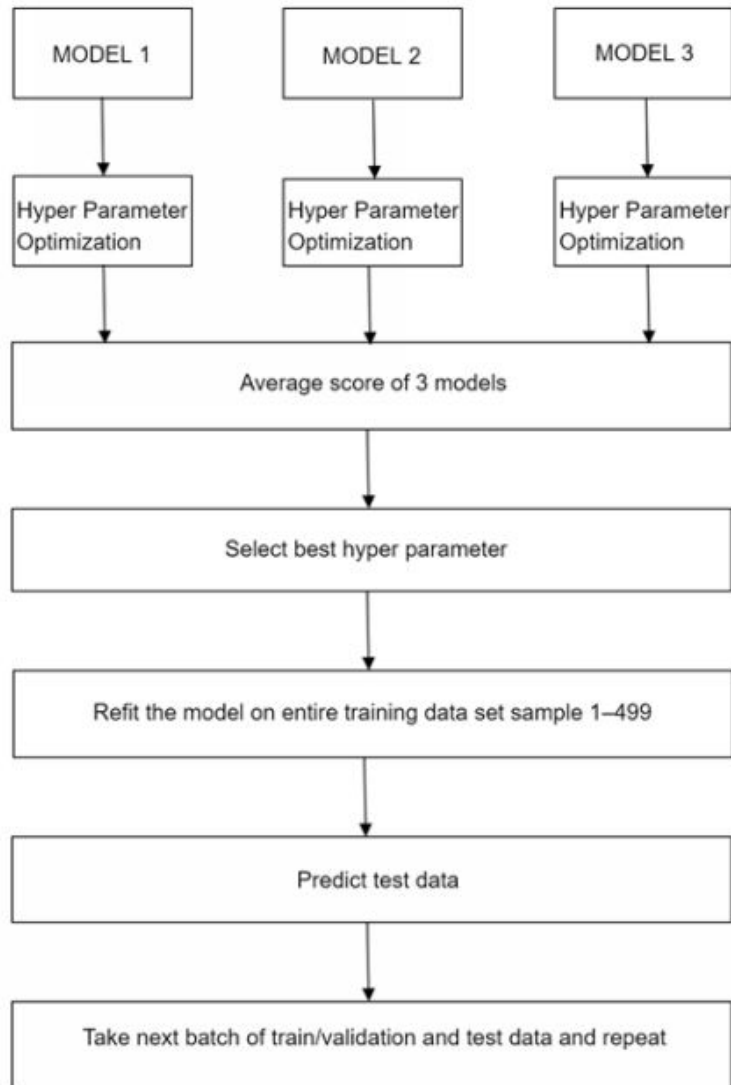


Figure 7. 8 The general flowchart of building models.

The idea behind doing the validations is to get the best set of hyper parameters based on a certain metric (roc auc score in this case). Once we have the best set of hyper parameters, we retrain the full model on sample numbers 1–499. One detail that we want to highlight is that we have deliberately dropped one sample at the end of training/validation sets. Details on this were discussed in Chapter 3 - the chapter on validation strategy.

Result

What we get after doing this entire exercise is a probability that an asset on a particular day will go up, go down, or remain unchanged.

A snapshot of the result would look like this.

asset	date1	Prob Dn	Prob Up
C1	2/12/2002	0.530683	0.469317
C1	2/13/2002	0.525383	0.474617
C1	2/14/2002	0.536554	0.463446
C1	2/15/2002	0.491026	0.508974
C1	2/18/2002	0.52618	0.47382
C1	2/19/2002	0.536554	0.463446
C1	2/20/2002	0.532293	0.467707
C1	2/21/2002	0.525383	0.474617
C1	2/22/2002	0.537824	0.462176
C1	2/25/2002	0.531332	0.468668
C1	2/26/2002	0.515993	0.484007
C1	2/27/2002	0.532249	0.467751
C1	2/28/2002	0.5515	0.4485
C1	3/1/2002	0.53496	0.46504
C1	3/4/2002	0.557578	0.442422
C1	3/5/2002	0.547528	0.452472
C1	3/6/2002	0.545074	0.454926
C1	3/7/2002	0.55262	0.44738
C1	3/8/2002	0.527395	0.472605

Risk Management

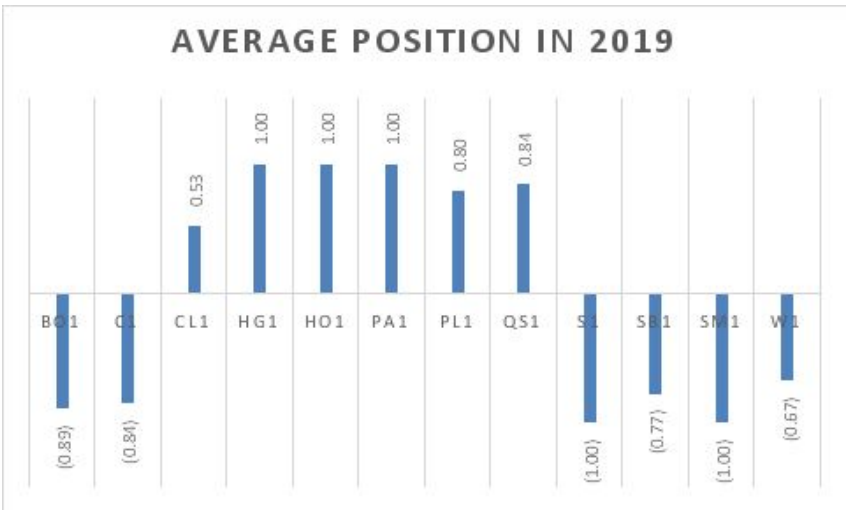
A simple risk-management rule can be deployed from here to see the \$ Profit and Loss.

We rank all assets based on the probability of the asset appreciating. Go long the top six and short the bottom six at the close of **the next day**. Close the position the following day.

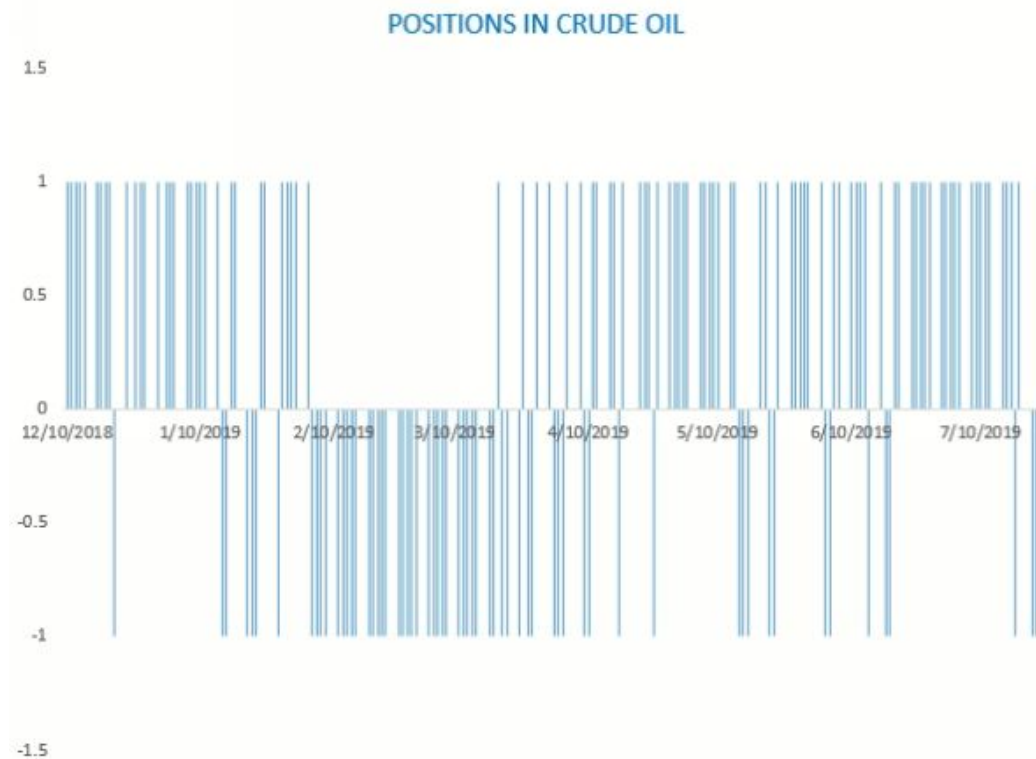


Figure 7. 9 Equity curve.

The equity curve is shown in Figure 7.8. Some key observations are as follows.



1. It is heartening to see the strategy was able to be long palladium and short agricultural markets. Palladium has been in significant backwardation and has had a stupendous rally. Agricultural markets have been in sizable contango.
2. The MOOT POINT is that The AI was able to learn, from data alone, an economic principle that, if the cost of borrowing a commodity is high, then there is upward pressure on price.



3. The AI was sitting long crude in Jan 2019 when the market rallied 20%. That is great, but it went short in February and long in May, which was not very smart. What can we do to change that?
4. The AI did well to be long corn in from 15th to 31 May 2019 and was short for most of July 2019

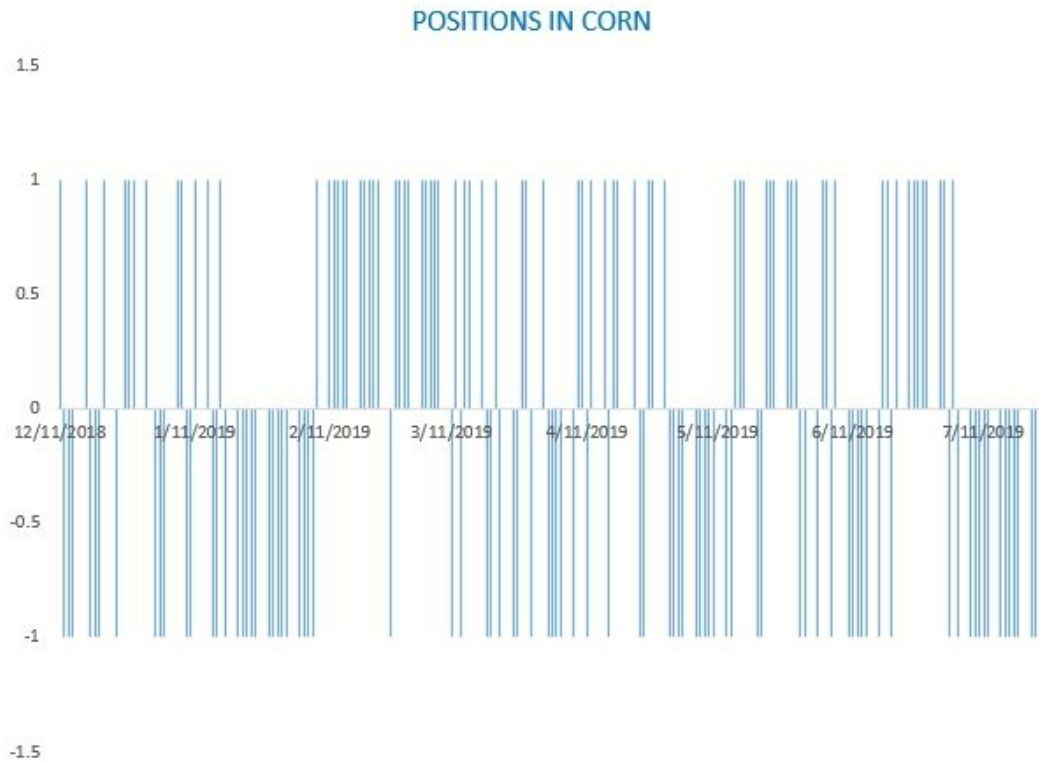


Figure 7. 10 Price of Corn (“Source: Bloomberg Finance L.P. Used with permission of Bloomberg Finance L.P.”)

Summary

- AI was able to discover from data itself a trading strategy based on economic principles.
- In this chapter, we give a complete rundown on the steps needed to create and test a trading strategy based on AI.

References

- Christian L. Dunis & Jia Miao (2007) Trading foreign exchange portfolios with volatility filters: the carry model revisited, Applied Financial Economics, 17:3, 249-255, DOI: [10.1080/09603100500447578](https://doi.org/10.1080/09603100500447578)
- Daniel, Kent & Hodrick, Robert & Lu, Zhongjin (2017). The Carry Trade: Risks and Drawdowns. Critical Finance Review. 6. 211-262. 10.1561/104.00000051.
- ECB working papers series: From carry trades to curvy trades

CHAPTER EIGHT - SUPPORT VECTOR MACHINES

SVM are supervised learning algorithms for classification and regression problem solving. We will start with some introductory linear algebra concepts to help us understand the maximal margin classifier. This will then be extended to SVM with nonlinear kernels.

After a quick recap of some linear algebra, we will discuss hyper planes and the maximal margin classifier. This is a very restrictive model, and, in this chapter, we will relax some of the constraints to see how we can get a more powerful model.

Basic Linear Algebra

Dot Product

Let us say we have a vector u in two-dimensional coordinate system (x,y) as in fig 8.1

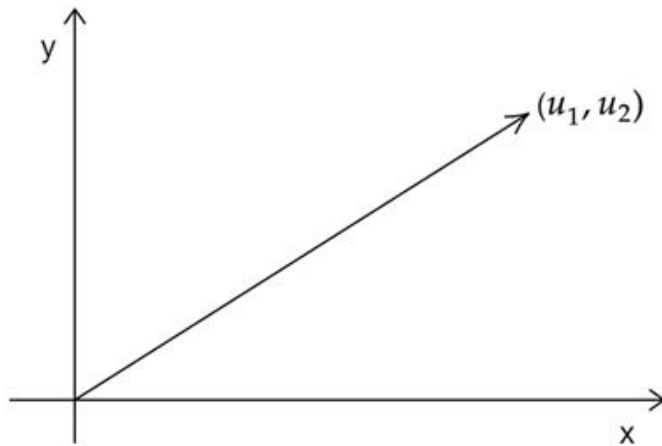


Figure 8. 1 A vector U in a Cartesian coordinate system.

The dot product of u with another vector v (Fig 8.2) is given by

$u \cdot v = u^T v = \|u\| \|v\| \cos \theta$ where θ is the angle between the two vectors.

The term $\|u\| \cos \theta$ is projection of vector u along vector v

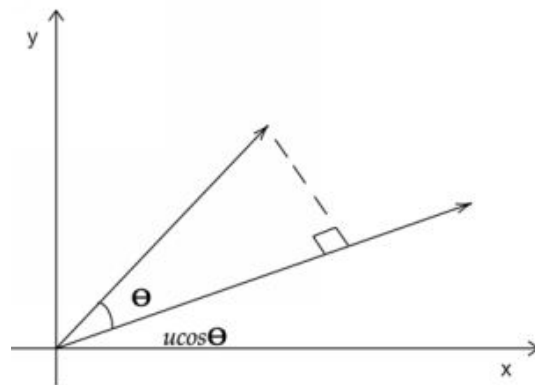


Figure 8. 2 Dot product of vectors u and vector v .

Perpendicular distance of a point from a line

Let us say we have an equation of the form

$$\beta^T X + \beta_0 = 0$$

The perpendicular distance of a point u from this line is given by

$$d = \text{Abs}(\beta^T u + \beta_0) / \|\beta\|$$

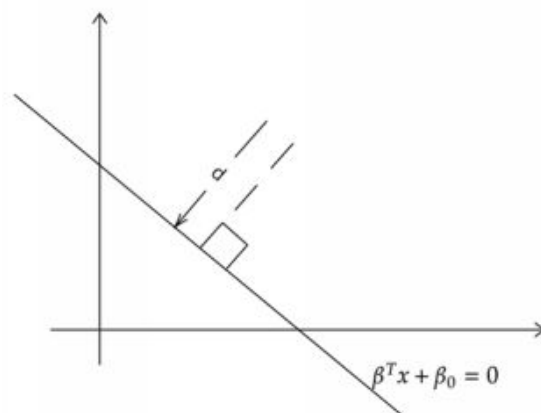


Figure 8. 3 Perpendicular distance of a u point from the line.

Another important consideration is that vector β is normal to line $\beta^T X + \beta_0$

Let us say we have 2 vectors u and v that lie on line L . vector $u-v$ (fig 8.4) is parallel to the line.

$$\beta^T u + \beta_0 = 0$$

$$\beta^T v + \beta_0 = 0$$

$$\beta^T (u-v) = 0$$

$$\beta \cdot (u-v) = \|\beta\| \|u-v\| \cos \theta = 0$$

$$\theta = 90 \text{ degrees}$$

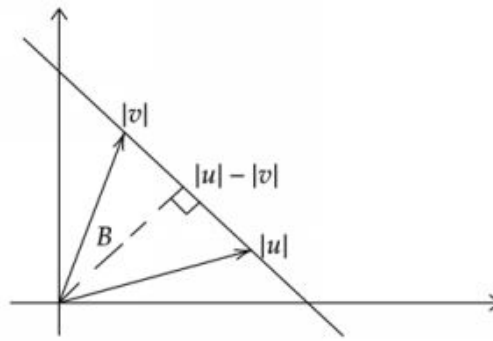


Figure 8. 4 Vector $u - v$ is perpendicular to vector β .

That should be the bare minimum amount of linear algebra we would require to understand SVM's. Let us start using this information to understand the SVM.

Classification

As you probably know by now, a classification problem is defined as below in Figure 8.5

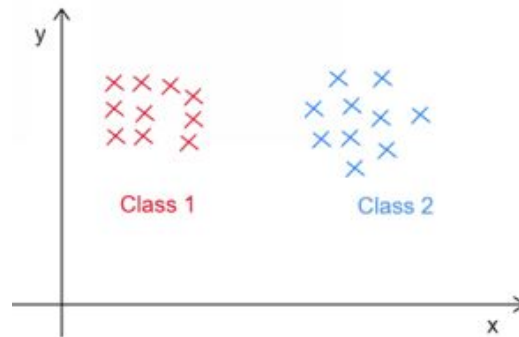


Figure 8. 5 A typical classification setup.

We have two ($p=2$) features X and Y and two classes class1 and class 2 represented by the red and blue crosses respectively . We will call the target variable T , Class1 samples have a target of 1 ($T=1$) and class2 have a target of -1 ($T=-1$)

The idea is to find a one ($p-1$) dimensional hyperplane (line in this case) that would perfectly separate all samples.

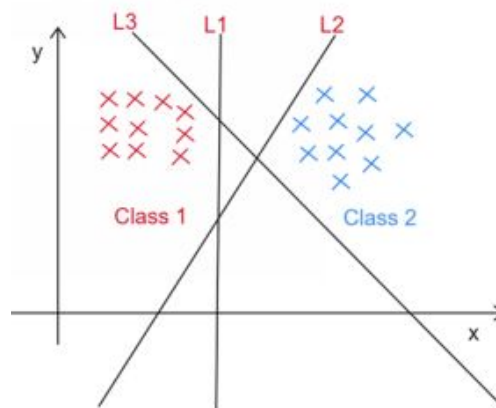


Figure 8. 6 Several possible ways to separate the two classes

As Figure 8.6 shows there are several possible solutions. Let us get the intuition behind the uniqueness of the solution that we are after. We start by

computing the perpendicular distance from all the samples to the line (any of the $L_1, L_2, L_3, \dots$ lines, we chose L_1 in this case) (Figure 8.7).

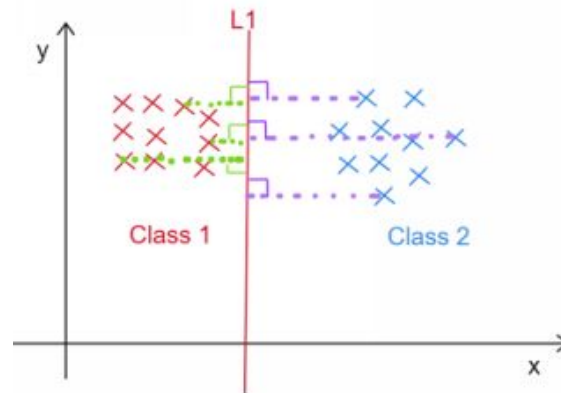


Figure 8. 7 calculate perpendicular distance from the samples of each class to L_1

For each class we identify only those samples which have the smallest such distance, as shown in Figure 8.8 (NOT drawn to scale).

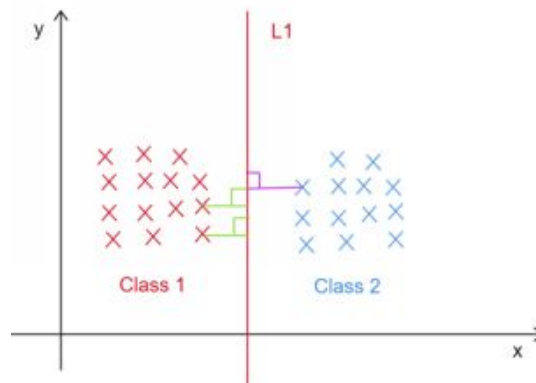


Figure 8. 8 Minimum distance for each class to this line.

And based on this, we draw the margin as indicated by the dotted lines Figure 8.9. This distance we denote by M .

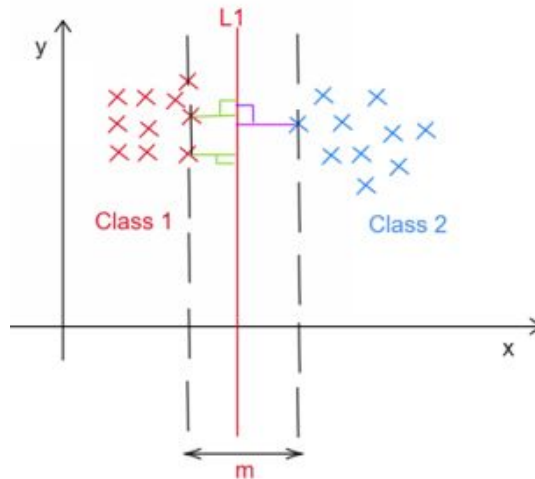


Figure 8. 9 Margin M for $L1$

We repeat this for all possible sets of lines, so for $L2$ it would look something like Figure 8.10 below

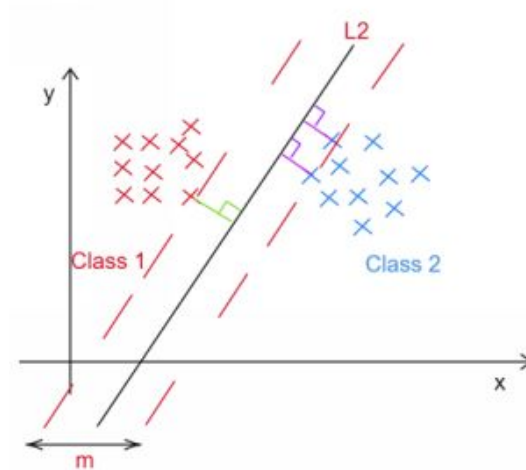


Figure 8. 10 Margin M for $L2$

We select the line that has the highest value of M , so between Line $L1$ and $L2$ we will select Line $L1$ as it has a higher margin (M)

The important considerations are:

- We want to select a linear classifier

- The classifier of choice should not only perfectly classify all the samples but should allow for a maximum margin (M) of error
- The samples nearest to the line determine the shape of the line; these samples are called support vectors. The samples further away from the line have less of an influence in defining the line.

Now that we know how to select the line that separates our two classes, let us look at formalizing our understanding.

Maximal Margin Classifier

So how do we classify the sample (u) once we have the line? We simply take the sign of $\beta^T u + \beta_0$.

If greater than 0, then it belongs to class 1 with a target value $T = 1$.

If less than 0, then it belongs to class -1 with a target value $T = -1$.

Effectively $T^*(\beta^T X + \beta_0) > 0$

What follows from here is a little more mathematical representation of the above intuition and how to generalise the search to a higher dimensional problem.

Let us start by defining the variables:

We have P features and n samples.

The i^{th} sample has $x_{i1}, x_{i2}, x_{i3}, \dots, x_{ip}$ features

The i^{th} sample has y_i , target

The classifier is given by

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} \dots \beta_p x_{ip} = 0$$

M ofcourse is the margin

So the **maximum margin classifier** is obtained by

Finding values of $\beta_0, \beta_1, \beta_2, \dots, \beta_p$

1) For which we maximize M

2) Such that $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} \dots \beta_p x_{ip}) \geq M$ for all the n samples ----**equation 2**

3) And $\beta_1^2 + \beta_2^2 + \beta_3^2 \dots + \beta_p^2 = 1$

The solution to the above set of equations gives us the maximal margin classifier

The reason we have 2) and 3) is because

$(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} \dots + \beta_p x_{ip}) / \sqrt{\beta_1^2 + \beta_2^2 + \beta_3^2 \dots + \beta_p^2}$ represent the perpendicular distance from a sample to a line, as discussed in the section on Linear Algebra.

We multiply by y_i because we want it to be correctly classified.

So 2) and 3) together imply that each sample needs to be correctly classified and at least M units away from the line. And then we select all the possible solutions that has the highest value of M . In other words, we want a linear classifier that perfectly separates all our samples. The best of bias and variance machines. Achieving this in reality though, is hard to satisfy. Let us see how we can relax some of these stringent requirements.

Misclassification, Non Linearities and the Kernel trick

Allowing Misclassifications

The reality of data sets may be a bit different. To begin with equation 2) above requires ALL samples to be correctly classified. That is too stringent. We may not be able to get a linear classifier that perfectly separates all samples. Figure 8.11 is an example of this:

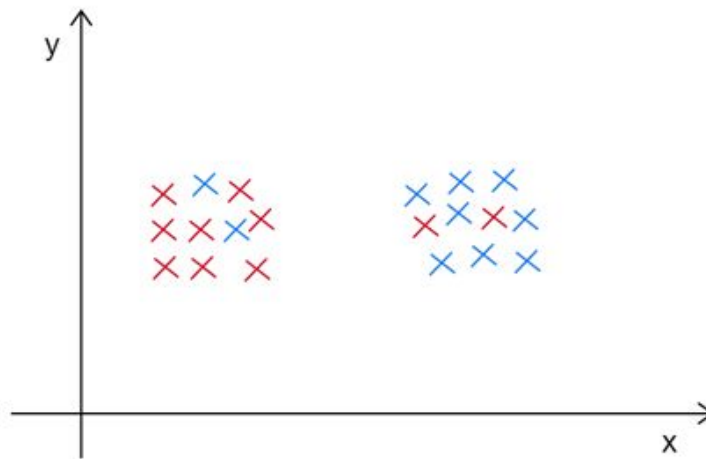
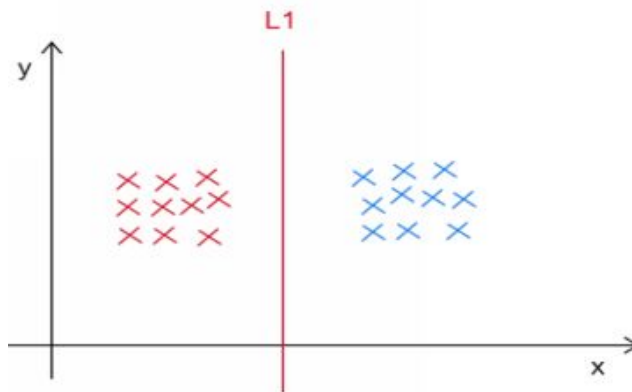


Figure 8. 11 Not possible to construct a linear classifier that perfectly separates the two classes.

Alternatively, we may not want a classifier that perfectly separates all samples, to avoid the chances of overfitting. For example, in the two solutions in Figure 8.12, the presence or absence of one sample can cause a significant change in the solution if we always desire to classify all samples.



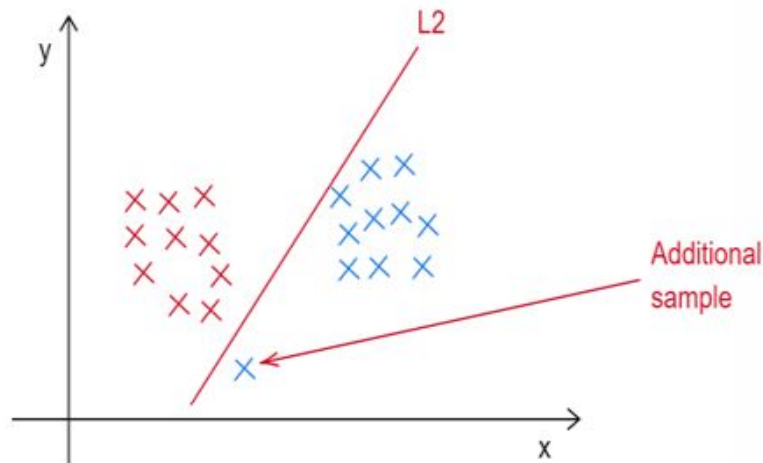


Figure 8. 12 Perfect classification on the training data may be a sub-optimal goal to achieve.

Either way, we may want to allow for some amount of misclassification of samples.

The way we account for that is by modifying equation 2 to

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} \dots \beta_p x_{ip}) \geq M(1 - e_i)$$

where $e_i > 0$, and

$$\sum (e_i) \leq C \text{ for } i = 1, 2, 3, \dots, n$$

If $e_i = 0$, then sample is correctly classified

If $e_i < 1$, then sample is correctly classified but the margin condition is violated

If $e_i > 1$, then sample is incorrectly classified

As discussed earlier, the solution to the above set of equations is dependent only on the support vectors. When we solve the equations using optimization techniques (beyond the scope of this books and quite frankly us), we get an equation of the form

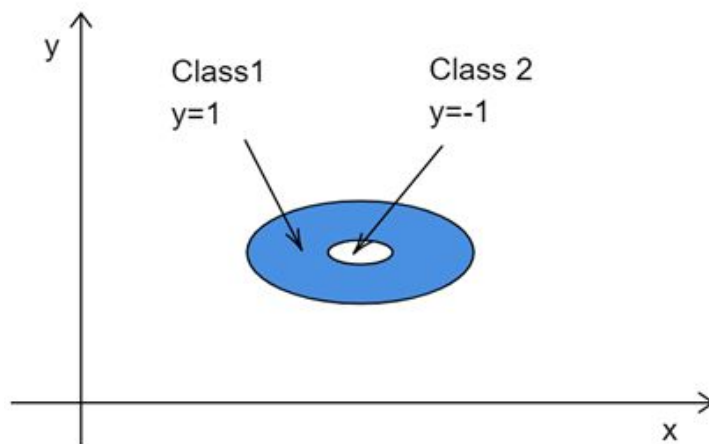
$$\beta_0 + \alpha_1 x \cdot x_1 + \alpha_2 x \cdot x_2 \dots + \alpha_s x \cdot x_s$$

where there are s support vectors and $x \cdot x_1$ is the dot product of two p -dimensional vectors (where p is the number of features) and x_1 is the first support vector, x_2 is the second support vector and so on.

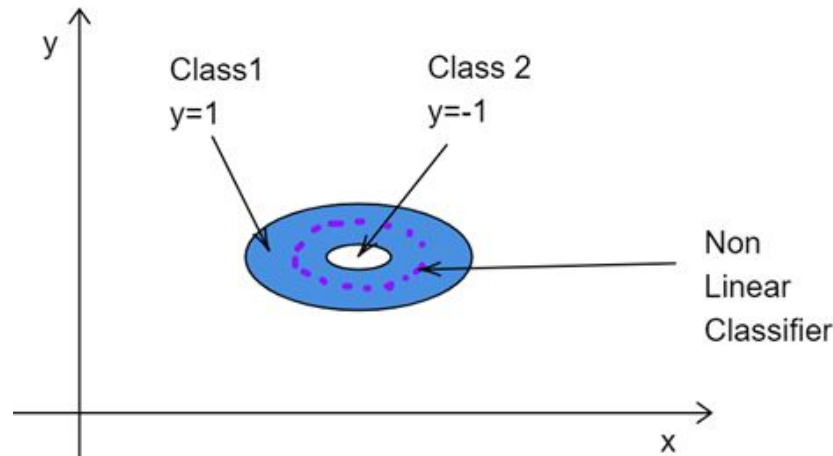
Now that we understand this, we move on to provisions in our algorithm to accommodate misclassifications.

Introducing Non-Linearity

The other overly restrictive condition is that the classifier needs to be linear. Let us say our dataset looks something like this:



Forcing a classifier to be a linear hyper plane (straight line in this case, as we have two-dimensional features) is obviously incorrect; the natural classifier is nonlinear, something like this:



The way we adapt the algorithm is the following, first the intuition.

Imagine pulling the red eclipse (class $y=-1$) out of the book and leaving the blue eclipse (class $y=1$) on the book. That way we convert a two-dimensional representation to a three-dimensional ($p=3$) representation. At this point, we can visualize the existence of a two- $(p-1)$ dimensional plane that classifies the two classes; this classifier is essentially the linear hyperplane now.

All that we need to do to modify the equations is:

Finding values of $\beta_0, \beta_1, \beta_2 \dots \beta_p, \beta_{12}, \beta_{22} \dots \beta_{p2}$

1. For which we maximize M
2. Such that $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} \dots \beta_p x_{ip} + \beta_{12} x_{i1} + \beta_{22} x_{i2} + \beta_{32} x_{i3} \dots \beta_{p2} x_{ip}) \geq M$ for all the n samples
3. And $\beta_1^2 + \beta_2^2 + \beta_3^2 \dots + \beta_p^2 + \beta_{12}^2 + \beta_{22}^2 + \beta_{32}^2 \dots + \beta_{p2}^2 = 1$

Some texts have the optimization stated in a slightly different way. Let us try and understand the equivalence.

The alternative statement of the problem is

Minimize $\beta_0^2 + \beta_1^2 + \beta_2^2 \dots + \beta_p^2$

$$\text{St } \beta^T x_i \geq 1 \text{ if } y_i = 1$$

$$\beta^T x_i \leq -1 \text{ if } y_i = -1$$

As stated at the beginning of this chapter, vector β is normal to the line $\beta^T x$. And

$$\beta^T x_i = \|\beta\| \|x_i\| \cos \theta \geq 1$$

$\|x_i\| \cos \theta$ is the projection of vector x_i along vector β , so $\|x_i\| \cos \theta$ is along a vector, which is perpendicular to the classifier $\beta^T x$. Since our optimization objective is to minimize $\|\beta\|^2 = \beta_0^2 + \beta_1^2 + \beta_2^2 + \dots + \beta_p^2$, it is the same as saying the objective is to maximize $\|x_i\| \cos \theta$. This is essentially stating that we need to select a classifier that maximizes $\|x_i\| \cos \theta$, the perpendicular distance of the sample to the classifier.

We have now seen that one of the ways to introduce nonlinearity is to introduce nonlinear features. A more robust method is the kernel trick.

What we do in the kernel trick is to take each sample, pass it through a kernel transformation and get a new representation of the sample. We then apply the above discussed algorithm on these transformed features.

So, what's a kernel? A kernel is simply a function. It could take any form but for what we are discussing it quite often takes the below form.

$$K(x, x_i) = e^{-(x - x_i)^T (x - x_i) / 2(\sigma^2)}$$

As x approaches x_i , $K(x, x_i)$ approaches 1

As x is very different to x_i , $K(x, x_i)$ approaches 0

So for a particular sample (x_u, y_u) , we calculate $K(x_u, x_i)$ for all values of i from 1 to n , where there are n samples. So x_u , which was originally p -dimensional, is now n -dimensional in the feature space. We represent all the n samples from the original data set as these n -dimensional features. What was

originally $n \times p$ is now $n \times n$. We then apply the SVM algorithm to this feature space.

One can use the same features as used with trees and train an SVM classifier as below

```
from sklearn.svm import SVC  
svc = SVC(gamma='auto')  
rf.fit(X,y)
```

Comparing different classifiers for the same input is often a good idea. For a given set of features it is recommended to run a grid search of the hyperparameters to get the best model. SVM usually outperforms other classifiers when the number of features is much higher, but the data is available is low.

Summary

We started this chapter with some basics on Linear Algebra, and we went on to discuss an ideal and overly restrictive classifier 'Maximal Margin Hyper plane'. We then relaxed some of the restrictions to build the SVM.

References

- <http://appliedpredictivemodeling.com/>
- <http://faculty.marshall.usc.edu/gareth-james/ISL/>

CHAPTER NINE - DEEP LEARNING

The use of machine learning is increasingly becoming synonymous with the use of neural network or more specifically, with deep learning. In this chapter, we will explore the workings of deep learning models in detail. To develop an intuitive understanding of the workings of a neural network, we will be exploring the use of neural networks for tackling supervised learning problems. We will break the network down to its components so that you, the reader, can play around and build your models to suit your specific requirements. In other words, we will try to demystify the black box, as these models are commonly referred to, layer by layer, component by component.

A deep learning architecture is a logical, structured, sometimes complex aggregation of some relatively simple building blocks. Let us open the structure and see why it has become the go-to-architecture for seemingly difficult problems and what makes it so powerful.

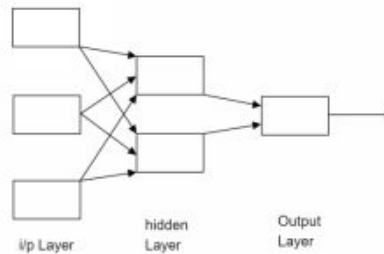


Figure 9. 1 Neural network basic architecture.

A neural network, as the name rightfully suggests, is a network of neurons that operates as a two-pass architecture. An example of a very simple network is provided in Figure 9.1. The first pass that proceeds from input to output is known as the forward propagation. The second pass traces the reverse path from the output to the input and is known as the backward propagation.

The idea is to quantify the relationship between the input features and the target outputs. In the first pass that comprises a forward propagation, a nonlinear function is used to derive the value of target output(s), using the

input values. A loss function is calculated to find the error between the actual and the predicted value of the output. The network aims to minimize this loss by finding the global minima of the loss. In the second pass, the backward propagation, the parameters (weights and biases) are tweaked in a systematic way to try and minimize this loss. The rate at which the biases and weights are changed is determined by a hyper parameter called the learning rate. The forward pass and the backward propagation are, often, repeated a few times for a step-by-step systematic move toward the global minima. The rate at which we can converge to the global minima is a function of the structure, data, and many other hyper parameters. This will be discussed in more detail in this and future chapters.

Neuron

A neural network is, as the name rightfully suggests, a network of neurons. Each neuron is a specialized unit of computation that does a two-step operation during the forward pass phase to convert an input to a processed output. Depending on the specific architecture, each neuron passes its output to one or more sets of neurons as their inputs. These neurons, in turn, use these inputs and perform a “similar” two-step operation to pass the processed output along further in the network.

Let us say the input to the neuron is “ X ” and the output of the neuron is “ Y^* .” Each neuron is associated with a few parameters (which the network learns during the process of training. More on that later). The parameters are called weights “ W ” and bias “ b .” These parameters appear in step 1 of the two-step process undertaken by each neuron (Figure 9.2).

Step 1: Calculates a value, let us call it Z , such that

$$Z = W^t X + b$$

The superscript “ t ” essentially implies that we need to take the transpose of the W matrix. We multiply or rather perform a dot product of the transpose of the W matrix and the input X . We then add the bias to this value to generate our intermediate output, Z .

Step 2: It involves applying a nonlinear function to the intermediate output Z to get the output Y^* . The nonlinear functions are known as activation functions, $A(Z)$.

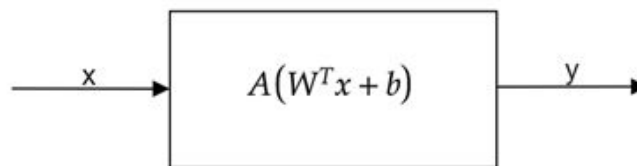


Figure 9. 2 Two steps, step 1 and step 2 that are performed in every neuron.

Why do we need these activation functions to be “nonlinear”? While we do have a linear activation function, as described ahead, it is rarely used

exclusively in a network. The reason why we need to introduce nonlinearity is to be able to approximate all possible relations or, mathematically speaking, all possible functions that link our input to our final desired output. The ability to do this mapping from any input to output is called universal function approximation. In other words, it is this nonlinearity that helps create the universal function approximator - hence providing the ability to the neural learning system to predict variables in multiple domains.

Activation Functions

Some of the more common activation functions are as follows:

Sigmoid

$$A(Z) = 1/(1 + e^{-Z})$$

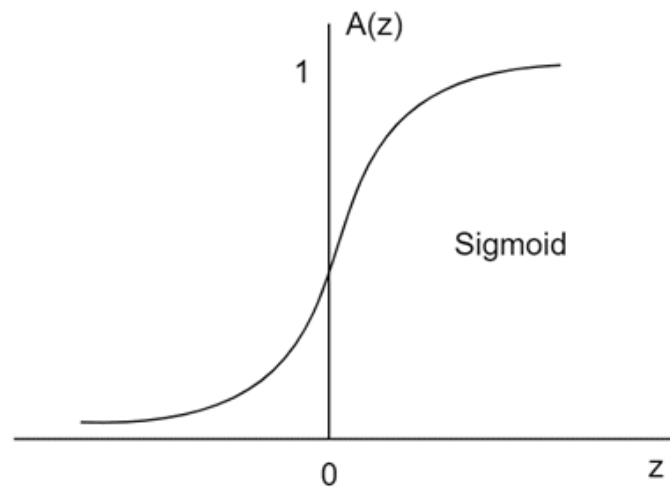


Figure 9. 3 Sigmoid activation function.

As shown in Figure 9.3, the sigmoid activation function is always positive and between 0 and 1. Let us say Z is a very small number, that is, Z moves to $-\infty$, e^{-Z} tends to infinity and $A(Z)$ will tend to 0. Similarly, when Z is very large, that is, Z moves to ∞ , e^{-Z} tends to 0 and $A(Z)$ will tend to 1. It is used for classification tasks - especially for binary classification final output with a cross-entropy loss.

Tanh

$$A(Z) = (e^Z - e^{-Z})/(e^Z + e^{-Z})$$

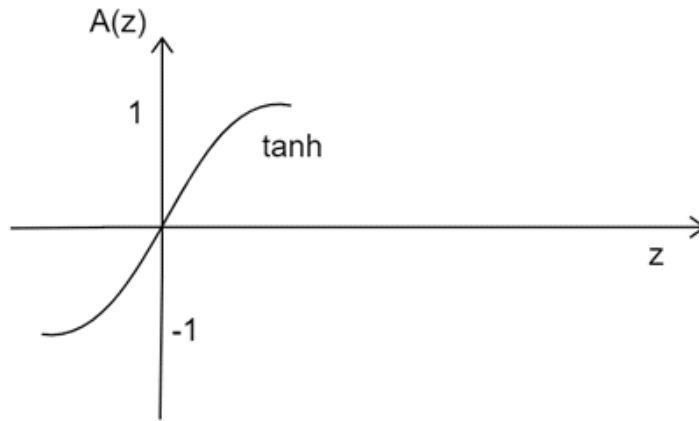


Figure 9. 4 Tanh activation function.

Characteristics

1. Lies between -1 and 1 (Figure 9.4)

If we are using cross entropy as a loss function, using tanh as an activation function is not advisable in the output layer.

Relu (Rectified Linear Unit)

$$A(Z) = \max(0, Z)$$

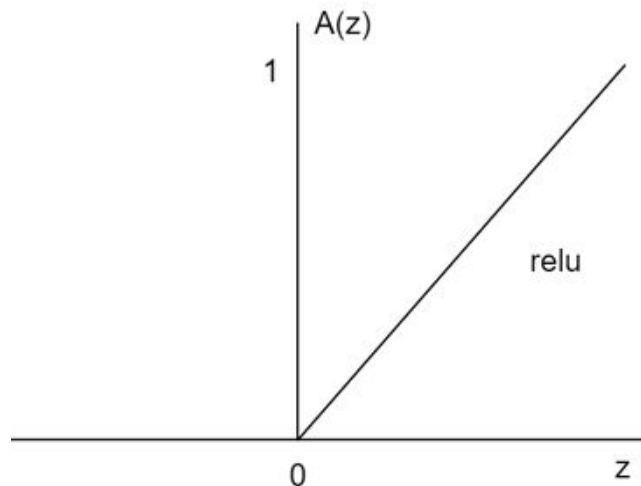


Figure 9. 5 Relu activation function.

Characteristics

1. Always positive (Figure 9.5)
2. One of the most popular activations used currently
3. It does not map negative values well as all are mapped to zero.

Leaky-Relu

$$A(Z) = \max(aZ, Z)$$

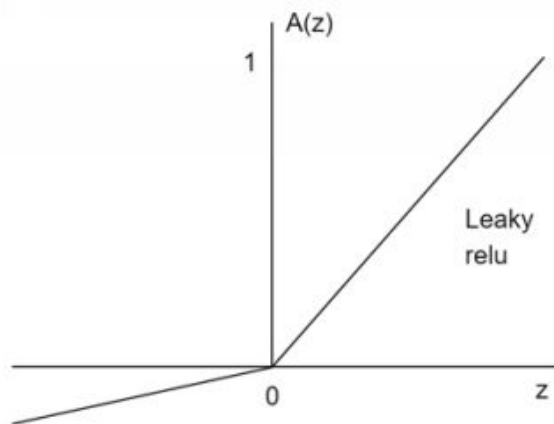


Figure 9. 6 Leaky Relu activation function.

Characteristics

1. Tries to reduce the issue with the negative values mapping in the Relu. (Figure 9.6)
2. Need to decide the value of hyper parameter a . It is usually a small value such as 0.01, 0.001, and so on.

Linear Activation

$$A(Z) = Z$$

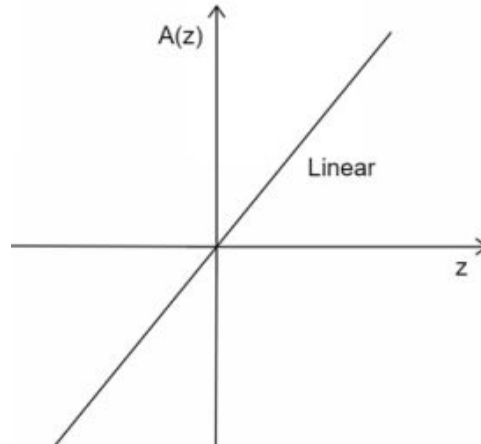


Figure 9. 7 Linear activation function.

Cannot be used exclusively as irrespective of how deep your network is, you will essentially be calculating the same function over and over again (Figure 9.7).

Softmax Activation

In case of a multiclass classification problem with C (≥ 2) possible class values, the softmax output is defined as

$$A(Z_i) = e^{Z_i} / \sum_{j=1}^C e^{Z_j}$$

This ensures that all the C predictions per sample sum to 1.

Layers

A group of neurons connected to another group of neurons, but not necessarily to each other. The types of layers that together form the framework of the neural network architecture are as follows:

Input layer

As the name suggests, this is the layer that contains the input features from the dataset. This is the 0th layer of the neural network. It connects to the hidden layer(s).

Hidden layer

These are the working horses of the neural network. They receive inputs from either the input layer or other hidden layers and provide its output to other hidden layers or output layer of the network.

Output layer

This is the final layer of the network and provides the output of the process. The output can be a probability, class, a real number, and so on based on whether we are dealing with a classification or regression problem.

A neural network is called an “L-layer network” if the total number of hidden layers and output layers is equal to “L”. Hence, an NN with an input layer, one output layer, and one hidden layer is called a two-layer network (input layer is the 0th layer). Example of such a network is illustrated in the [Figure 9.8](#). The term “deep learning” essentially implies a large number of hidden layers in the network. In Figure 9.8 we only show the direction of the forward pass. Backward propagation goes in the opposite direction (not shown here).

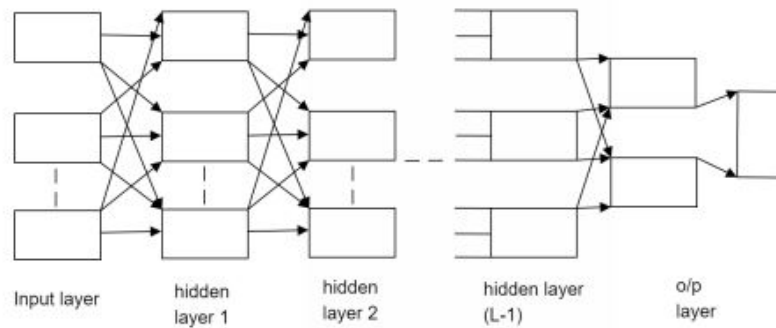


Figure 9. 8 Deep network.

A connection between two neurons in layers L_a and L_b simply implies that during the forward pass, the receiving neuron in layer L_b uses the output of the neuron in layer L_a as part of its input to calculate the activation function. During backward propagation, layer L_b 's gradient descent output goes as an input to layer L_a 's gradient.

Fully Connected Network

In a neural network, if every neuron of a layer is connected to every neuron of the preceding and succeeding layer, it is known as a fully connected network. Why do deep networks work well? The initial layers can look at detecting low-level features of the inputs exploring simpler relations between inputs and outputs and the latter layers can work on more “complex” relations by using the simpler features detected in the earlier layers, hence exploring the entire gamut. This has contributed to the popularity of deep networks across domains even though they are computationally much more expensive than shallow networks.

Epoch

Once all the layers and neurons have completed the two-step process, we come to the end of phase 1, known as forward pass. At this point, we have a “predicted”/calculated value of the output(s), known as Y_{pred} . This value may not be the same as the actual output Y_{act} . Depending on the kind of problem we are dealing with, we find an appropriate representation of this error. For example, in the case of a regression problem, we may represent the error as mean square error or mean absolute error. In the classification setup, we may choose to go with a metric such as log loss. The aim is to minimize this error to get a function that “best” approximates the relationship between the input and the output. Let us call this $L(Y_{\text{act}}, Y_{\text{pred}})$. To minimize this, we use partial derivatives of $L(Y_{\text{act}}, Y_{\text{pred}})$ with respect to the parameters, that is, weights and biases. Let us call these dW_l' and db_l' for the weights and biases related to layer “l.” The parameter values of the layer “l” are updated as:

$$W_l = W_l - \text{alpha} * dW_l'$$

$$b_l = b_l - \text{alpha} * db_l'$$

Similar steps are carried out starting from the final layer right up to the first layer. This is the backward propagation step. One such pass through the entire training set is known as an epoch.

Parameters and Hyper Parameters

As described in the chapter so far, several variables affect the outcome of the neural network in various ways. Some of these variables, called parameters, are the ones that are learned during the process of training via forward pass and backward propagation. Weights and biases of the various layers fall under this category. On the other hand, there are variables such as learning rate that are provided by the user. Such variables are known as hyper parameters. These are user-chosen but affect the outcome and performance of the network. Other hyper parameters that we have come across so far are the number of layers, the number of neurons in each layer, and so on. It is very important to tune these hyper parameters to get the best possible performance from the network. There are a few ways to select the best possible hyper parameters.

Grid search

Specify a range of values for all the hyper parameters.

1. Train the neural network for each of the possible hyper-parameter value combinations.
2. Select the values that construct the best performing neural network of all possible ones based on the hyper-parameter combinations.
3. The biggest issue with this is the exponential increase in the number of possible hyper-parameter combinations.

Random search

Specify a range of values for all the hyper parameters.

1. Train the neural network for a few randomly selected possible hyper-parameter value combinations.
2. Select the values that construct the *best performing* neural network out of all the possible ones from the randomly selected hyper-parameter combinations.

3. The biggest issue with this is that, while the number of models created is much lower than grid search but because of random selection, you may miss the “best” possible hyper-parameter combination.

Coarse to fine search

1. Specify a range of values for the hyper parameters.
2. Train the neural network on a comparatively widely spaced (or coarse) sample of the hyperparameter combinations.
3. Select the range of values that get the highest performing neural network.
4. Retrain neural networks for all possible values between the range of values selected earlier.
5. Select the hyper-parameter combination that gives the highest performing neural network.

While selecting values within a given range, it is not necessary to search by uniformly selecting values. Values should be selected based on the range of values. For example, if the range one is looking at for a hyper parameter is 2–8, one can select uniform values of 2, 3, 4, 5, 6, 7, and 8. If the range is 32–1024, one can select values separated by a power of 2 so it will be 32, 64, 128, 256, 512, and 1024. It is important to note that, if it is allowed to train long enough or with a complex-enough network, the neural network architecture is capable of learning the exact outputs for the inputs provided in the training set. However, it will not necessarily be able to predict the outputs on unseen data. This is known as overfitting.

Overfitting

Let us look at some of the important and common ways of reducing overfitting.

More data

If the neural network has overfitted on the existing training data, then the logical step to solve this issue is to add more training data. However, this is not always easy as this may be time consuming and/or expensive.

Regularization

Penalize the complexity of the network by adding a regularizing component to the backward propagation. This essentially helps to reduce the impact of lots of hidden units which, in effect, makes the network much simpler.

Dropout

For every example that the neural network is trained on, some neurons are dropped at random. The neurons chosen to be dropped, change with every example. This makes the network simpler and prevents any one neuron to be dominant in determining the output. Two things to remember while using dropout:

- Dropout is used only for training and not at the time of prediction.
- As the expected value of the activation is reduced by the proportion of the dropped neurons, the expected value of the activation is pumped up to compensate for this. This is known as an inverse dropout.

Both the aforementioned issues are handled by all standard implementations of neural networks, so you do not need to explicitly code for this.

Early stopping

One of the visual tools to determine if overfitting is happening, is to plot the cost function against the number of iterations. If both training and development/validation set costs are decreasing with an increase in the number of iterations, we can continue training.

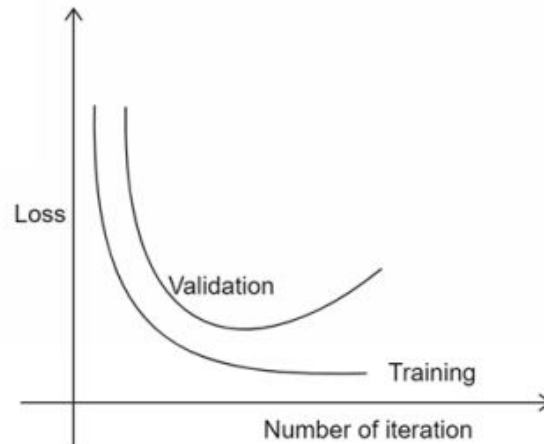


Figure 9. 9 Identifying when overfitting is happening

The problem with early stopping is that it mixes two orthogonal (or independent) tasks of preventing overfitting and optimizing the cost function.

Methods to improve optimization

This brings us to the next topic: How do we optimize the cost function, that is, how do we make the training faster and more efficient?

Scale/Normalize the inputs

The scaling parameters are based on the training set only. Both validation and test set inputs or features need to be scaled according to the learnt scaling parameters of the training set. How does scaling help? Scaling ensures that, even if the actual value of the underlying inputs change, the mean and variance of it are known beforehand. Using scaling, one can thus build more optimal models.

Batch normalization

On the same lines of reasoning as the preceding text, it is helpful to normalize the activations going from one layer to another. It is open to debate whether one should normalize the intermediate value of $Z = wX + b$ or the activation value $Act(Z)$.

Another point to note is that you may not want all your hidden units to have 0 mean and 1 variance. To avoid this, the normalized intermediate values are multiplied by a parameter “gamma” and the parameter “Beta” is added. These are both learnable parameters via gradient descent, or other methods such as RMSprop, Adam's optimizer as described in the succeeding text.

Parameter initialization

The weights and biases linked to various layers and neurons need to be initialized before the training begins. Based on practical experience, it is clear that the initialization method affects the learnability of the network by reducing the impact of vanishing or exploding gradients. Various initializations such as He initialization, Xavier initialization (details beyond the scope of this book), and so on are provided in standard implementations of neural networks or you can use a self-defined initializer. Note that the

weights matrix should not be initialized to 0 as that would negate the learning due to multiple layers.

Mini-batch gradient descent

Split the training set into smaller subsets and apply the gradient descent on one such smaller batch at a time. This is called mini-batch gradient descent. If the size of each subset = 1, it is known as stochastic gradient descent. Mini-batch size is also a hyper parameter.

Gradient descent with momentum

This uses an exponential average of gradients to update the learnable parameters such as weights and biases.

RMSprop(Root Mean Square Propagation)

At every iteration, the exponentially weighted average of the square of the gradient is calculated. The weights and biases are updated by a scaled value of the current gradient (scaled by the square root of the exponentially weighted average of the square of the gradient).

Adam's optimization

It uses a combination of both gradient descent with momentum as well as RMSprop. Most of the standard libraries come with built-in Adam optimization.

Loss Functions for classification and regression problems

Multiclass classification

If we have a classification problem with C possible class values, the cross-entropy loss is defined as

$$CE = - \sum_{i=1}^C Y_i * \log(P_i)$$

Where Y_i is the ground truth and P_i is the probability of the prediction belonging to class i . This is suitable for problems where the same sample could belong to multiple classes, thereby Y_i could be vector of dimension C and multiple values equal 1.

The categorical cross entropy is very similar, except for the fact Y is ONE HOT encoded representation of dimension C . For $C = 3$, Y would take the form as $[1 \ 0 \ 0]$, where $Y_1 = 1$ and $Y_2 = Y_3 = 0$.

Regression

The loss functions most commonly used in a regression setting are mean squared error, mean absolute error, and mean absolute percentage error.

Keras Implementation

The package of choice for implementing deep neural networks is Keras. Let us look at some sample code.

The target values as represented by `y_train`, `y_test`, and `y_valid` have values 1, -1, or 0 per sample and are converted to ONE HOT representation below.

```
scaler=StandardScaler().fit(x_train)
x_train=scaler.transform(x_train)
x_test=scaler.transform(x_test)
x_valid=scaler.transform(x_valid)
y_train=keras.utils.to_categorical(y_train,3)
y_test=keras.utils.to_categorical(y_test,3)
y_valid=keras.utils.to_categorical(y_valid,3)
```

Add the first hidden layer with `dl1= 64` neurons, `af1= "relu"` activation and weights `l2` regularized with a `parameter=0.08`

```
model.add(Dense(dl1,activation=af1,input_shape=(x_train.shape[1],),W_regularizer = l2(.08)))
```

Add dropout as next layer with drop out probability set as `.25`

```
model.add(Dropout(.25))
```

Add output layer with three neurons and activation as softmax

```
model.add(Dense(3,activation='softmax'))
```

Compile and fit, loss function set as categorical cross entropy, Adam optimizer and learning rate = `lr1`. Fit on training data, validate on validation data, and have early stopping on validation accuracy

```
es = keras.callbacks.EarlyStopping(monitor='val_acc', min_delta=0.0005, patience=2)
model.compile(loss='categorical_crossentropy', optimizer = Adam(lr=lr1), metrics=['acc'])
model.fit(x_train, y_train, batch_size=32, epochs=20, validation_data=(x_valid, y_valid), callbacks=[es])
```

Hyper-parameter optimization for lr1(learning rate), dfl1(number of neurons), and af1(activation function)

```
af=['softplus','relu','sigmoid']
dense_layer1= [8,32,64,96,256]
lr_list=[0.0001,.00005,.0005,.001,.005]

for i in range(number_of_architecture_searches):
    dl1=dense_layer1[np.random.randint(0,len(dense_layer1))]
    lr1=lr_list[np.random.randint(0,len(lr_list))]
    af1=random.choice(af)
```

Financial Application: dealing with unbalanced datasets

As have mentioned before, neural networks are universal function approximators that can be used to solve various problems in finance. Let us discuss one such setting.

We often encounter unbalanced classes in classification problems. Let us say we are interested in predicting the next tick of CME gold futures. We set it up as a 3-class problem, wherein we assign

Label 1: to samples where the next tick is higher than the current tick. We call it class 1.

Label 0: to samples where the next tick is the same as the current tick. We call it class 0.

Label -1: to samples where the next tick is lower than the current tick. We call it class 2.

Figure 9.10 is a snapshot of what our dataset would look like.

Time	Asset	Bid Quantity	Bid Price	Ask Quantity	Ask Price	Traded Price	Traded Quantity
00:00.0	GC1	1	1505.5	2	1505.6	1505.6	16
00:00.0	GC1	1	1505.4	2	1505.6	1505.6	16
00:00.0	GC1	1	1505.5	2	1505.6	1505.6	16
00:00.1	GC1	3	1505.5	3	1505.6	1505.6	17
00:00.1	GC1	5	1505.5	2	1505.6	1505.6	19
00:00.1	GC1	5	1505.5	7	1505.8	1505.6	19
00:00.1	GC1	4	1505.6	8	1505.8	1505.6	19
00:00.1	GC1	1	1505.7	8	1505.8	1505.6	19
00:00.2	GC1	6	1505.7	7	1505.8	1505.7	20
00:00.2	GC1	5	1505.7	7	1505.8	1505.7	21
00:00.2	GC1	2	1505.7	7	1505.8	1505.7	22
00:00.2	GC1	2	1505.7	7	1505.8	1505.7	23
00:00.4	GC1	1	1505.7	8	1505.8	1505.7	24
00:00.4	GC1	6	1505.6	8	1505.8	1505.7	24
00:00.4	GC1	1	1505.7	8	1505.8	1505.7	24
00:00.9	GC1	5	1505.7	8	1505.8	1505.8	25
00:00.9	GC1	5	1505.7	8	1505.8	1505.7	26
00:00.9	GC1	4	1505.7	8	1505.8	1505.7	27
00:00.9	GC1	3	1505.7	8	1505.8	1505.7	30
00:00.9	GC1	7	1505.6	8	1505.8	1505.7	30
00:01.1	GC1	7	1505.6	8	1505.8	1505.6	37
00:01.1	GC1	7	1505.5	8	1505.8	1505.6	37
00:01.1	GC1	2	1505.7	9	1505.8	1505.6	37
00:01.1	GC1	2	1505.7	9	1505.8	1505.7	39
00:01.1	GC1	6	1505.5	9	1505.8	1505.7	39
00:01.1	GC1	4	1505.5	2	1505.7	1505.7	39
00:01.1	GC1	4	1505.6	3	1505.7	1505.7	39
00:01.1	GC1	5	1505.5	5	1505.7	1505.7	39

Figure 9. 10 Snapshot of high frequency data for CME gold futures.

For all of 2018 we had about 20 million samples, of which 15% belonged to class 1, 15 % belonged to class 2 and 70% belonged to class 0. So, if our machine was to predict that all samples belong to class 0, we would register an accuracy of 70%, which may be quite reasonable. Now during training, our machine will be seeing samples that belong to class 0 70% of the time. If the choice of hyperparameters is such that the machine is a “biased” one, then it may well happen that all samples are predicted to belong to class 0, which essentially means we won’t be trading and hence we won’t be making money. This is a classic problem when we deal with unbalanced data sets. The solution we propose involves modification of the cost function. Let us look at the details:

The categorical cross entropy for a 3-class problem for one particular sample would be

$$CE = -y_0 \log(p_0) - y_1 \log(p_1) - y_2 \log(p_2)$$

Where $Y_i = 1, \forall i = 0, 1, 2$

And p_i : the probability that the sample belongs to class i

The modified cost function for unbalanced classes would be

$$CE_{\text{modified}} = -(w_0 y_0 \log(p_0)) - (w_1 y_1 \log(p_1)) - (w_2 y_2 \log(p_2))$$

Where w_i is the weight for class i calculated as the inverse of the frequency of occurrence of that class in our training data set. Since class 1 occurs in 15% of the samples $w_1 = 1/(15\%) = 6.66$, $w_0 = 1/(70\%) = 1.42$ and $w_2 = 6.66$

Finally, this is how we implement it in Keras

```
def weighted_loss(y_true, y_pred):  
    return K.mean([weights[0]**(y_true[:,0])*(weights[1]**(y_true[:,1]))*(weights[2]**(y_true[:,2]))*K.categorical_crossentropy(y_true, y_pred)], axis=-1)
```

This function is called at the time of compiling the model

```
model.compile(loss=weighted_loss,metrics=['categorical_accuracy'])
```

Summary

- We introduced all the building blocks necessary to build a deep neural net architecture.
- We also introduced you to the Keras API for implementing neural networks.
- Finally, we discussed how the cost function can be modified to deal with unbalanced data sets.

References

1. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting" by Srivastava, Hinton et al.
2. <https://www.deeplearningbook.org/>
3. <https://keras.io/>
4. Gu, Shihao & Kelly, Bryan & Xiu, Dacheng (2018). Empirical Asset Pricing Via Machine Learning. SSRN Electronic Journal. 10.2139/ssrn.3159577.
5. Lim, Bryan & Zohren, Stefan & Roberts, Stephen (2019). Enhancing Time Series Momentum Strategies Using Deep Neural Networks.

CHAPTER TEN - CONVOLUTION NEURAL NETWORKS

This chapter introduces convolutional neural networks. We then try and uncover how this network can be used to discover chart patterns to trade markets, an approach that some traders have been able to successfully deploy. It starts by discussing the dot product operation and how it relates to the convolution operation as defined in the context of neural networks.

Convolution Operation

Let us start by looking at two simple vectors U and V .

$$U = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$$

The dot product operation between the two vectors yields

$$U.V = V.V = 91$$

Let us take another vector W defined as

$$W = \begin{bmatrix} -1 & 2 & -3 & 4 & -5 & 6 \end{bmatrix}$$

Do note $\|W\| = \|V\|$. The dot product operation between vectors U and W yields

$$U.W = 21$$

The more similar the two vectors, the higher the value of their dot product, after adjusting for their modulus $\| \cdot \|$. Since vectors U and V have a higher dot product value than vectors U and W , i.e. $U.V > U.W$, we infer vectors U and V are more similar to each other than vectors U and W .

We will now discuss the convolution operation between two vectors, the way it is applied to convolutional neural networks. Let us define a vector U of length 6 as

$$U = \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix}$$

A vector f that we call a filter of length 2 is defined as

$f=$	-1	1
------	------	-----

We take the dot product of U and f , two elements at a time (two because that is the length of our filter).

2	4	⊗	-1	1	=	$2 \cdot -1 + 4 \cdot 1 = 2$
---	---	---	----	---	---	------------------------------

4	6	⊗	-1	1	=	$4 \cdot -1 + 6 \cdot 1 = 2$
---	---	---	----	---	---	------------------------------

6	8	⊗	-1	1	=	$6 \cdot -1 + 8 \cdot 1 = 2$
---	---	---	----	---	---	------------------------------

8	10	⊗	-1	1	=	$8 \cdot -1 + 10 \cdot 1 = 2$
---	----	---	----	---	---	-------------------------------

10	12	⊗	-1	1	=	$10 \cdot -1 + 12 \cdot 1 = 2$
----	----	---	----	---	---	--------------------------------

Figure 10. 1 Convolution operation between U and f

Therefore, the convolution ⊗ between U and f is a vector of length 5.

2	4	6	8	10	12	⊗	-1	1	=	2	2	2	2	2
---	---	---	---	----	----	---	----	---	---	---	---	---	---	---

The final step is to add a bias to each element of the output vector and pass that through some activation function (Relu for instance). The final output on convolving U with filter f would be something like Figure 10.2:

Relu(2+b)	Relu(2+b)	Relu(2+b)	Relu(2+b)	Relu(2+b)
-----------	-----------	-----------	-----------	-----------

Figure 10. 2 Activation and bias added to get the final output of convolution

The same operation can be generalised to a higher dimension. $An_1 \times n_2$ matrix when convolved with a filter of dimension $f_1 \times f_2$ will give a matrix of dimension

$(n_1 - f_1 + 1) \times (n_2 - f_2 + 1)$. As an example (Figure 10.3) if we have a two-dimensional input represented as a 4×3 matrix, which is convolved with a 3×3 filter, we get a $(4 - 3 + 1) \times (3 - 3 + 1) = 2 \times 1$ -dimensional output as shown:

1	2	3
4	5	6
7	8	9
10	11	12

⊗

1	0	1
-1	0	1
1	1	1

=

30
45

Figure 10. 3 1:2-dimensional convolution example

When we add the bias and pass through an activation function such as Relu it would be as shown Figure 10.3.2:

Relv (30+b)
Relv (45+b)

Figure 10. 4 Bias and activation added

The convolution operation for the above example would work as shown in figure 10.5 and 10.6:

1	2	3
4	5	6
7	8	9

⊗

1	0	1
-1	0	1
1	1	1

Figure 10. 5 Convolution operation for 2-dimensional matrix

Here (in figure 10.5) the convolution works out to be:

$$1 * 1 + 2 * 0 + 3 * 1 + 4 * -1 + 5 * 0 + 6 * 1 + 7 * 1 + 8 * 1 + 9 * 1 = 30$$

And:

4	5	6
7	8	9
10	11	12

⊗

1	0	1
-1	0	1
1	1	1

Figure 10. 6 Convolution operation for 2-dimensional matrix

Here (in figure 10.6) the convolution works out to be:

$$4*1 + 5*0 + 6*1 + 7*-1 + 8*0 + 9*1 + 10*1 + 11*1 + 12*1 = 45$$

To summarize:

- We have learnt the working of the convolution operation and that it is closely tied to the dot product operator.
- The higher the value of a dot product operation, the more similar the two operands, for a given value of the modulus of the operands.

Let us try and see in the next section where this may be useful and how.

Financial Application: trading chart patterns

Most traders look at charts when trading and many books have been written on various chart patterns that traders have been observing for a fairly long period of time. We do not have a view on the efficacy of these methods. Some traders such as Peter L. Brandt, author of “*Diary of a Professional Commodity Trader*”, have been able to use them very effectively with utmost discipline; however, there would be many more who have used them without much success. It clearly seems to be more of an art than anything else.

Let us try and see how we can understand CONVOLUTIONAL NEURAL NETWORKS (CNN) better using some of these charts. One of the many patterns that traders like to follow are the heuristics of Elliot wave (Figure 10.7). Among other things, it says that when five waves move in the direction of the main trend, we expect a consolidation or a reversal.

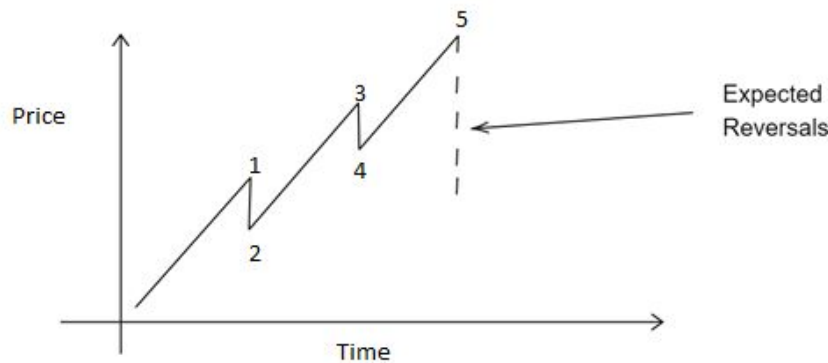
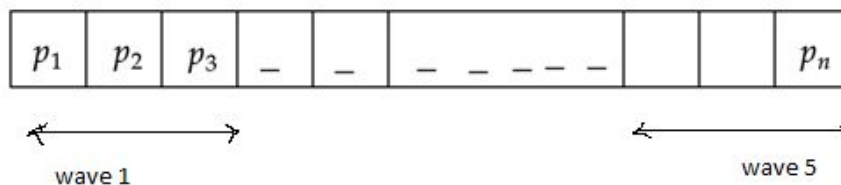


Figure 10. 7 A hypothetical 5 wave rally.

Just to keep the analogy clear, the way we would represent this price history in a vector P is like this:



Like in the previous section, we also have a filter f , which is represented like this:

f_1	f_2	f_3	— — — —	f_n
-------	-------	-------	---------	-------

We also understand what the convolution between $P \odot f$ does. It essentially finds a rolling dot product that is maximized at those locations where P and f have a similar shape.

Therefore, if f assumed a shape like the one in Figure 10.8 (the dotted lines represents the discrete nature of the filter), and we convolve it with the time series P of Figure 1, then the convolution operation will assume high values somewhere around the end of wave 1 and wave 3, which is where the similarity between the time series P and the filter f is the highest. The convolution operation therefore helps us find where in the time series P does the shape of filter f occur.

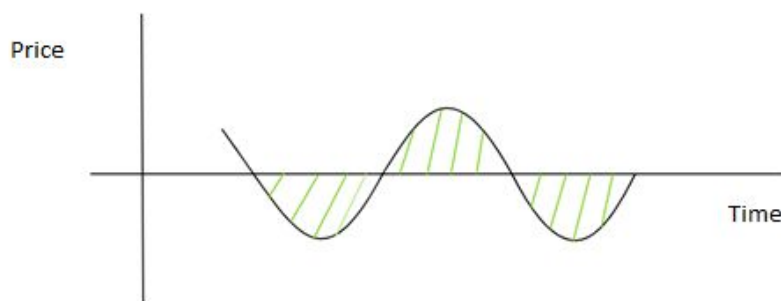


Figure 10. 8 A hypothetical discrete filter.

So far, we have understood what convolution means and what it does. Where do neural networks fit into all this?

The neural network helps us find this filter f . In other words, which shapes in this time series are necessary to occur to meet the objective, that is, which cause the selloff.

The neural network will essentially need to find the shape/values and length of this filter such that the market sells off. The back-propagation algorithm will be used to train and adjust the values of the filter such that the market sells off. In that sense, the convolutional neural

network helps us find patterns that occur in the time series that meet our objective, which as defined by the target is for the market to sell off. This helps in feature abstraction.

Whatever the filter values are, it is the same set of values that are applied across the entire time series, to find a pattern. Wherever the same pattern occurs in the timeseries, the filter can identify it. The value of the filter doesn't need to change to achieve the objective of finding the pattern. This property is known as translation invariance. Hopefully, we have been able to explain how CNN can be used to find features that meet the target function. In the next few sections we will discuss a few more important elements of the CNN architecture.

Padding

In its current form, the price points at the edges of the vector seem to be given less importance than the price points in the middle.

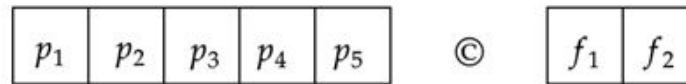


Figure 10. 9 Time series convolved with a filter

In Figure 10.9 during convolution, P_1 and P_5 are just used once; each with the filter values f_1 and f_2 respectively. P_2 , P_3 , and P_4 , on the other hand, are used twice with the filters. Besides, each time we convolve a time series of length n with a filter of length f , the resulting time series is of length $n - f + 1 < n$. This may negatively affect learning capability as we get deeper with the architecture. We may choose to pad the time series with $2p$ zeros, p on either side Figure 10.10. The time series is now of length $n + 2p$, which, when convolved with a filter f , produces a time series of $n + 2p - f + 1$.

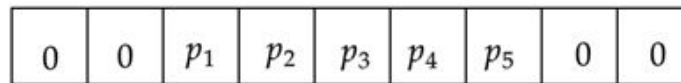


Figure 10. 10 Padding time series

If we desire to keep the length of the time series the same, $n + 2p - f + 1 = n$

$$P = (f - 1)/2$$

This is called “same convolution”.

If we have $P=0$, it is called “valid convolution”.

Strides

When we were moving our filter along the vector after performing each dot product operation, we were doing it in steps of one. The stride was effectively assumed to be one. That need not be the case always. Let us say we have a vector u given by

$$u = \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix}$$

And the filter f given by

$$f = \begin{bmatrix} -1 & 1 \end{bmatrix}$$

The convolution operation **with a stride of 2** is given by the following steps in Figure 10.11:

$$\begin{array}{l} \begin{bmatrix} 2 & 4 \end{bmatrix} \odot \begin{bmatrix} -1 & 1 \end{bmatrix} = 2 \cdot (-1) + 4 \cdot 1 = 2 \\ \begin{bmatrix} 6 & 8 \end{bmatrix} \odot \begin{bmatrix} -1 & 1 \end{bmatrix} = 6 \cdot (-1) + 8 \cdot 1 = 2 \\ \begin{bmatrix} 10 & 12 \end{bmatrix} \odot \begin{bmatrix} -1 & 1 \end{bmatrix} = 10 \cdot (-1) + 12 \cdot 1 = 2 \end{array}$$

Figure 10. 11 stride 2 convolution operation

Effectively,

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix} \odot \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix}$$

To generalize, when we convolve an n dimensional vector with an f dimensional filter and use a stride of s , we get an output of size $[(n+2p - f)/s + 1]$

$$1 \times n \odot 1 \times f = 1 \times [(n + 2p - f)/s + 1]$$

Why would we want to add strides to a time series for trading? One way to think about it is that we are looking for opportunities across time frames. The pattern of interest does not necessarily need to occur at the most granular time step - it may be more visible at a higher time step. The same way chartists tend to look at monthly, weekly, daily, and intraday time frames to firm up on a view, using various strides allows the neural network to find shapes at multiple time frames that may impact the target.

Multiple Filters

So far, we have been working with one filter. As discussed earlier, the values of the filter are learnt via back propagation and it represents a certain pattern that exists in the time series that helps us predict the target/outcome.

However, there could be several meaningful patterns that could give us meaningful information to help us predict the outcome. We are free to add as many filters as deemed necessary. Let us see how the CNN adapts to it. We take our vector U to be

$$U = \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix}$$

And convolve U with three different filters as shown in Figure 10.12

$$\begin{array}{l} \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix} \otimes \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 & 2 & 2 \end{bmatrix} \\ \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix} \otimes \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 6 & 8 & 10 & 12 \end{bmatrix} \\ \begin{bmatrix} 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 & 8 & 10 \end{bmatrix} \end{array}$$

Figure 10. 12 Multiple filter convolution

We can stack these three vectors as a $1 \times 5 \times 3$ matrix Figure 10.13.

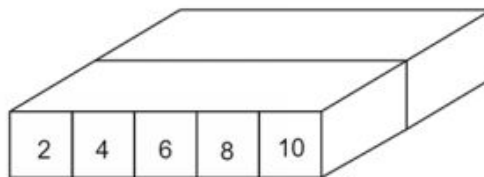


Figure 10. 13 Stacking the output of Multiple filter convolution

So, whenever we convolve $1 \times n$ with $1 \times f$ with m filters, we get $1 \times (n - f + 1) \times m$

Again, the analogy to a chartist would be that the chartist for instance may have the 20, 50, and 100 day moving averages always loading up on her charts. In a CNN, one filter really captures “one shape”. Using multiple filters allows us to have the capability to learn multiple “shapes” that may impact the target.

Multiple Channels as the Input

Our input could be multi-dimensional, of the form $1 \times n \times m$. Say we have a vector U as $1 \times 6 \times 3$. Where,

vector $1 \times 6 \times 1$ is

1	2	3	5	5	6
---	---	---	---	---	---

vector $1 \times 6 \times 2$ is

1	-2	3	-5	5	-6
---	----	---	----	---	----

vector $1 \times 6 \times 3$ is

-1	2	-3	4	-5	6
----	---	----	---	----	---

This U is a three-dimensional vector; the last dimension is the depth or the number of channels, which in this case is 3.

The filter f we design needs to have the same depth or channels as our input vector U , so that each filter can be applied to the whole image. Let us define a filter $1 \times 2 \times 3$, where again we define the filter values at each depth separately.

filter $1 \times 2 \times 1$ is

1	-1
---	----

filter $1 \times 2 \times 2$ is

1	0
---	---

And finally filter $1 \times 2 \times 3$ is

0	1
---	---

The first convolution operation between 1,2 and 1,-1 is $1*1 - 2*1 = -1$

The second convolution between 1,-2 and 1,0 which is $=1$.

The third convolution $-1,2$ and $0,1$ is $=2$.

We sum it up to $-1+1+2 = 2$.

The full operation is as follows in figure 10.14:

$$\begin{array}{rcl}
 \begin{array}{|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 5 & 5 & 6 \\ \hline \end{array} \otimes \begin{array}{|c|c|} \hline 1 & -1 \\ \hline \end{array} & = & \begin{array}{|c|c|c|c|c|} \hline -1 & -1 & -1 & -1 & -1 \\ \hline \end{array} \\
 \begin{array}{|c|c|c|c|c|c|} \hline 1 & -2 & 3 & -5 & 5 & -6 \\ \hline \end{array} \otimes \begin{array}{|c|c|} \hline 1 & 0 \\ \hline \end{array} & = & \begin{array}{|c|c|c|c|c|} \hline 1 & -2 & 3 & -5 & 5 \\ \hline \end{array} \\
 \begin{array}{|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 \\ \hline \end{array} \otimes \begin{array}{|c|c|} \hline 0 & 1 \\ \hline \end{array} & = & \begin{array}{|c|c|c|c|c|} \hline 2 & 3 & 4 & 5 & 6 \\ \hline \end{array} \\
 & & & & & \\
 & = & \begin{array}{|c|c|c|c|c|} \hline 2 & 0 & 6 & -1 & 10 \\ \hline \end{array}
 \end{array}$$

Figure 10. 14 Multiple channels convolution

In effect, a $1 \times n \times m$ time series is convolved with a $1 \times f \times m$ filter to produce a $1 \times [(n+2p-f)/s + 1] \times 1$ output.

The multi-dimensional inputs are also known as multiple channels. Different representations of the same time series could be put into these different channels. For instance, one channel could contain the closing prices of the time series, the other channels could contain the open, high, and low respectively. We may sample our time series differently; for instance, once every week or each time a certain number of lots has traded (volume-based sampling). These different representations of the same time series could go into the various channels of our input.

Pooling

So far, we have seen the operation of a filter of dimension $1 \times f \times m$. This has $f \times m$ learnable parameters that are learnt via back propagation. A pooling filter or a pooling layer is different in the sense that it does not have any learnable parameters. It does have two hyper parameters f (filter size) and s (stride). In addition, all it does, is to calculate the maximum (or average value) over the time series. Let us take U to be a vector equal to

$U =$

2	4	6	8	10	12
---	---	---	---	----	----

Let us say the filter has length $f = 2$ and stride $s = 1$. As we slide f across U , we keep calculating the maximum value as in Figure 10.15

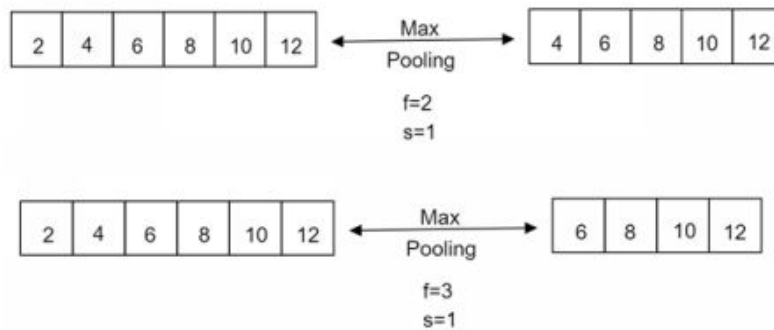


Figure 10. 15 Max Pooling operation

The pooling operation when applied to inputs with multiple channels is applied independently to each channel. The output therefore has the same number of channels as the input.

The typical setup for convolutional neural networks could look something like Figure 10.16:

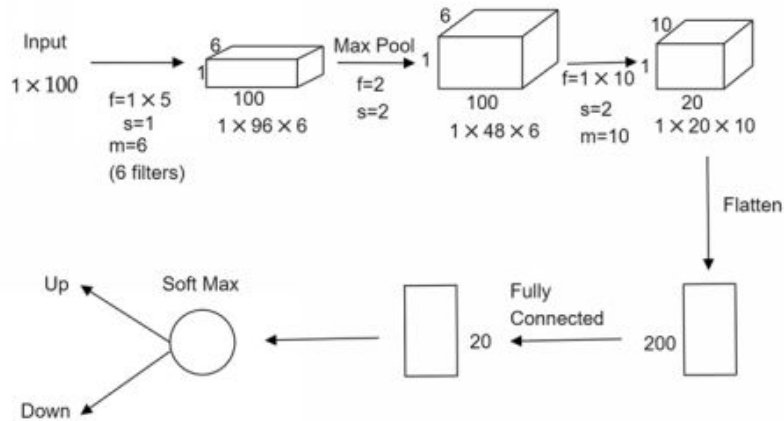


Figure 10. 16 Full convolution setup

In figure 10.16 We start off with an input of dimension 1×100 , say the returns of the previous 100 days. We pass this through six filters; each filter has a dimension of 1×5 . The dimension of the output is $1 \times (100 - 5 + 1) = 1 \times 96$ and since there are six filters, we stack each of their respective outputs, resulting in an output of dimension $1 \times 96 \times 6$.

We then pass this output through a max pooling layer with $f=2$ and stride $s=2$. The pooling operator does not change the number of channels, which have been set to 6. The max pooling reduces the dimension from $1 \times 96 \times 6$ to $1 \times [(96 - 2) / 2 + 1] \times 6 = 1 \times 48 \times 6$.

We then pass this $1 \times 48 \times 6$ output through 10 filters - each of size 1×10 - but this time, we use a stride of 2, which results in an output of dimension $(1 \times (48 - 10) / 2 + 1 \times 10) = 1 \times 20 \times 10$.

We then flatten this two-dimensional representation into a one-dimensional vector, which simply has a length of $1 \times 20 \times 10 = 200$. This is then passed through a regular fully connected multi-layered perceptron with 20 neurons. The weight matrix connecting the flattened vector to this fully connected dense layer will be of dimension $20 \times 200 = 4000$ parameters.

Finally, the output of the 20 neurons is passed to the softmax function, which helps us predict whether the market goes up or down.

The aforementioned is a general setup with numerous hyper parameters, requiring a thorough random search.

Let us take a concrete example.

Financial Application: extracting features for multi asset trading

Our sample data set looks something like Figure 10.17:

S. NO	Date	Gold Return	Dollar Return	Gold Target	Dollar Target
1	20180101	...	..	..	..
2	20180201	..	..	..	..
3	20180301	..	..	..	..
n	20180401	..	..	..	..

Figure 10. 17 Data Template

Where the targets assume two values: 0 (market down) and 1 (market up). We modify it to look like Figure 10.18:



Figure 10. 18 reorienting the raw data

Where `n_col` is a certain look back window, say 100. The function that does this transformation is Figure 10.19

```
def __data_generation(  
    batch_size=32,  
    n_channels=1,  
    n_rows=2,  
    n_col=100, data=df[["Gold Return", "Dollar Return"]],  
    datay=df[["Gold Target", "Dollar Target"]]  
):  
  
    x = np.empty((batch_size, n_channels, n_rows, n_col))  
    y = np.empty((batch_size, n_rows))  
    for i in range(n_col, dataX.shape[0], 1):  
        rows = np.arange(i-n_col, i)  
        x[i-n_col, 0] = data[rows, :].T  
        y[i-n_col] = datay[rows[-1]]  
    return x, [keras.utils.to_categorical(y[:, 0], num_classes=2), keras.utils.to_categorical(y[:, 1], num_classes=2)]
```

Figure 10. 19 Code to re orient raw data to get shape as shown in figure 10.16

We now define our architecture

in figure 10.20

```
n_assets =2
n_columns =100
input_layer = Input(shape=(1, n_assets, n_columns))
#50 filters of dimension 2x 3
x = Conv2D(50, (n_assets, 3), padding='valid', strides=(1, 1), activation='relu',
kernel_initializer=kernel_init, bias_initializer=bias_init,data_format="channels_first")(input_layer)
#50 filters of dimension 2x 10
x1 = Conv2D(50, (n_assets, 10), padding='valid', strides=(1, 1), activation='relu',
kernel_initializer=kernel_init, bias_initializer=bias_init,data_format="channels_first")(input_layer)

#50 filters of dimension 2x 20
x2 = Conv2D(50, (n_assets, 20), padding='valid', strides=(1, 1), activation='relu',
kernel_initializer=kernel_init, bias_initializer=bias_init,data_format="channels_first")(input_layer)
#Max pooling for each of the filter units
x_mp=MaxPool2D((1,(n_columns-3+1)), strides=(1, 1),data_format="channels_first")(x)
x1_mp=MaxPool2D((1,(n_columns-10+1)), strides=(1, 1),data_format="channels_first")(x1)
x2_mp=MaxPool2D((1,(n_columns-20+1)), strides=(1, 1),data_format="channels_first")(x2)

#Flatten and concatenate max pooled layers
x_mp=Flatten()(x_mp)
x1_mp=Flatten()(x1_mp)
x2_mp=Flatten()(x2_mp)
x3 = concatenate([x_mp,x1_mp,x2_mp])
#Fully connected layer
x3 = Dense(16, activation='relu')(x3)
o1 = Dense(2, activation='softmax', name='auxilliary_output_1')(x3)
o2 = Dense(2, activation='softmax', name='auxilliary_output_2')(x3)
model = Model([input_layer], [o1,o2])
```

Figure 10. 20 Sample code for convolution operation

CNN should be the choice when the input is images like satellite images. For other types of data, experimentation is required.

Summary

- We have shared insights on how convolution operation can be used for feature generations.
- Convolution neural networks have attributes of parameter sharing and sparsity of connection.
- Same weights and biases are used across the entire time series. This is called parameter sharing.
- The sparsity of connections exists because “local” parts of the time series are connected to the network.
- Sparsity of connections and parameter sharing reduce potential overfit when compared to a fully connected multilayer perceptron.

CHAPTER ELEVEN - RNN- RECURRENT NEURAL NETWORKS

In this chapter we will understand recurrence and discuss architectures that capture recurrent relationships. Let us start by explaining what a recurrent relationship is and why it is required. We will then modify our understanding of the regular neural network and see what changes need to be made so that we can capture recurrent relationships. We will discuss several architectures, namely RNN (recurrent neural networks), LSTMs and GRU. Finally, we will discuss with an example how we can present data to this architecture and which parameters need to be trained.

Implementing Recurrent Function in Neural Networks

Neural networks are known as universal function approximators. That is given a set of inputs X we can model a function F to capture the relationship from X to Y . Let us say we are interested in modelling the price of a financial instrument at time t as X_t . We hypothesize that the instrument X is dependent on a set of variables at time t denoted by V_t . Needless to say, these set of variables are also observed at various point in time. We want to create a function F , such that

$$X_t = F(V_{t-1}, V_{t-2}, \dots V_{t-n}) \quad \text{Equation 11.1}$$

Where n is the look back window. V_t is an array of dimension m . If $m = 1$, and $V_i = X_i$, then

$$X_t = F(X_{t-1}, X_{t-2}, X_{t-3}, \dots X_{t-n}) \quad \text{Equation 11.2}$$

Essentially saying that the price of the instrument X at time t is dependent on the price of the instrument up to a lag to n . This simplification of $m = 1$ and $V_i = X_i$ was only used to give an understanding of the notation. We revert to the more general setting of

$$X_t = F(V_{t-1}, V_{t-2}, \dots V_{t-n})$$

Based on what we have discussed in Chapter 9, the neural network representation would look like Figure 11.1.

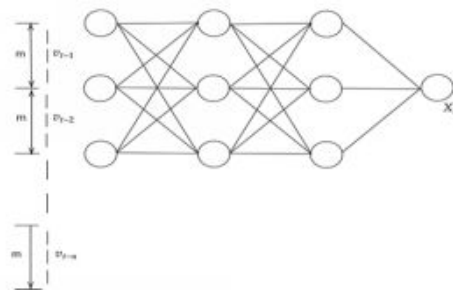


Figure 11. 1 Neural network representation of Equation 11.1.

The input layer is of dimension $n*m$. As the look back increases, the size of the input increases linearly and the neural network gets bulkier. There is a more efficient way of representing this. Let us try to understand how. From Equation 11.1 we have

$$X_t = F(V_{t-1}, V_{t-2}, \dots V_{t-n})$$

Moving it one time step forward, we get

$$X_{t+1} = F(V_t, V_{t-1}, \dots V_{t-n+1})$$

There are a lot of common terms in the two equations that are used to predict X_t and X_{t+1} . We can represent all historical information as H_t . This H_t is something we will learn by training with data. Effectively,

$$X_t = F(V_{t-1}, H_t) \text{ and } X_{t+1} = F(V_t, H_{t+1})$$

H_t represents the relevant historic information. Therefore, our network looks like Figure 11.2 for time step t and like Figure 11.3 for time step $t+1$.

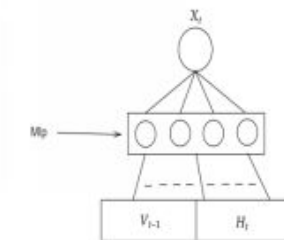


Figure 11. 2 $V_{t-2}, V_{t-3}, \dots V_{t-n}$ replaced by H_t .

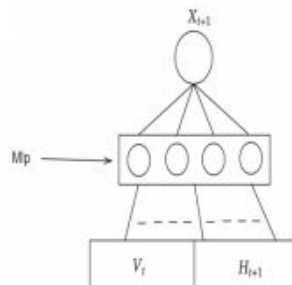


Figure 11. 3 $V_{t-1}, V_{t-2}, \dots V_{t-n+1}$ replaced by H_{t+1} .

Focusing on time step t (Figure 11.2), the output of the hidden layer can be represented as in Equation 11.3.

$$A_t = g(W[V_{t-1}, H_t] + b) \text{ Equation 11.3}$$

In addition, the output X_t as

$$X_t = g'(W' A_t + b') \text{ Equation 11.4}$$

Now, in a RNN (Recurrent Neural Network), we set $A_t = H_{t+1}$. This way the history is learnable and now the graphical representation looks like Figure 11.4. Here “MLP” is the regular neural network with variable number of neurons and hidden layers, which is learning the relationship between H_t and V_{t-1} to predict X_t .

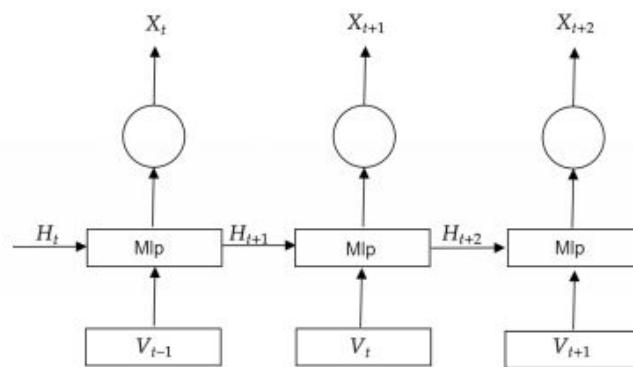


Figure 11. 4 RNN for various time steps

Dimensions of Various Vectors

In our notation so far, we have introduced several matrices and vectors, and it is important to get the dimensions of these right to improve our understanding of what is going on.

$$\mathbf{V}_t = 1 \times m$$

$\mathbf{H}_t = 1 \times h$, h is a hyper parameter, how many neurons needed to effectively learn the information in the entire look back period.

W is the weight matrix in Equation 11.3 and W' is the weight matrix in Equation 11.4.

$$\mathbf{W} = h \times (m + h)$$

$\mathbf{W}' = 1 \times h$, assuming X_t is price at time t so of dimension 1×1 .

$\mathbf{b} = h \times 1$, where b is the bias term of Equation 11.3.

$\mathbf{b}' = 1 \times 1$, where b' is the bias term of Equation 11.4.

The important thing to note is that the parameters W , b , W' , and b' do not have any subscript t . Which means it is the same set of parameters and hyper parameters that are used across time. In other words, only ONE MLP is used to predict the outcome at various time steps. The parameters are shared across time. This setup is the RNN.

Financial Application : trading CME futures

An example here will help us concretize the idea. We will try and predict the return next day, for CME gold futures.

Notations

$X_t = (CP_{t+1} - CP_t)/CP_t$, Where CP_t represents the closing price on day t .

$X_t^O = (OP_{t+1} - OP_t)/OP_t$, Where OP_t represents the opening price on day t .

$X_t^H = (HP_{t+1} - HP_t)/HP_t$, Where HP_t represents the high price on day t .

$X_t^L = (LP_{t+1} - LP_t)/LP_t$, Where LP_t represents the low price on day t .

$X_t^R = (CP_{t+1} - CP_t)/CP_t$, Where CP_t represents the closing price on day t for a related instrument say the dollar index.

$X_t^{OR} = (OP_{t+1} - OP_t)/OP_t$, Where OP_t represents the opening price on day t for a related instrument (dollar index).

$X_t^{HR} = (HP_{t+1} - HP_t)/HP_t$, Where HP_t represents the high price on day t for a related instrument (dollar index).

$X_t^{LR} = (LP_{t+1} - LP_t)/LP_t$, Where LP_t represents the low price on day t for a related instrument (dollar index).

Ideally, we would want to have more meaningful features, derived from the order book, maybe some information about the kind of day it is expected to be (high/low on economic data), any seasonality effects, option skew, implied —historical volatility spread, CFTC open interest positioning, daily volume, and so on. Here we are just discussing the structure; therefore, we'll keep it simple. Let us further define

$$V_{t-1} = [X_{t-1}, X_{t-1}^O, X_{t-1}^H, X_{t-1}^L, X_{t-1}^R, X_{t-1}^{OR}, X_{t-1}^{HR}, X_{t-1}^{LR}]$$

$$m = 8$$

Let us take $h = 32$, which is the number of neurons in the hidden layer that we have designated as MLP in the aforementioned diagrams. For appropriateness henceforth, we will call it RNN and not MLP.

Let us set this up as a classification problem

$$Y_t = 1, \text{ if } X_t > 0$$

$$Y_t = 0, \text{ if } X_t = 0$$

$$Y_t = -1, \text{ if } X_t < 0$$

We will be using Keras to implement this. There is another important hyper parameter that requires some explanation, which is the number of time steps. Number of time steps = n in our definitions so far.

If $n = 10$, our setting would look like figure 11.5

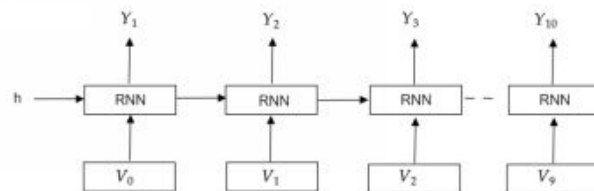


Figure 11. 5 10-time steps unfolded view of RNN

To reiterate what we are doing: we are taking V_0 and h_0 (which is initialized to 0), learning its relationship in h_1 via the RNN, and predicting Y_1 .

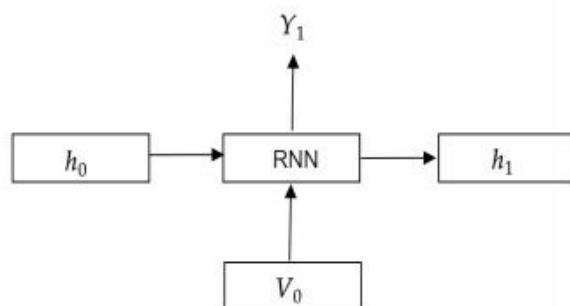


Figure 11. 6 Using the RNN to predict the output at the first-time step.

Next, we are taking these learnings h_1 and the next time steps inputs V_1 and passing it through the same network to predict Y_2 . We are also updating our understanding of the function that maps V_i to Y_i , that is, we have updated h_1 to h_2 with the aim of better understanding what predicts Y_0 and Y_1 .

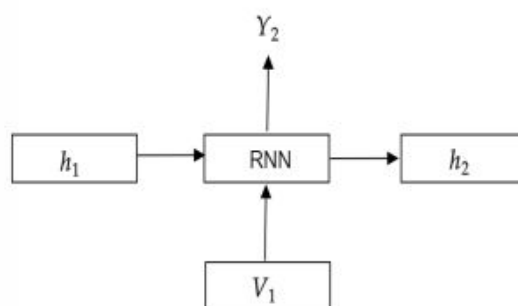


Figure 11. 7 Using the same RNN as in Figure 11.6 to predict the output at the second time step.

Now that we have a better understanding of what maps V_0 and V_1 to Y_1 and Y_2 , we move to the next time step, where we take h_2 and V_2 to get Y_3 . In addition, in the process we again update our understanding of the relation between V_i and Y_i . The updated relationship is captured in h_3 as shown in Figure 11.8.

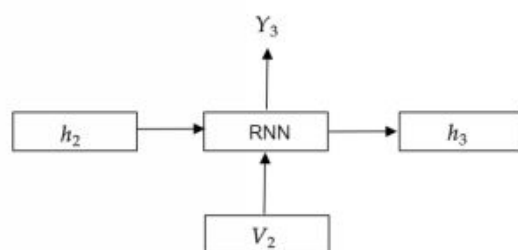


Figure 11. 8 Using the same RNN as in Figures 11.6 and 11.7 to predict third time step.

We repeat this process till we have reached $n = 10$ -time steps.

A few important considerations remain. How are we updating the weights and biases of the RNN; in other words, how are we doing the back propagation. Before we go there, let us first complete the setup, that is, how we present the data to a package such as Keras.

Keras Implementation: Setting up the features

In our raw data, the samples as of now look something like this:

Sample 1: $[V_0, Y_1]$

Sample 2: $[V_1, Y_2]$

Sample 3: $[V_2, Y_3]$

Sample 4: $[V_3, Y_4]$

...

Sample t: $[V_t, Y_{t+1}]$

However, the RNN needs to be presented $n = 10$ time steps at one time as a sample. Therefore, one sample of RNN would look like:

$[[V_0], [V_1], [V_2], [V_3], [V_4], [V_5], [V_6], [V_7], [V_8], [V_9]]$ to predict $[[Y_1], [Y_2], [Y_3], [Y_4], [Y_5], [Y_6], [Y_7], [Y_8], [Y_9], [Y_{10}]]$

Since each vector is V has eight elements, there is a 10×8 matrix, with 10 being the number of time steps. Therefore, one sample is represented as a matrix of dimension $[10, 8]$. Our entire batch will therefore be represented as a three-dimensional matrix of shape $[\text{batch size}, 10, 8]$. That is the shape the input takes for RNNs. A function that does that in python would look something like Fig 11.9:

```
def generator(V,Y,timestep=10):  
    samples = np.zeros((V.shape[0]-timestep,timestep,V.shape[-1]))  
    targets = np.zeros((Y.shape[0]-timestep,2))  
    for i in range(timestep,V.shape[0],1):  
        rows = np.arange(i-timestep,i)  
        samples[i-timestep]=V[rows]  
        targets[i-timestep]=Y[i]  
    return samples,targets
```

Figure 11. 9 Keras code for reorganising the input before presenting it to RNN

Where V is of shape [batch size, 8]. In addition, *samples* (which are returned by the *generator* function) are of dimension [batch size, 10, 8].

Keras Implementation : Setting up the target Y

There are a few ways we can set it up.

Many to one

Here we are only interested in predicting the output Y_{10} , as the previous predictions are already known. In this case, the dimension of Y would be [batch size, 3]. We have set the $-1,0,1$ entry as one hot encoded format and assumed the activation function in the output neuron is softmax. Figure 11.10 is an illustration of the many to one concept.

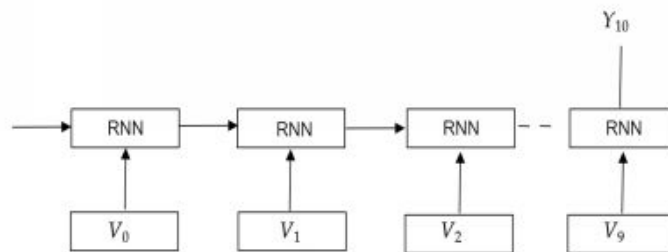


Figure 11. 10 Many to one.

Many to many

We are interested in obtaining the output at various time steps, in which case the dimension of the output would be [batch size, time steps, 3]. Figure 11.11 illustrates the many to many setup of target.

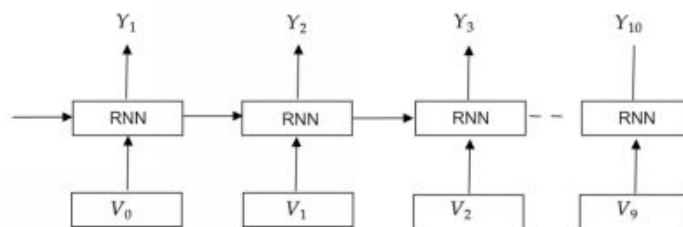


Figure 11. 11 Many to many.

Which of the two we desire is determined by the parameter **`return_sequence in Keras`**. If set to `TRUE`, we get output at various time

steps, else we get output at the last time step. The Keras code for setting up the RNN is given in figure 11.12

```
#x_train , y_train , x_valid , y_valid , x_test , y_test contain our raw data
data_dim =x_train.shape[1] #8
timestep=10
x_train,y_train= generator(x_train,y_train,timestep)
x_val,y_val=generator(x_valid,y_valid,timestep)
x_test,y_test=generator(x_test,y_test,timestep)
model = Sequential()
model.add(SimpleRNN(32,input_shape=(timestep, data_dim),return_sequences=True))
model.add(SimpleRNN (16,return_sequences=True))
model.add(SimpleRNN (8))
model.add(Dense(num_classes, activation='softmax'))
model.compile(loss='categorical_crossentropy',optimizer='rmsprop',metrics=['accuracy'])
model.fit(x_train, y_train,batch_size=64, epochs=5,validation_data=(x_val, y_val))
```

Figure 11. 12 Keras implementation of RNN

Now we will discuss how back propagation happens in RNNs.

Back propagation through time

We simply do the forward propagation and get the prediction for each sample at each time step. We calculate the loss at each time step and sum it over all the time steps. We then sum it over all the samples, and it is this aggregated loss across all the time steps and all the samples (of the batch) that is back propagated, essentially using the chain rule of differentiation. Here we may encounter the problem of vanishing or exploding gradients. Since this loss has to be back propagated across all time steps, it is often known as back propagation through time. In addition, if the number of time steps is quite large, the learnings of the early stages of the time series may be difficult to implement as the gradients may vanish (approach zero) or explode (become very large).

We look to solve this problem using a slight modification to the RNN architecture. This is where the GRU (Gated Recurrent Units) and LSTM (Long Short-Term memory) come into play.

For each cell of the GRU, we need to specify the memory size. It is this memory that's retained and passed through all the time steps. Each time step can read from this memory and make necessary predictions. Each time step is also allowed to write or update all or parts of this memory. That way any relevant information coming at any time step can be written to this so-called memory bus. Going forward, this information is available for use at any future time step. It can also be over written.

Let us look at how this is done. The general architecture looks like this: Across various time steps we have a memory bus running. The RNN can read data from this memory bus to make predictions and, based on new information, can overwrite the memory bus too.

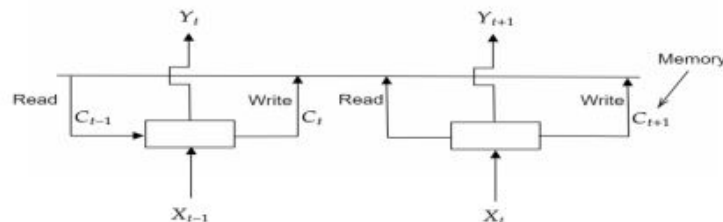


Figure 11. 13 GRU

The equations that enable us to do this are discussed next.

$$A_t = \tanh(W_A [C^{(t-1)}, X_{(t-1)}] + b_A)$$

The proposed activation function is tanH. In addition, all that the aforementioned equation is stating is that the activation A_t is calculated based on what the memory has stored so far $C^{(t-1)}$, and the input $X_{(t-1)}$.

We define a new operation, which is called the “update gate”. The justification for this name will become clear in a moment.

$$G_U = \text{Sigmoid}(W_U [C^{(t-1)}, X_{(t-1)}] + b_U)$$

Since G_U is computed using an activation of sigmoid its value is between 0 and 1. It is used to update the memory as follows:

$$C^{(t)} = G_U * A_t + (1 - G_U) * C^{(t-1)}$$

If $G_U = 0$, $C(t) = C(t-1)$ hence memory is not updated.

If $G_U = 1$, $C^{(t)} = A_t$, hence memory is overwritten by A_t .

To summarize the equations that describe the behaviour of GRU is

$$A_t = \tanh(W_A [C^{(t-1)}, X_{(t-1)}] + b_A)$$

$$G_U = \text{Sigmoid}(W_U [C^{(t-1)}, X_{(t-1)}] + b_U)$$

$$C^{(t)} = G_U * A_t + (1 - G_U) * C^{(t-1)}$$

$$A_t = C^{(t)}$$

These equations may seem overbearing but the implementation in Keras is relatively straightforward.

LSTM

The concept behind the LSTM is very similar, it just has more gates. For one, the LSTM differentiates between the A_t and $C^{(t)}$.

It defines the output gate that bridges the gap between A_t and $C^{(t)}$.

$$G_O = \text{Sigmoid} (W_O[C^{(t-1)}, X_{(t-1)}] + b_O)$$

$$A_t = G_O * \tanh(C^t)$$

LSTM retains the G_U , which is the update gate. The role of $(1 - G_U)$ or the forget gate is modelled separately as

$$G_F = \text{Sigmoid} (W_F[C^{(t-1)}, X_{(t-1)}] + b_F)$$

Therefore, the update of the memory is written as

$$C^{(t)} = G_U * A_t + (G_F) * C^{(t-1)}$$

The equations that describe the behaviour of LSTM is as follows:

$$A_t = \tanh (W_A[C^{(t-1)}, X_{(t-1)}] + b_A)$$

$$G_U = \text{Sigmoid} (W_U[C^{(t-1)}, X_{(t-1)}] + b_U)$$

$$G_F = \text{Sigmoid} (W_F[C^{(t-1)}, X_{(t-1)}] + b_F)$$

$$G_O = \text{Sigmoid} (W_O[C^{(t-1)}, X_{(t-1)}] + b_O)$$

$$C^{(t)} = G_U * A_t + (G_F) * C^{(t-1)}$$

$$A_t = G_O * \tanh(C^{(t)})$$

All these equations and other points may seem overwhelming at first, but if you understand why we replace RNN with GRUs and LSTMs, it is a pretty decent start. Most APIs (Application Programming Interface) take care of this

rigor for us and, to be totally honest, I hardly bother about the equations myself.

Let us take a simple example. A snapshot of our input looks like Figure 11.14:

Date	Open	High	Low	Close	Target
19950112	-2.05	-4.9	-2.5	-5.25	0
19950113	-3.8	0.85	-1.8	2.3	1
19950116	0.9	1.45	1.5	1.2	1
19950117	1.75	-1.2	-0.3	-2.85	0
19950118	-2.45	-4.25	-2.5	-1.8	0
19950119	-2	1.35	-1.7	-1.35	0
19950120	-1	-1.4	-1.4	-0.45	0
19950123	-2.4	-0.95	3.2	2.7	1
19950124	4	3.45	-0.9	-0.25	0
19950125	0.2	0.2	0.1	0.65	1
19950126	0.6	0.5	-0.05	-0.4	0
19950127	-0.6	0.1	6.25	3.25	1
19950130	4.5	3.7	-1.2	1.25	0
19950131	0.3	0.6	1.3	1.9	1
19950201	1.7	0.7	0.4	-1.1	0
19950202	-1.2	0.2	-1.5	-0.05	0
19950203	-0.35	-0.4	0.3	-0.05	0
19950206	0.8	0.8	0.7	0.4	0
19950207	0.1	0.45	-0.2	1.1	1
19950208	0.95	-0.25	0.1	-1.2	0

Figure 11. 14 Sample input into the LSTM

We set time steps = 50 and number of features = 4. After passing through the data_generator function, we reshape our input as:

(Number of samples, 50, 4)

We set up the LSTM as follows, with a memory bus of size 32.

```
model = Sequential()
model.add(LSTM(32,input_shape=(timestep,data_dim),return_sequences=False))
model.add(Dense(num_classes, activation='softmax'))
print model.summary()
```

the model looks like this:

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 32)	4736
dense_1 (Dense)	(None, 2)	66
Total params: 4,802		
Trainable params: 4,802		
Non-trainable params: 0		

One LSTM layer with 32 memory size in this setup has 4736 parameters, which is the total number of weights and biases.

How do we get that?

$X_t = (\text{batch size}, 4)$

$h_{t-1} = (\text{batch size}, 32)$

Input size is therefore $= [X_t, h_{t-1}] = [\text{batch size}, 36]$

Output has dimension of $[\text{batch size}, 32]$.

Weight matrix $= 36 \times 32$.

Bias parameters $= 32$.

$W_A, W_C, W_U, W_O = 4 \times 36 \times 32$

$b_A, b_C, b_U, b_O = 4 \times 32$

which sums up to 4736.

Summary

In this chapter, we have learnt many new concepts like RNN, LSTM and GRU. With the help of an example we have explained how a recurrent neural network can be set up and what is the meaning of various parameters. These concepts help in learning temporal features effectively for automated trading.

References

- <http://deeplearning.cs.cmu.edu/>
- <https://www.deeplearningbook.org/>
- Lim, Bryan & Zohren, Stefan & Roberts, Stephen (2019). Enhancing Time Series Momentum Strategies Using Deep Neural Networks
- <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

CHAPTER TWELVE - K NEAREST NEIGHBOURS

In this chapter we will discuss the K Nearest Neighbour (KNN) algorithm that can be applied to classification and regression tasks. We will then discuss how this algorithm can be used as a 'timing indicator' in equity markets. As always, the focus will be on building the intuition rather than developing and selling a full-fledged trading system. Finally, we will draw insights about the equity market based on what our machine has learnt.

KNN Algorithm

To understand the KNN algorithm, we need to first understand Minkowski Distance. Figure 12.1 gives a snapshot of our data set.

date	FIN COND	ISM M PMI	ISM NM PMI	initial_jo	us_capaci	leiYOY	leiMoM	ISM Price	CPI Yoy It	FIN COND	ISM M PMI	ISM NM PMI	us_capacity	utilization_chg	ISM Price Index	chg
20050104	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050105	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050106	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050107	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050110	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050111	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050112	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050113	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050114	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050118	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050119	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050120	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050121	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050124	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050125	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050126	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050127	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050128	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		
20050131	-0.445539249	0.813521944	0.875564256	-0.55158	0.945943	1.108375	0.447824	0.704974	0.816033	0.545095	-0.20148	-0.84763	0.598112261	-0.427867938		

Figure 12. 1 Dataset under consideration

Let us pick up two samples x_a and x_b from this dataset as shown in Figure 12.2 with n samples and P features.

date	FIN COND	ISM M PMI	ISM NM PMI	initial_jo	us_capaci	leiYOY	leiMoM	ISM Price	CPI Yoy It	FIN COND	ISM M PMI	ISM NM PMI	us_capacity	utilization_chg	ISM Price Index	chg
20050104	-0.636220112	0.902541466	1.28295062	0.064378	0.846859	1.204355	1.270411	0.922653	0.577185	-0.7136	0.603154	0.665735	1.008790933	-0.275815762		

$|x_{a1} - x_{b1}|^{1/q}$

date	FIN COND	ISM M PMI	ISM NM PMI	initial_jo	us_capaci	leiYOY	leiMoM	ISM Price	CPI Yoy It	FIN COND	ISM M PMI	ISM NM PMI	us_capacity	utilization_chg	ISM Price Index	chg
20050131	-0.445539249	0.813521944	0.875564256	-0.55158	0.945943	1.108375	0.447824	0.704974	0.816033	0.545095	-0.20148	-0.84763	0.598112261	-0.427867938		

Figure 12. 2 x^a and x^b sampled from our dataset

The minkowski distance is defined as in Equation 12.1 below

$$(\sum_{j=1}^P |x_{aj} - x_{bj}|^q)^{1/q} \quad \text{Equation 12.1}$$

When $q = 1$, it is known as the Manhattan distance and when $q = 2$ we call it the Euclidean distance. It is therefore a hyper parameter the user needs to specify.

All that we do in the KNN algorithm is that, for a test sample, we calculate the Minkowski distance to each sample of the training data and select the k samples that are the closest to this test sample. Closeness, of course, is

measured by the Minkowski distance. It is these k samples that form the nearest neighbours.

The **prediction** is simply an aggregation or a vote of the target values of these k nearest neighbours. k itself is a hyper parameter and is found through the process of cross validation. We may also choose to weigh the target value of the k nearest neighbours by the distance to the test sample for purposes of prediction. Finally, there are two important considerations:

- Since our distance measure is highly dependent on the scale of each feature, it is highly recommended to apply the standard scalars on the input features. Any of the standard scalars discussed in Chapter 2 could be used.
- Computation can be slow. For each test sample we need to find the Minkowski distance to each sample of the training data set from which we then select the k nearest neighbours. This can be quite time consuming, and several modifications have been suggested. A k – dimensional tree or a k – d tree is one such modification, wherein in Minkowski, distance is computed only with a select number of samples. We refer the readers to the references for further reading.

Without further ado, let us jump into some exciting applications.

Financial Application : macro timing of various equity styles

We will try and uncover patterns in macro-economic variable that help us position for moves in various equity markets. The macro economic variables that we have considered are shown in Figure 12.3. These may at best be representative of various regimes but by no means are exhaustive. These variables will go as inputs into our model and help us identify various regimes. Based on these regimes, we hope to be able to decide on an optimal investments style or strategy. In common parlance these macro-economic variables are the features to our model.

Macro Economic Variables		
FIN COND		ISM Price Index
ISM M PMI		CPI Yoy lfe
ISM NM PMI		FIN COND_chg
initial_jobless_claims		ISM M PMI_chg
us_capacity_utiliozation		ISM NM PMI_chg
leiYOY		us_capacity_utiliozation_chg
leiMoM		ISM Price Index_chg

Figure 12. 3 Macro economic variables, which go as inputs into our models.

Data Set Targets

As targets we create Indices for various equity investment styles. We essentially take US listed stocks and bucket them into various sectors in accordance with the Global Industry Classification Standard (GICS) Figure 12.4.

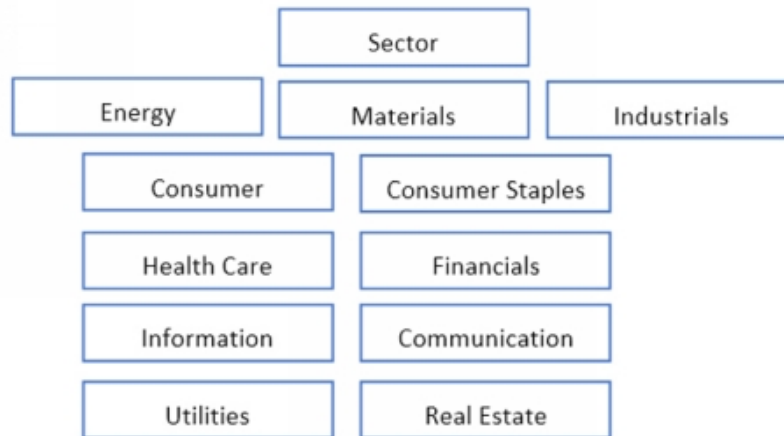


Figure 12. 4 <https://www.msci.com/gics>

Stocks in each sector bucket are further split into four sub buckets based on fundamental and technical information of the stock.

The four sub buckets applied to each sector and their respective definitions are

1. Value :

Stocks are ranked according to the three criteria mentioned below. Each criterion is given equal weight and we look to buy stocks in the top 10 percentile. The portfolio is rebalanced monthly.

- Operating cash flow per share trailing 12 months
- Dividends that are expected to be paid in the next 12 months

- c. Sales per share trailing 12 months divided by enterprise value per share

2. Momentum:

Stocks are ranked according to the below score and we buy the top 10 percentile. The portfolio is rebalanced monthly

- a. $(12 \text{ month return} - 1 \text{ month return}) / 1 \text{ month standard deviation}$

3. Growth

Stocks are ranked according to the below 2 criteria. Each criterion is given equal weight and we look to buy stocks in the top 10 percentile. The portfolio is rebalanced monthly.

- a. Percentage change in earnings per share trailing 12 months over trailing 12 months
- b. Percentage change in sales trailing 12 months over trailing 12 months

4. Quality

Stocks are ranked according to the criteria below. Each criterion is given equal weight and we look to buy stocks in the top 10 percentile. The portfolio is rebalanced monthly.

- a. Gross Margin
- b. Standard deviation of earnings per share, lower the better

Effectively we have an index for each sector and each style. The Indices we have created and used in this example are as shown in Figure 12.5. These indices will serve the purpose of targets.

TARGETS			
ConsumerDiscretionaryValueLong	TransportMomLong	TechnologyMomLong	IndustrialQualityLong
ConsumerDiscretionaryQualityLong	TransportGrowthLong	TechnologyValueLong	IndustrialMomLong
ConsumerDiscretionaryMomLong	UtilValueLong	TechnologyGrowthLong	FinancialsValueLong
ConsumerDiscretionaryGrowthLong	UtilQualityLong	TechnologyQualityLong	FinancialsGrowthLong
HealthCareValueLong	UtilMomLong	MaterialsValueLong	FinancialsQualityLong
HealthCareQualityLong	UtilGrowthLong	MaterialsGrowthLong	FinancialsMomLong
HealthCareMomLong	ConsumerStaplesValueLong	MaterialsQualityLong	EnergyValueLong
HealthCareGrowthLong	ConsumerStaplesQualityLong	MaterialsMomLong	EnergyGrowthLong
TransportValueLong	ConsumerStaplesMomLong	IndustrialValueLong	EnergyQualityLong
TransportQualityLong	ConsumerStaplesGrowthLong	IndustrialGrowthLong	EnergyMomLong

Figure 12. 5 The targets of our data set.

Just so that we do not lose sight of the objective, we are going to build a machine that takes macro-economic variables as the input and predicts the performance for each style.

Training, Validation and Test data sets

Our complete data set is from 2005 to 2020.

Jan 2005 – Jul 2017 Training Data

Aug, 2017 – Feb 2019 Validation Data

Mar, 2019 – Mar 2020 Test Data

Ideally, we should have used a more rigorous validation strategy like the ones we have discussed earlier in this book, but the idea here is not to design a robust trading system, but rather to understand how it can be done.

1. The python code that helps us do the split and pre-process the data is in Figure 12.6.
2. For a particular value of the hyper parameter 'param' (which is essentially the 'k' nearest neighbours), we fit the KNN object as in Figure 12.7 and make the prediction for the validation data sets, which essentially means that, for a given set of macro-economic conditions, we are able to predict the returns for each strategy.
3. We then proceed to rank the performance of each sample in our validation datasets (Figure 12.8), go long the best 10 strategies and short the worst 10 strategies and calculate the profit and loss of our portfolio.

4. Steps 1), 2) and 3) are repeated for various values of param and we select that with the highest profit and loss.

```
df_train = df[['date']+strategy_vectors+macro_vector].iloc[1:3200,:]  
# Validation data set  
df_test = df[['date']+strategy_vectors+macro_vector].iloc[3201:3600,:]  
# Test data set  
df_testf= df[['date']+strategy_vectors+macro_vector].iloc[3601,:]  
  
# Macro_vectors is list of input features  
# S{strategy_vectors is list of targets  
  
X_train = df_train[macro_vector]  
X_test= df_test[macro_vector]  
Y_train = df_train[strategy_vectors]  
Y_test = df_test[strategy_vectors]  
scaler=StandardScaler().fit(X_train)  
X_train=scaler.transform(X_train)
```

Figure 12. 6 Python code to split and pre-process the data

```
from sklearn.neighbors import NearestNeighbors  
  
param_list=  
[5,10,20,50,100,150,200,250,300,350,400,450,500,550,600,700,800,850,900,1000,1200,1500,1600,1  
700,2000,2100,2200,2300,2400,2500,2600,2700,2800]  
  
for param in param_list:  
    NN = NearestNeighbors(n_neighbors=param, metric='minkowski',n_jobs=1,algorithm='kd_tree')  
    NN.fit(X_train)  
    NN_output = NN.kneighbors(X_test)  
  
    output = []  
  
    for i in range(X_test.shape[0]):  
        neighs= NN_output[1][i]  
        output.append(Y_train.iloc[neighs,:].mean().values/(Y_train.iloc[neighs,:].std().values))
```

Figure 12. 7 python code to fit and predict validation data

```

output_rank = output.rank(1,ascending = False,method = 'first')
rankcols = []

for f in strategy_vectors:
    rankcols.append(f+"_rank")
    output_rank.columns = rankcols

for i in (output_rank.columns):
    output_rank[i] = np.where(output_rank[i]<=10,1,np.where(output_rank[i]>=31,-1,0))

```

Figure 12. 8 python code to rank and calculate profit and loss

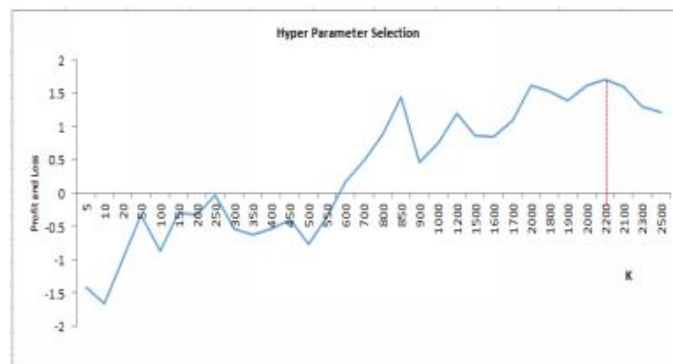


Figure 12. 9 Hyper parameter selection.

From Figure 12.9 it is clear that $k = 2200$ is the value of the hyper parameter we want to choose. Before we move on to discussing the performance on test data sets, let's highlight some clear weaknesses that you would want to account for in case you are building a robust strategy.

Pitfalls in the current strategy

1. Not accounted for slippages, broker commissions, turnover costs and, most importantly, cost of shorting stocks or for that matter even the ability to short stocks, given various regulatory constraints around it.
2. Some data vendors only provide revised economic data time series , thereby introducing forward bias.
3. Macro-economic variables used to predict regimes are quite restrictive. Some obvious additions could be yield curve slope, oil prices, gold prices, state of the credit market, loan surveys, freight indices performance of Non US markets etc.
4. The validation strategy is poor and needs to be more in line with what we have discussed in earlier chapters.
5. Personally we feel for this type of a strategy we need way more data to build a robust strategy. Ideally from 1970's onwards, if not more.

Test Strategy performance

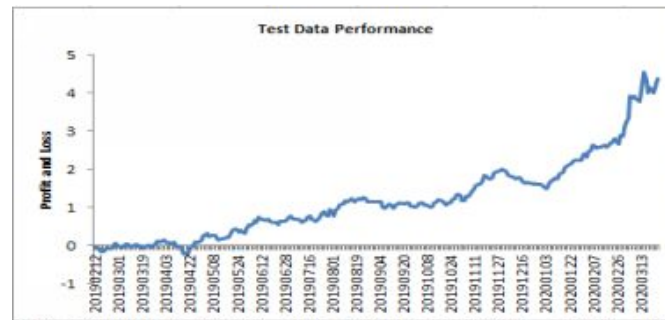


Figure 12. 10 Hypothetical test data set performance.

Figure 12.10 gives the equity curve of the strategy's performance during the test period. Figure 12.11 gives the portfolio for the month of March 2020. It is somewhat interesting to note the short position in energy and consumer discretionary and the long position in health care and consumer staples.

Strategy	Position
ConsumerDiscretionaryValueLong	SHORT
ConsumerDiscretionaryQualityLong	SHORT
ConsumerDiscretionaryGrowthLong	SHORT
TransportValueLong	SHORT
TransportQualityLong	SHORT
TransportGrowthLong	SHORT
EnergyValueLong	SHORT
EnergyGrowthLong	SHORT
EnergyQualityLong	SHORT
EnergyMomLong	SHORT
HealthCareValueLong	LONG
HealthCareQualityLong	LONG
HealthCareMomLong	LONG
ConsumerStaplesValueLong	LONG
TechnologyValueLong	LONG
TechnologyQualityLong	LONG
MaterialsValueLong	LONG
IndustrialGrowthLong	LONG
IndustrialMomLong	LONG
FinancialsMomLong	LONG

Figure 12. 11 Strategy composition in March 2020.

There is a lot of information that can be learnt from this exercise, so let us dig in.

Optimal portfolio for a particular macro-economic environment

It may happen that we may be looking to express a macro view via the indices we have created. For instance, we would like to know what the best portfolio is when financial conditions are really tight (e.g. have a score of 2.5, after applying standard scaler). Alternatively, we may feel the economy may overheat and we may enter an inflationary environment, or we may enter an environment of stagflation (high inflation coupled with low growth). How do stocks perform in this environment? Which sectors outperform? Which styles outperform? How does each style do within each sector? All these are legitimate questions and the current model, if created well, is in a position to answer these questions.

All we need to do is populate our macro-economic vector with values that reflect the macroeconomic condition that we are trying to model. So, if we are trying to model tight financial conditions, our vector would look like Figure 12.12. We call this macro-economic vector the scenario vector (Figure 12.12).

'FIN COND'	'ISM M PMI'	'ISM NM PMI'	'initial_jobless_claims'	'us_capacity_utilization'	'leiYOY'	'leiMoM'	'ISM Price Index'	'CPI YOY life'	'FIN COND_chg'	'ISM M PMI_chg'	'ISM NM PMI_chg'	'us_capacity_utilization_chg'	'ISM Price Index_chg'
3	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 12. 12 Scenario Vector: Values to be fed into the macroeconomic vector to reflect tight financial conditions.

This scenario vector is now like any other vector from our test data set. We need to find the k ($=2200$ as deciphered earlier) nearest neighbours from our training data set. Once we have done that, we would know the performance of each strategy when these macro-economic conditions are met. Figure 12.13 lists the best and worst performing strategy, which make a lot of economic sense. The one concern though is that growth is being identified among the worst performing strategies, which is a bit counter intuitive. The code that does this, is in Figure 12.14

Best Performing Strategies	Worst Performing Strategies
<i>ConsumerStaplesValueLong</i>	<i>EnergyGrowthLong</i>
<i>UtilValueLong</i>	<i>TransportGrowthLong</i>
<i>TechnologyQualityLong</i>	<i>MaterialsGrowthLong</i>
<i>HealthCareValueLong</i>	<i>ConsumerDiscretionaryQualityLong</i>
<i>UtilMomLong</i>	<i>ConsumerDiscretionaryGrowthLong</i>

Figure 12. 13 Portfolio recommendation for scenarios in Figure 12.12

```

X_test= pd.read_csv("/scenarios.csv")
NN = NearestNeighbors(n_neighbors=2200, metric='minkowski',n_jobs=1,algorithm='kd_tree')
NN.fit(X_train)
NN_output = NN.kneighbors(X_test)
for i in range(X_test.shape[0]):
    neighs= NN_output[1][i]
    output.append(Y_train.iloc[neighs,:].mean().values/(Y_train.iloc[neighs,:].std().values))

```

Figure 12. 14 Code to find optimal portfolio for a given scenario

Optimal macro environment for a particular strategy

Another logical inference that we may want to draw, is which macro-economic variable is the biggest driver of a particular strategy. Let us say the strategy of interest is ConsumerDiscretionaryValueLongvs. ConsumerStaplesValueLong.

The way we approach this problem is slightly different to anything we have seen thus far. We draw motivation from the study of feature importance. So far, we are quite comfortable in predicting the expected performance of a strategy given a certain set of macro-economic conditions. This is step one of what we need to do, and we make a note of the expected profit and loss.

Next we replace one of the macroeconomic variables with random numbers (from a standard normal distribution since we have scalarized the input). We re-compute the expected profit and loss and note the change to what was calculated in step one. If the profit and loss now is way lower than what was calculated in step one, this particular macroeconomic variable is important in predicting the strategies return. This process is repeated for all the macroeconomic variables, which can then be ranked based on the difference in profit and loss when compared to what was calculated in strategy one.

Step 1: The index created by going long ConsumerDiscretionaryValueLong and short ConsumerStaplesValueLong based on the macro economic variables appreciates 25% per annum in the training data. The code that does it is shown in Figure 12.15. All that we are doing is, if the expected return is positive, we are long the portfolio (long ConsumerDiscretionaryValueLong and short ConsumerStaplesValueLong). If the expected return is negative, we are short the portfolio (short ConsumerDiscretionaryValueLong and long ConsumerStaplesValueLong)

```
NN =NearestNeighbors(n_neighbors=2200, metric='minkowski',n_jobs=1,algorithm='kd_tree')
NN.fit(X_train)
NN_output = NN.kneighbors(X_test)
output = []
for i in range(X_test.shape[0]):
    neighs= NN_output[1][i]
```

Figure 12. 15 Calculate PnL of the portfolio.

Step 2: We replace each macroeconomic variable with random numbers (sampled from a standard normal distribution) one at a time and recalculate the profit and loss as done in step 1. The lower this PnL, the greater the importance of this macroeconomic variable. Figure 12.16 lists what turn out to be the most important features impacting the portfolio.

<i>Most Important Features</i>
<i>FIN COND</i>
<i>ISM NM PMI</i>
<i>us_capacity_utiliozation</i>

Figure 12. 16 Most important features for ConsumerDiscretionaryValueLongvsConsumerStaplesValueLong.

Figure 12.17 shows the positioning of the portfolio in 2008, wherein for most part it was long ConsumerStaples and short ConsumerDiscretionary.

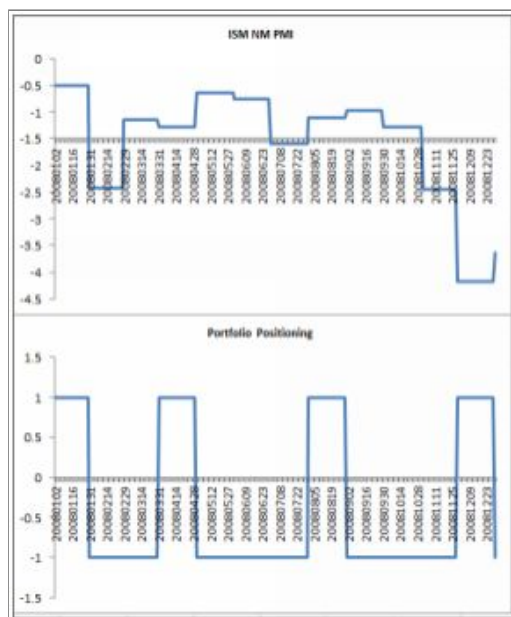


Figure 12. 17 Positioning of the portfolio.

We make loads of other discoveries by analysing this model, which can offer numerous opportunities to us.

- Which strategy performs the best 2 months after a recession?

- Does this rally have the characteristics of a bull market or is it a bear market squeeze higher?
- What is happening in the macroeconomic environment when there is a sudden and drastic change in the rank of one of these strategies?
- How much capital is expected to flow into a particular strategy in a given economic environment? How much has flown in currently and what are the risks and opportunities?
- What is the mean, median, maximum and minimum performance of a strategy in a given macro-economic environment? How much has it performed so far? What are the opportunities and risks?
- Why has value suddenly stopped performing? Why is growth outperforming? What are the major long term trends that seemed over stretched and are beginning to falter? What has happened earlier when such trends have faltered?
- What happens when our model breaks down, i.e. what happens when our macro-economic indicators are pointing south and the market is heading north?

The list is endless and open to a lot of research with a whole lot of findings.

That's where we conclude the chapter and the book.

Some parting thoughts from our trials and tribulations

- Data quality is of paramount importance. The number of vendors providing usable high quality data can be counted in one hand. We have yet to come across a vendor who provides high quality data for all frequencies. Data access is expensive. We have found errors in data provided by some of the top vendors. We repeat: Data is expensive, and if you can get it directly from the source, that's simply the best option. We cannot stress this enough.
- The cloud is fantastic, but again its expensive and realistically you will need a whole lot of computing power to conduct meaningful experiments.

- Pick an economic theory and use the AI to model, improve, understand that.
- Validation strategy is a very important part of the problem.
- Rule of thumb, if you have to run too much iteration to get a problem to work, you are doing something wrong or fishing in a pond with no fish.
- Focus ONLY on improving the quality of your prediction, not on the strategies' ability to make money. The latter should be a natural result of the former. If you take the risk management away from your trading strategy and your back test loses money, you have overfit to your historical data and your signal quality is weak.
- A lot of data helps. Select your problems accordingly.
- Each and every decision that you make along the way should be backed by statistical evidence in favour of the path you intend to take. All decisions are important.
- Have a plan on tracking how and when your model is breaking down and how and when should you recalibrate.
- The error in your prediction has vital information.

References

- Bentley J (1975).“Multidimensional Binary Search Trees Used for Associative Searching.”
- <https://www.msci.com/gics>